

机器视觉-实践项目

国嘉通*

2025 年 11 月 4 日

摘要

机器视觉实践项目的前三章主要包含三大部分的内容，一是实验环境的搭建，二是基础的图像操作，三是 $\text{\LaTeX}$ 的基本使用操作。对于实验环境的基本搭建部分，从基于ANACONDA的Windows系统及基于Homebrew的Mac系统的基础环境的搭建两方面分别展开。对于基础图像操作部分，将分多个部分进行展开讲解。

第四章-第六章主要包含三大部分的内容，一是灰度变换基础，二是空域平滑滤波，三是空域锐化滤波。对于灰度变换基础部分，首先讲解如何使用OpenCV读取图像并显示其灰度直方图；在此基础上，还会讲解如何实现幂律（伽马）变换，并通过尝试不同的伽马值观察其对整体亮度与对比度的调控作用；进一步地，对灰度直方图进行均衡化并应用于低对比图像。对于空域平滑滤波部分，主要陈述了如何对图像进行加噪，并实现均值滤波、高斯滤波、中值滤波，讨论其对抑制高斯噪声和椒盐噪声方面的效果差异。对于空域锐化滤波部分，主要讲解如何进行拉普拉斯算子锐化操作，并实现高提升滤波，讨论锐化与噪声之间的平衡关系。

第七章与第八章主要包含两大部分的内容，一是频域平滑图像，二是频率锐化图像。对于频域平滑图像部分，分别从理想低通频域滤波、高斯低通频域滤波、巴特沃斯低通频域滤波三个方面进行详细阐释。对于频率锐化图像部分，分别从理想高通频域滤波、高斯高通频域滤波及巴特沃斯高通频率滤波。

第九章至第十一章主要包含两大部分的内容，一是图像形态学处理，二是边缘检测处理。对于图像形态学处理部分，分别从腐蚀、膨胀、开运算、闭运算、骨架提取五个方面进行详细阐释。对于边缘检测处理部分，分别从Sobel 算子与 Canny 算子两个角度进行阐释。另外，还介绍了K-means与Mean Shift两种图像分割方法。

第十二章至第十三章主要包含两大部分的练习，一是张正有标定法，二是SIFT算法。以十张照片为例，使用张正有标定法实现对图片的畸形矫正。对于SIFT算法部分，分别从两个不同的角度拍摄相同物品的照片，调用SIFT算法实现匹配。

希望本篇文章是一篇内容翔实、方便随时翻阅复习学习OpenCV的学习笔记。文章完成较为仓促，有许多细节作者无法在文中一一落实到位，敬请谅解。

长沙理工大学

卓越工程师学院 绿色智慧交通 2401班

202404080608

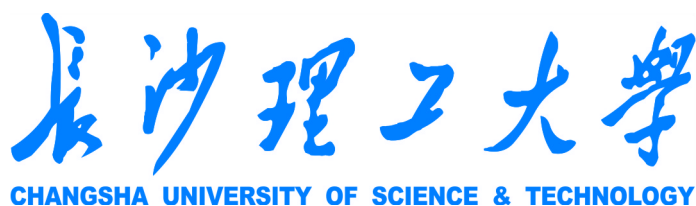
国嘉通

*Tel.+86-166 8130 6085, E-mail:guojiatom2006@outlook.com

目录

1 Python环境配置	6
1.1 Windows系统情况下的相关配置	6
1.2 Mac系统情况下的相关配置	7
1.3 配置完成后的pip list截图(部分)	8
2 OpenCV基础图像操作	10
2.1 读取与显示图像	10
2.2 图片属性输出	12
2.3 图片灰度转换操作	13
2.4 图像变换操作	15
2.4.1 图像缩放操作	15
2.4.2 图像旋转操作	16
2.4.3 图像剪裁操作	19
2.5 图像运算操作	20
2.5.1 加法运算	20
2.5.2 减法运算	23
2.5.3 乘法运算	25
2.5.4 除法运算	28
2.6 逻辑运算	30
3 L^AT_EX使用基础命令	34
3.1 代码基本结构	34
3.2 单双栏版式变换	35
3.3 公式插入	36
3.3.1 普通公式	36
3.3.2 矩阵	39
3.3.3 转义符号	40
3.4 枚举	40
3.5 图片插入	41
3.5.1 单图插入	41
3.5.2 多图插入	41
3.6 表格插入	42
3.7 文本框插入	43
3.7.1 基于\fbbox简单文本框	43
3.7.2 基于\tcolorbox复杂文本框	44
3.8 引用插入	44

3.8.1	正文引用	44
3.8.2	文献引用	44
4	灰度变换基础	46
4.1	直方图显示	46
4.2	幂律(伽马)变换	48
4.3	灰度直方图均衡化	50
5	空域平滑滤波	54
5.1	图像加噪	54
5.2	图像滤波	58
6	空域锐化滤波	64
6.1	拉普拉斯算子锐化操作	64
6.2	高提升滤波操作	66
6.3	锐化与噪声的权衡讨论	68
7	频域平滑图像操作	73
7.1	理想低通频率滤波操作	73
7.2	高斯低通频率滤波操作	78
7.3	巴特沃斯低通频率滤波操作	85
8	频率锐化图像操作	90
8.1	理想高通频域滤波操作	90
8.2	高斯高通频域滤波	93
8.3	巴特沃斯高通频率滤波	96
9	图像形态学处理	101
9.1	腐蚀运算(Erosion)	101
9.2	膨胀运算(Dilation)	102
9.3	开运算(Opening)	104
9.4	闭运算(Closing)	105
9.5	骨架提取(Skeletonization)	107
10	边缘检测操作	111
11	图像分割操作	115
12	张正有标定法	121

13 SIFT算法**131**

Now the CODE is available at:

<https://github.com/GplasT0810/Machine-Vision-Experiment.git>

Completed Time: Sep.15th.2025

Guo Jiatong , Green & Intelligent Transportation 2401
Elite Engineering School , Changsha University of Science & Technology.
Changsha 410114 , Hunan , China

|| 第一章

Python环境配置

1 Python环境配置

1.1 Windows系统情况下的相关配置

使用Anaconda 作为Python内核，搭配VSCode作为编辑器可完成相关设置。具体步骤如下：

1. 在Anaconda Prompt中搭建环境，指令为“conda create -n 环境名 python=版本号”。
例如，欲想搭建一个名为myenv的Python版本为3.11.11的环境，我们可以输入以下指令：

```
conda create -n myenv python=3.11.11
```

2. 搭建完虚拟环境后，要将其激活。

激活指令为“conda activate 环境名”。假设已经以“myenv”搭建好环境，我们可以输入以下指令以进入此虚拟环境。（若不进行此步骤，后续安装包的过程中则会直接安装在base基环境下）

```
conda activate myenv
```

3. 完成环境激活后，则可以安装相应的软件包。

一些较为常见的软件包如下表1所示：

Package	Package	Package	Package
absl-py	cachetools	certifi	charset-normalizer
cloudpickle	colorama	contourpy	cycler
et-xmlfile	filelock	fonttools	google-auth
google-auth-oauthlib	grpcio	gym	gym-notices
idna	Jinja2	kiwisolver	Markdown
MarkupSafe	matplotlib	mpmath	networkx
numpy	oauthlib	opencv-python	openpyxl
packaging	pandas	pathlib	pillow
pip	protobuf	pyasn1	pyasn1 _ modules
pyparsing	jupyter notebook	pytz	PyYAML
requests	requests-oauthlib	rsa	scipy
seaborn	setuptools	six	sympy
tensorboard	tensorboardX	tk	torch
torchaudio	torchvision	tqdm	tzdata
urllib3	opencv-python	wheel	xlrd

可以使用“pip install Package”指令安装相关软件包，其中Package为表1中的软件包或之外的软件包。假设欲安装numpy软件包，我们可以输入如下指令：

```
pip install numpy
```

注意!

上述指令可能安装最新版本的对应的软件包或者是最适配当前Python版本的软件包。如果想要安装指定版本的软件包，应当使用“pip install `Package`==`Version`”指令。以安装2.2.6版本的numpy包为例：

```
pip install numpy==2.2.6
```

4. 其余的部分有用指令见下表2.

指令	代码
删除环境中的包	pip uninstall <code>Package</code>
删除环境	conda remove -n <code>Env.name</code>
退出虚拟环境（回到base）	detective
查看虚拟环境中的包库	pip list
查看Env列表（可以用于改环境名）	conda env list

1.2 Mac系统情况下的相关配置

对于Mac系统，我们可以直接使用系统带有的终端进行配置，而不需要安装Anaconda。不过我们需要在终端中安装Homebrew，此插件源为Github，有时内网可能不灵，解决方案是多试几次或者科学上网（可以使用Infiniport）。

1. 首先可以使用指令来检测Mac是否自带Python。指令如下：

```
python -version (对于Mac自带Python版本为2.x)
python3 -version (对于大部分Mac Python版本为3.x)
```

2. 若无预置Python，则需要通过Homebrew包管理器来安装最新的Python版本。如果还没有安装Homebrew，应先安装它。（源在Github，或许需要科学上网> > [infiniport.xyz](https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)）
在终端中输入：

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

3. Homebrew安装完成后，使用其安装Python。指令为：

```
brew install python
```

4. 为了避免不同项目之间的冲突，应搭建虚拟环境，并在虚拟环境中搭建对应的包。此步骤大体包含创建虚拟环境、激活虚拟环境、在虚拟环境中安装包、退出虚拟环境四部分。
首先，创建虚拟环境。以搭建环境名为“myenv”的Python环境为例，其代码如下所示。

```
python3 -m venv myenv
```

然后，激活虚拟环境，代码如下所示。

```
source myenv/bin/activate
```

在所创建的虚拟环境中安装对应的软件包，以安装numpy包为例，代码如下所示。

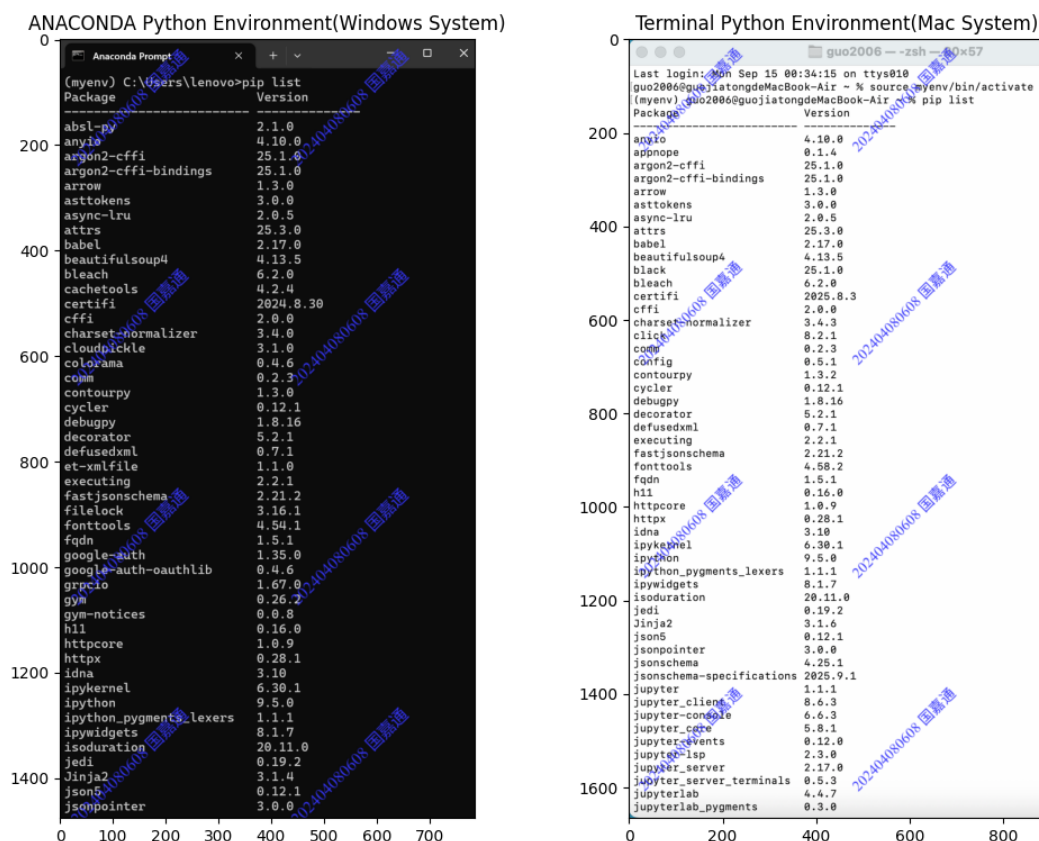
```
pip install numpy
```

最后，所有上述步骤完成后，可以使用deactivate指令退出虚拟环境，并在IDE中进行配置，不再过多赘述。

1.3 配置完成后的pip list截图(部分)

最终，在Windows系统下的Anaconda中配置的Python环境所安装的软件包(部分)以及Mac系统下的终端-Homebrew中配置的Python环境所安装的软件包(部分)如下图所示。

可以在激活对应环境后键入“pip list”以显示它。



|| 第二章

OpenCV基础图像操作

2 OpenCV基础图像操作

OpenCV的基础操作包含读取→显示→属性输出→转换→变换→运算→解码等基本操作。本大节利用Jupyter Notebook分别书写练习了相关语法。下面将一一展开进行叙述。

2.1 读取与显示图像

本小节主要展示如何使用opencv-python库以及matplotlib库中的pyplot来导入图像并显示图像。

```
#导入图像并显示图像
import cv2
import matplotlib.pyplot as plt
#读取图片
img_path1 = '/Users/guo2006/myenv/Machine Vision Experiment/bxc.jpg'
img_path2 = '/Users/guo2006/myenv/Machine Vision Experiment/background.jpg'
img_bxc = cv2.imread(img_path1) #opencv默认以BGR顺序读入
img_background = cv2.imread(img_path2)
if img_bxc is None or img_background is None:
    raise FileNotFoundError(f'未读取到图片，请检查路径')
#将BGR图像转换为RGB图像，使用matplotlib显示
img_bxc = cv2.cvtColor(img_bxc, cv2.COLOR_BGR2RGB)
img_background = cv2.cvtColor(img_background, cv2.COLOR_BGR2RGB)
#显示
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].imshow(img_bxc)
ax[0].set_title('BXC Original')
ax[0].axis('on')
ax[1].imshow(img_background)
ax[1].set_title('Background')
ax[1].axis('off')

plt.tight_layout()
plt.show()
#可以使用opencv自带的GUI
#cv2.imshow('BXC Original Picture', img_bxc)
#cv2.imshow('Background', img_background)
#cv2.waitKey(0)
#cv2.destroyAllWindows
```

- 导入图像——>使用cv2.imread('PATH')函数进行导入操作。

*注：为方便后期的调用与修改，可以将PATH作为字符串存入新变量，然后再在cv2.imread(New Variables)中调用此新变量即可。

- 显示图像——>有两种方式，如下所示：

1. 使用matplotlib库的pyplot绘制显示，值得注意的是，在Jupyter Notebook中，只支持此种plt形式的显示。若使用opencv-python库进行imshow会卡死报错。

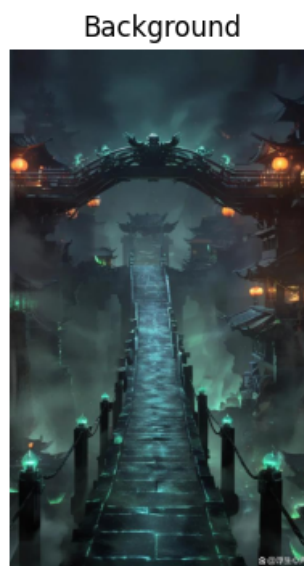
其具体语法如下所示：（假设图片变量名为img）

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
#转换色彩格式很重要，cv2读入为BGR，而plt为RGB.
plt.imshow(img) #显示图片
plt.title('Image Title Name') #图标题名
plt.axis('off/on') #坐标轴有无
plt.show()
```

2. 使用opencv-python库的cv2绘制显示。此种方法无需进行BGR——>RGB转换，但不能在Jupyter Notebook中使用。

具体语法如下所示：（假设图片变量名为img）

```
cv2.imshow('Image Title Name', img) #图片名称及显示对象
cv2.waitKey(0) #等待退出窗口的时间(ms)，0为无限等待
cv2.destroyAllWindows() #按任意键退出所有窗口
```



2.2 图片属性输出

本小节主要展示如何输出给定图片的尺寸信息，包含其高度(Height)、宽度(Width)以及通道数(Channels)。

```
#输出图像尺寸(高度、宽度及通道数)
import cv2
import matplotlib.pyplot as plt

#读取图像
img_path1 = '/Users/guo2006/myenv/Machine Vision Experiment/bxc.jpg'
img_path2 = '/Users/guo2006/myenv/Machine Vision Experiment/background.jpg'
img_bxc = cv2.imread(img_path1)
img_background = cv2.imread(img_path2)
if img_bxc is None or img_background is None:
    raise FileNotFoundError(f'未读取到图片，请重新检查路径')

img_bxc = cv2.cvtColor(img_bxc, cv2.COLOR_BGR2RGB)
#img_background = cv2.cvtColor(img_background, cv2.BGR2RGB)

height_1, width_1, channels_1 = img_bxc.shape
height_2, width_2, channels_2 = img_background.shape

print(f'图像1尺寸: {height_1} * {width_1} * {channels_1}')
print(f'图像1高度: {height_1}, 宽度: {width_1}, 通道数: {channels_1}')
print(f'图像2尺寸: {height_2} * {width_2} * {channels_2}')
print(f'图像1高度: {height_2}, 宽度: {width_2}, 通道数: {channels_2}')
```

图像1尺寸: 1165 * 874 * 3

图像1高度: 1165, 宽度: 874, 通道数: 3

图像2尺寸: 1001 * 584 * 3

图像1高度: 1001, 宽度: 584, 通道数: 3

本小节主要使用.shape语法，获得图片尺寸数据。

其中，有：

$$ImageData = img.shape = \begin{bmatrix} Height & Width & Channels \end{bmatrix}$$

也就是说，使用img.shape得到的是一个1×3的矩阵，分别为图片的高度、宽度、通道数。之后，我们再使用print()函数进行打印输出即可。

后续当我们只需要使用图片的高度和宽度数据，而不需要通道数数据时，可以对上面矩阵进行切片。例如：

```
img_height, img_width = img.shape[:2] #只切片使用前两个数据
```

2.3 图片灰度转换操作

灰度转换操作在图像预处理部分是十分重要且实用的。其内容较为简单，本部分将详细阐述。

```
#灰度图像处理
import cv2
import matplotlib.pyplot as plt
#导入图像
img_path1 = '/Users/guo2006/myenv/Machine Vision Experiment/bxc.jpg'
img_path2 = '/Users/guo2006/myenv/Machine Vision Experiment/background.jpg'
img_bxc = cv2.imread(img_path1)
img_background = cv2.imread(img_path2)
if img_bxc is None or img_background is None:
    raise FileNotFoundError(f'未读取到照片，检查路径后重试')
else:
    img_bxc_gray = cv2.cvtColor(img_bxc, cv2.COLOR_BGR2GRAY)
    img_background_gray = cv2.cvtColor(img_background, cv2.COLOR_BGR2GRAY)

    fig, ax = plt.subplots(1, 2, figsize=(10, 4))
    ax[0].imshow(img_bxc_gray, cmap='gray')
    ax[0].set_title('BXC Gray Image')
    ax[0].axis('off')

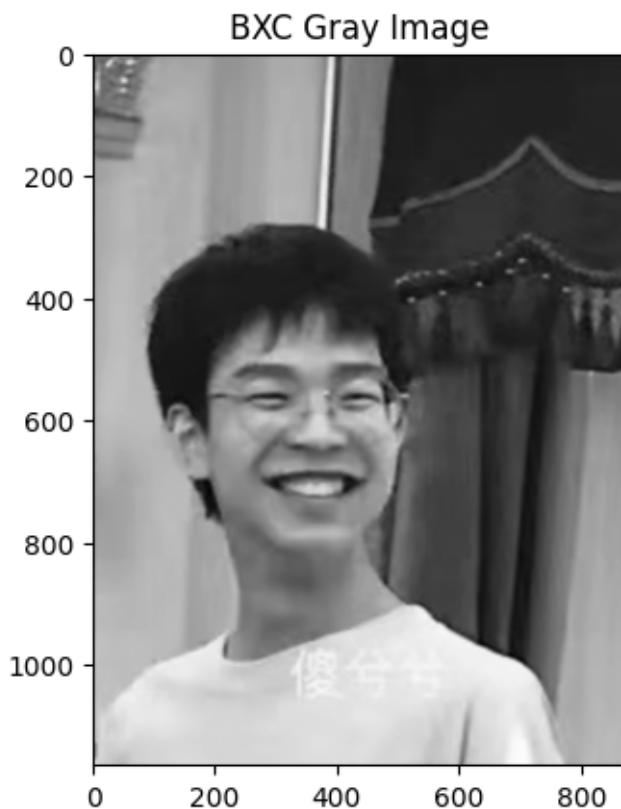
    ax[1].imshow(img_background_gray, cmap='gray')
    ax[1].set_title('Background Gray Image')
    ax[1].axis('off')
```



事实上，本部分依托于opencv-python库调用cv2十分简单。代码如下所示。

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # BGR—>GRAY
```

```
#灰度图像转换(plt导出)
import cv2
import matplotlib.pyplot as plt
#导入图像
img_path = '/Users/guo2006/myenv/Machine Vision Experiment/bxc.jpg'
img_bxc = cv2.imread(img_path)
if img_bxc is None:
    raise FileNotFoundError(f'未读取到照片，检查路径后重试')
else:
    img_bxc_gray = cv2.cvtColor(img_bxc, cv2.COLOR_BGR2GRAY)
    plt.imshow(img_bxc_gray, cmap='gray')
    plt.axis('on')
    plt.title('BXC Gray Image')
    plt.show()
```



2.4 图像变换操作

此部分包含了图像的缩放操作、旋转操作及剪裁操作。下面将分步骤进行展开。

2.4.1 图像缩放操作

图像缩放操作主要包含两种类型：指定横纵长宽度放缩、指定缩放比例放缩。

```
#缩放操作
import cv2
import matplotlib.pyplot as plt

img_path1 = '/Users/guo2006/myenv/Machine Vision Experiment/bxc.jpg'
img_path2 = '/Users/guo2006/myenv/Machine Vision Experiment/background.jpg'

img_bxc = cv2.imread(img_path1)
img_background = cv2.imread(img_path2)

img_bxc = cv2.cvtColor(img_bxc, cv2.COLOR_BGR2RGB)
```

```
img_bxc_resized = cv2.resize(img_bxc,(300,200)) #指定pixel大小

scale = 0.1 #指定缩放比例
img_background_resized = cv2.resize(img_background,None,fx = scale,fy = scale)

fig,ax = plt.subplots(1,2,figsize=(12,4))
ax[0].imshow(img_bxc_resized)
ax[0].set_title('BXC Resized Image 1')
ax[1].imshow(img_background_resized)
ax[1].set_title('Background Resized Image')
```

对于两种不同情况的放缩方式，分别进行阐释：

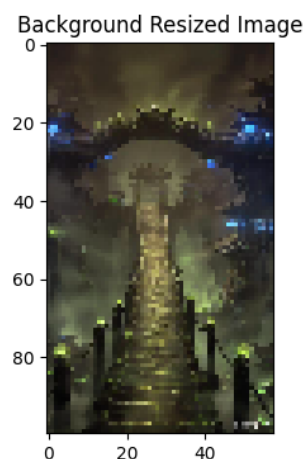
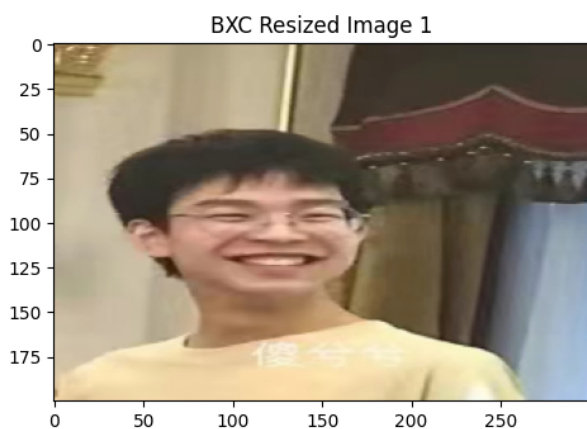
- 指定纵横长宽度放缩（图像比例可能改变）

```
img_new = cv2.resize(img, (Width, Height))
```

- 指定缩放比例放缩（图像比例不会改变）

```
scale = 0.1 # 指定缩放比例
img_new = cv2.resize(img, None, fx = scale, fy = scale)
```

Text(0.5, 1.0, 'Background Resized Image')



2.4.2 图像旋转操作

本质上，图像旋转操作并不难，但是图像在旋转后可能会有因画布大小不足而四角无法显示全的问题。故如何实现画布自适应大小，确保旋转后图像能够被显示完全是旋转操作最大的难点。

我们所需要解决的问题无非是绕哪旋转、如何旋转、旋转多少度、是否缩放等问题。

- **绕哪旋转**：要绕中心旋转，先要得到中心坐标。即是有：

$$\text{Rotation Center} = (X, Y) = (\frac{\text{Width}}{2}, \frac{\text{Height}}{2})$$

根据img.shape进行切片得到Width和Height即可。

- **旋转多少度**：给定一个参数Rotation Angle即可，在后续带入cv2函数。
- **缩放比例**：给定一个参数Scale即可，后续带入cv2函数。

在简单旋转的情况下，也就是不考虑四角会因画布大小不足被裁切的情况下，可以直接调用cv2.getRotationMatrix2D函数。如下所示：

```
M = cv2.getRotationMatrix2D(Rotation Center, Rotation Angle, Scale)
img_rotated = cv2.warpAffine(img, M, (Width,Height)) #更新后的矩阵重采样
```

```
#旋转操作(难点)
import cv2
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/before.jpg'
img_before = cv2.imread(img_path)
if img_before is None:
    raise FileNotFoundError(f'未找到对应文件，检查路径后重试')
else:
    img_before = cv2.cvtColor(img_before, cv2.COLOR_BGR2RGB)
    height, width = img_before.shape[:2] #对.shape返回的三数数组进行切片，通道数值舍弃

    center = (width//2, height//2) #找到对应旋转中心
    rotation_angle = 30 #逆时针旋转30deg
    scale = 1 #缩放系数

    #计算旋转矩阵
    M = cv2.getRotationMatrix2D(center, rotation_angle, scale)
    #img_before_rotated = cv2.warpAffine(img_before, M, (width,height)) #更新后的矩阵重采样

    #cv2.imshow('<before> Rotated Picture',img_before_rotated)
    #cv2.waitKey(0)
```

```

#cv2.destroyAllWindows() #显示出的四个角由于画布大小不够将会被切掉

#自动调整画布大小
cos, sin = abs(M[0,0]), abs(M[0,1])
new_width = int(height*sin + width*cos)
new_height = int(width*sin + height*cos)
#补齐偏移量
M[0,2] += new_width/2 - center[0] #center[0]是width//2
M[1,2] += new_height/2 - center[1] #center[1]是height//2
#用更新后的矩阵M和扩大后的画布大小(new_width,new_height)做重采样
img_before_rotated = cv2.warpAffine(img_before, M, (new_width,new_height))

plt.imshow(img_before_rotated)
plt.title('<Before> Rotated Image')
plt.axis('on')
plt.show()

```

在考虑复杂旋转情况下，需要一定的数学变换来进行画布自适应大小的调节。具体数学原理如下：
`cv2.getRotationMatrix2D`返回一个 2×3 的仿射阵，具体如下：

$$M = cv2.getRotationMatrix2D = \begin{bmatrix} \cos\theta & -\sin\theta & tx \\ \sin\theta & \cos\theta & ty \end{bmatrix}$$

其中， θ 为旋转角度， tx 、 ty 为偏移量。

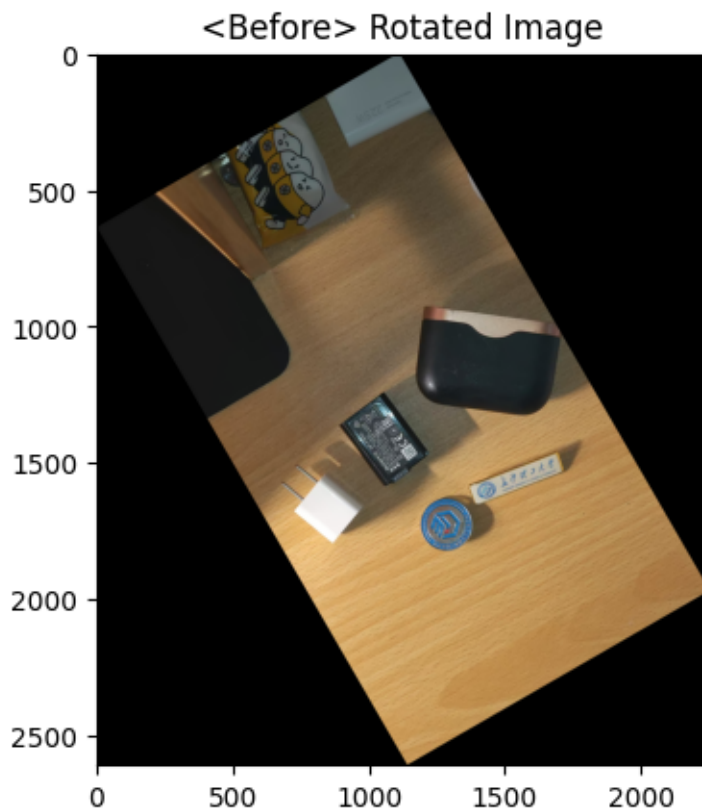
旋转后所需的最小宽度 $Width_{min} = Height_{Original} \times |\sin\theta| + Width_{Original} \times |\cos\theta|$

旋转后所需的最小高度 $Height_{min} = Width_{Original} \times |\sin\theta| + Height_{Original} \times |\cos\theta|$

画布中心有偏移，要通过 tx 、 ty 补齐偏移量。具体如下：

原图中心($\frac{Width}{2}, \frac{Height}{2}$)——>新画布中心($\frac{Width_{new}}{2}, \frac{Height_{new}}{2}$)

- $tx = M_{0,2} + \frac{Width_{new}}{2} - center[0]$
- $ty = M_{1,2} + \frac{Height_{new}}{2} - center[1]$



2.4.3 图像剪裁操作

给出变量存储行范围与列范围，对指定范围进行裁剪。

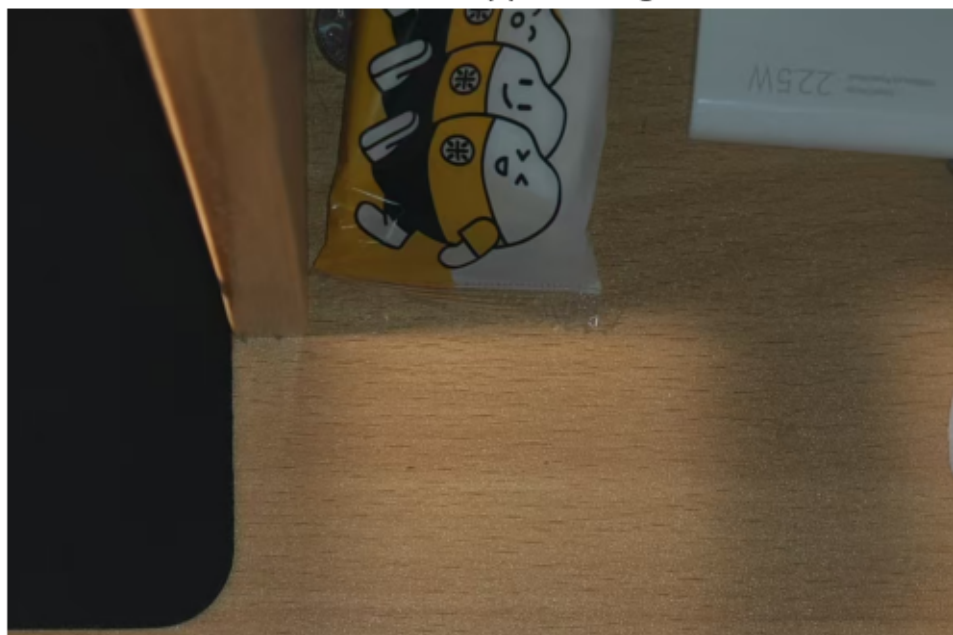
```
#剪裁操作
import cv2
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/after.jpg'
img_after = cv2.imread(img_path)
if img_after is None:
    raise FileNotFoundError(f'未找到对应文件，检查路径后重试')
else:
    img_after = cv2.cvtColor(img_after, cv2.COLOR_BGR2RGB)
    y1, y2 = 50, 820 #行范围
    x1, x2 = 115, 1530 #列范围

    img_after_cropped = img_after[y1:y2, x1:x2]
```

```
plt.imshow(img_after_cropped)
plt.axis('off')
plt.title('<After> Cropped Image')
plt.show()
```

<After> Cropped Image



给出四变量存储行范围列范围，再对目标进行切片即可。

2.5 图像运算操作

图像运算操作包括加法运算、减法运算、乘法运算、除法运算四种。

值得注意的是，进行运算的图像必须要将图片大小统一。需要运用相似数学原理。

设原图高度为 h_0 ，原图宽度为 l_0 ；修改后的图像高度为 h ，宽度为 l 。

令改后的高度等于所传入的第一张图的高度，宽度根据相似原理进行计算。

则有：

$$\frac{h}{h_0} = \frac{l}{l_0}$$
$$l = \frac{h \times l_0}{h_0}$$

下面将详细阐释各种运算方法。

2.5.1 加法运算

加法运算包含平均降噪和双重曝光两种，下面将分别进行阐释。


```
#图像加法运算->平均降噪
import cv2
import matplotlib.pyplot as plt
import numpy as np
import os
import random

#显示函数（更简便地展示多图对比）
def show(title, *imgs):
    #多张图片水平拼成一行，同一高度后显示
    h0 = imgs[0].shape[0] #传入的第一张图为统一高度

    resized_list = []
    for img in imgs:
        resized_img = cv2.resize(img, (int(img.shape[1]*h0/img.shape[0]), h0))
        resized_list.append(resized_img)
    canvas = cv2.hconcat(resized_list)
    canvas = cv2.cvtColor(canvas, cv2.COLOR_BGR2RGB)

    plt.imshow(canvas)
    plt.title(title)
    plt.axis('off')
    plt.show()

#图像平均函数（降噪）
def add_noise(img, sigma=25):
    #添加高斯噪声
    noise = np.random.normal(0, sigma, img.shape).astype(np.uint8)
    return cv2.add(img, noise)

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/bxc.jpg'
img_bxc_original = cv2.imread(img_path)
height_bxc, width_bxc = img_bxc_original.shape[:2]
N = 50 #生成50张噪声图像

noise_img_list = [] #创建随机噪声图像集
for _ in range(N):
```

```

noise_img_list.append(add_noise(img_bxc_original))
#求噪声抑制的平均图像
avg_noised = np.mean(noise_img_list, axis=0).astype(np.uint8)

show('Addition--Image Denoising',img_bxc_original,noise_img_list[0],avg_noised)

```

Addition——Image Denoising



```

#图像加法运算->双重曝光
import cv2
import matplotlib.pyplot as plt
import numpy as np
import os
import random

img_bxc_path = '/Users/guo2006/myenv/Machine Vision Experiment/bxc.jpg'
img_background_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↳background.jpg'

img_bxc = cv2.imread(img_bxc_path)
img_background = cv2.imread(img_background_path)
#色彩模式转换
img_bxc = cv2.cvtColor(img_bxc, cv2.COLOR_BGR2RGB)
img_background = cv2.cvtColor(img_background, cv2.COLOR_BGR2RGB)

```

```

height_bxc , width_bxc = img_bxc.shape[:2]
#调整背景图片与主体图片尺寸一致
img_background = cv2.resize(img_background,(width_bxc,height_bxc))
#双重曝光加权
double_exposure = cv2.addWeighted(img_bxc,0.6,img_background,0.4,0)

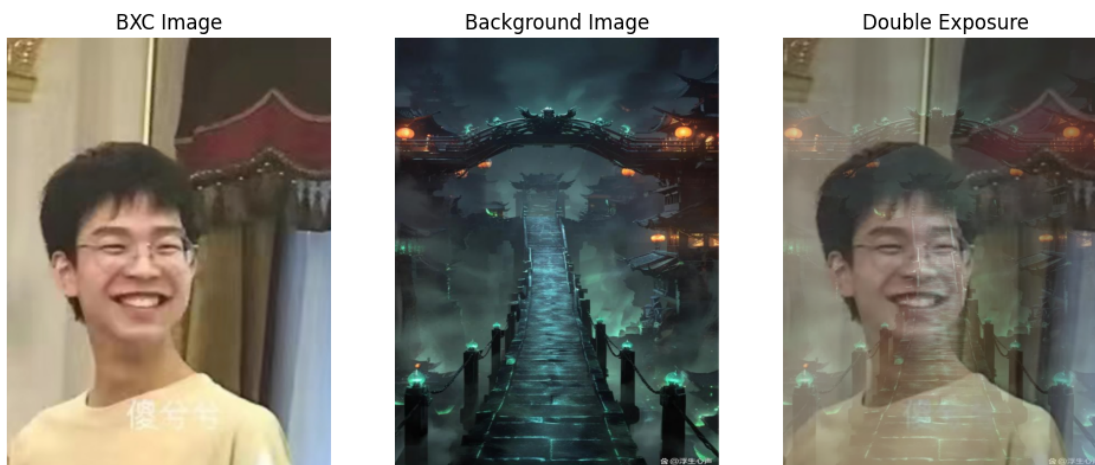
fig, ax = plt.subplots(1,3,figsize=(12,5))
ax[0].imshow(img_bxc)
ax[0].set_title('BXC Image')
ax[0].axis('off')
ax[1].imshow(img_background)
ax[1].set_title('Background Image')
ax[1].axis('off')
ax[2].imshow(double_exposure)
ax[2].set_title('Double Exposure')
ax[2].axis('off')

```

```

(np.float64(-0.5), np.float64(873.5), np.float64(1164.5), np.float64(-0.5))

```



2.5.2 减法运算

减法运算主要运用于静物识别，其可以将两张相同视角的不同静物照片进行相减，显示出高光部分为差异区。减去的部分为黑色阴暗区。

利用大津算法来寻找最佳二值化值，并输出最佳二值化阈值。大津二值化法用来自动对基于聚类的图像进行二值化，或者说，将一个灰度图像退化为二值图像。

```
#减法->静物差异识别
import cv2
import matplotlib.pyplot as plt
import numpy as np
import os
import random

img_path_before = '/Users/guo2006/myenv/Machine Vision Experiment/before.jpg'
img_path_after = '/Users/guo2006/myenv/Machine Vision Experiment/after.jpg'
img_before, img_after = cv2.imread(img_path_before), cv2.imread(img_path_after)

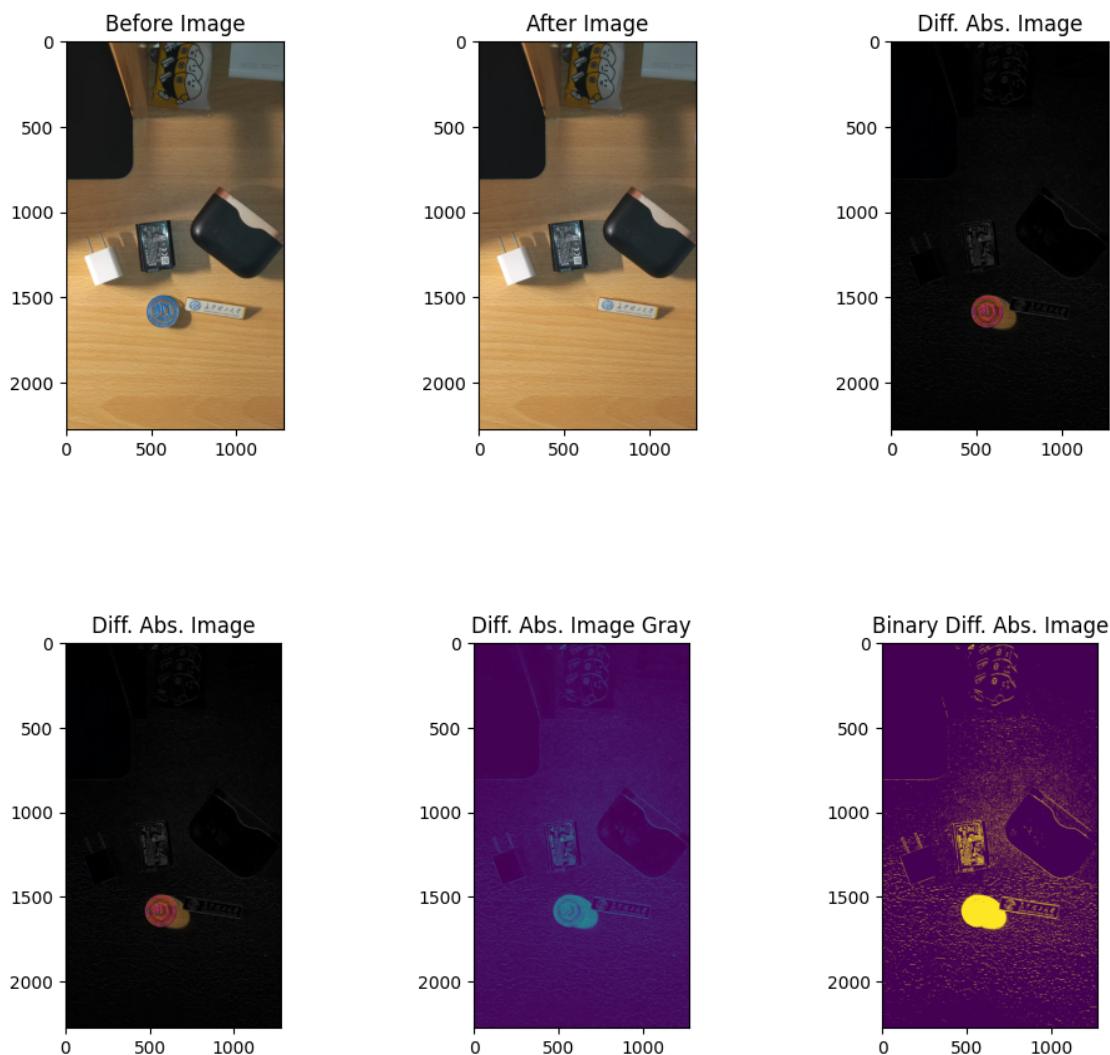
#色彩模式转换
img_before = cv2.cvtColor(img_before, cv2.COLOR_BGR2RGB)
img_after = cv2.cvtColor(img_after, cv2.COLOR_BGR2RGB)
#修改为统一尺寸
height_before, width_before = img_before.shape[:2]
img_after = cv2.resize(img_after, (width_before, height_before))
#做差值图
img_diff_abs = cv2.absdiff(img_before, img_after)
img_diff_abs_gray = cv2.cvtColor(img_diff_abs, cv2.COLOR_BGR2GRAY)
#调用大津算法计算最佳阈值
thresh_used, mask = cv2.threshold(img_diff_abs_gray, 0, 255, cv2.THRESH_BINARY +
    ↪cv2.THRESH_OTSU)
print('Otsu Automatic Threshold=', thresh_used)

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(img_before)
ax[0].set_title('Before Image')
ax[1].imshow(img_after)
ax[1].set_title('After Image')
ax[2].imshow(img_diff_abs)
ax[2].set_title('Diff. Abs. Image')
fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(img_diff_abs)
ax[0].set_title('Diff. Abs. Image')
ax[1].imshow(img_diff_abs_gray)
```

```
ax[1].set_title('Diff. Abs. Image Gray')
ax[2].imshow(mask)
ax[2].set_title('Binary Diff. Abs. Image')
```

Otsu Automatic Threshold= 22.0

```
[37]: Text(0.5, 1.0, 'Binary Diff. Abs. Image')
```



2.5.3 乘法运算

图像乘法是将两幅图像的对应像素值相乘，生成一幅新的图像。

乘法运算有许多用途，比如我们可以为各个像素乘上一个系数，整体提亮或减暗张图片的某个局部或整体；还可以单独提取出来图像的某一个部分。但是无论哪种用法，我们都要先为图像创建一张‘掩膜’(Mask)。它相当于量化照片的一种规则或模版。掩膜的系数不同，对图像有不同的作用。图像的乘法运算主要包含两种类型：

- 二值掩膜<0 / 255>
- 灰度掩膜<0 ~ 255>

与其说乘法运算有两种类型，不如说掩膜的量化模型有两种。不过，不同的类型起到的作用不相同，具体要看情况分析。

```
#乘法（掩膜抠图）
import cv2
import matplotlib.pyplot as plt
import random
import numpy as np
import os

img_bxc_path = '/Users/guo2006/myenv/Machine Vision Experiment/bxc.jpg'
img_bxc = cv2.imread(img_bxc_path)
height_bxc, width_bxc = img_bxc.shape[:2]

#二值掩膜
mask_bin = np.zeros((height_bxc,width_bxc),np.uint8) #创建黑色掩膜
cv2.circle(mask_bin,(width_bxc//2,height_bxc//2),min(height_bxc, width_bxc)//3,
→255, -1) #使黑色掩膜的中间部分出现一个白色的圆域

result_bin = cv2.bitwise_and(img_bxc, img_bxc, mask=mask_bin) #掩膜内白色圆域内容
留下，其余部分删除

#灰度掩膜
Y, X = np.ogrid[:height_bxc, :width_bxc] #创建欧式坐标网格
dist = np.sqrt((X-width_bxc)**2+(Y-height_bxc)**2) #计算各个像素距离中心的欧氏距
离

#把距离线性映射到0-255的灰度（设定掩膜规则）
mask_gray = np.clip((255-dist*(255/min(height_bxc, width_bxc)/2)),0,255).
→astype(np.uint8)

#可能导致灰度变化不明显？怎么解决
result_gray = cv2.bitwise_and(img_bxc, img_bxc, mask=mask_gray)

img_bxc = cv2.cvtColor(img_bxc, cv2.COLOR_BGR2RGB)
mask_bin = cv2.cvtColor(mask_bin, cv2.COLOR_BGR2RGB)
mask_gray = cv2.cvtColor(mask_gray, cv2.COLOR_BGR2RGB)
```

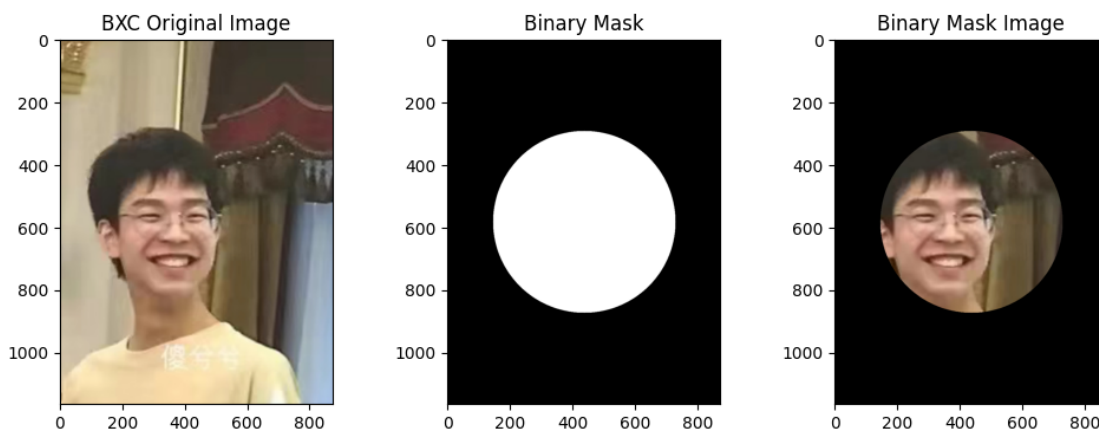
```

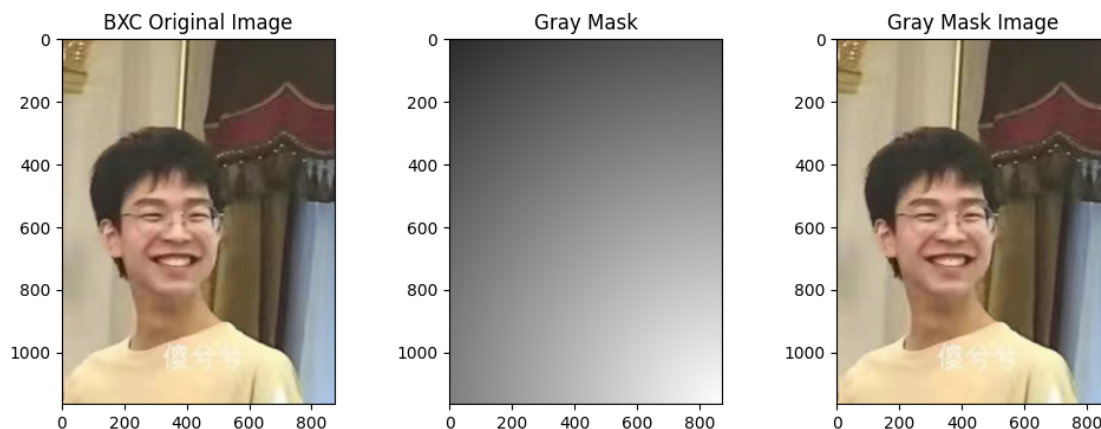
result_bin = cv2.cvtColor(result_bin, cv2.COLOR_BGR2RGB)
result_gray = cv2.cvtColor(result_gray, cv2.COLOR_BGR2RGB)

fig, ax = plt.subplots(1,3,figsize=(12,4))
ax[0].imshow(img_bxc)
ax[0].set_title('BXC Original Image')
ax[1].imshow(mask_bin)
ax[1].set_title('Binary Mask')
ax[2].imshow(result_bin)
ax[2].set_title('Binary Mask Image')
fig, ax = plt.subplots(1,3,figsize=(12,4))
ax[0].imshow(img_bxc)
ax[0].set_title('BXC Original Image')
ax[1].imshow(mask_gray)
ax[1].set_title('Gray Mask')
ax[2].imshow(result_gray)
ax[2].set_title('Gray Mask Image')

```

Text(0.5, 1.0, 'Gray Mask Image')





2.5.4 除法运算

图像除法是两幅图像的对应像素值相除生成新图像的过程，其包含了两种不同的模型：

- 常数除法——>均匀的将整张图片变暗
- 非常数除法——>抵消光照不均匀 用于图像矫正

本质上两者的不同也是在掩膜层面上的，事实上，原图与掩膜层在光照参数上的叠加便是处理后的图像。在这一点上，乘法与除法是共同之处的。

```
#除法
import cv2
import matplotlib.pyplot as plt
import random
import os
import numpy as np
#常数除法--->把整张图均匀变暗
img_bxc_path = '/Users/guo2006/myenv/Machine Vision Experiment/bxc.jpg'
img_bxc = cv2.imread(img_bxc_path).astype(np.float32) #转换成0-255的浮点数，方便做除法

img_bxc_dark = (img_bxc/3).clip(0,255).astype(np.uint8) #除整+截断+转换
img_bxc_dark = cv2.cvtColor(img_bxc_dark, cv2.COLOR_BGR2RGB)
#非整除法--->抵消光照不均(图像矫正)
height_bxc, width_bxc = img_bxc.shape[:2]
Y, X = np.ogrid[:height_bxc, :width_bxc] #创建网格
```



```

light_illum = np.sqrt((X-width_bxc)**2+(Y-height_bxc)**2) #距离图片中心欧式距离
light_illum = (light_illum/light_illum.max()*0.8+0.2).astype(np.float32) #归一化
到0.2~1.0

#light_illum为双通道, img_bxc为三通道, 二者无法用np相除, 办法有两种:
#1.将img_bxc转换为img_bxc_gray(BGR->GRAY), 两个双通道可相除<双通道灰度图>
#2.将light_illum转换为三通道<三通道彩图>

#法1.双通道灰度图像
img_bxc_gray = cv2.cvtColor(img_bxc, cv2.COLOR_BGR2GRAY)
#除法抵消暗区 + 0~255截断 + 格式转换
corrected_img_2ch = (img_bxc_gray / light_illum).clip(0,255).astype(np.uint8)

#法2.三通道彩色图像
#扩充光线模版为三通道
light_illum_3ch = cv2.merge([light_illum, light_illum, light_illum])
#除法抵消暗区 + 0~255截断 + 格式转换
corrected_img_3ch = (img_bxc / light_illum_3ch).clip(0,255).astype(np.uint8)

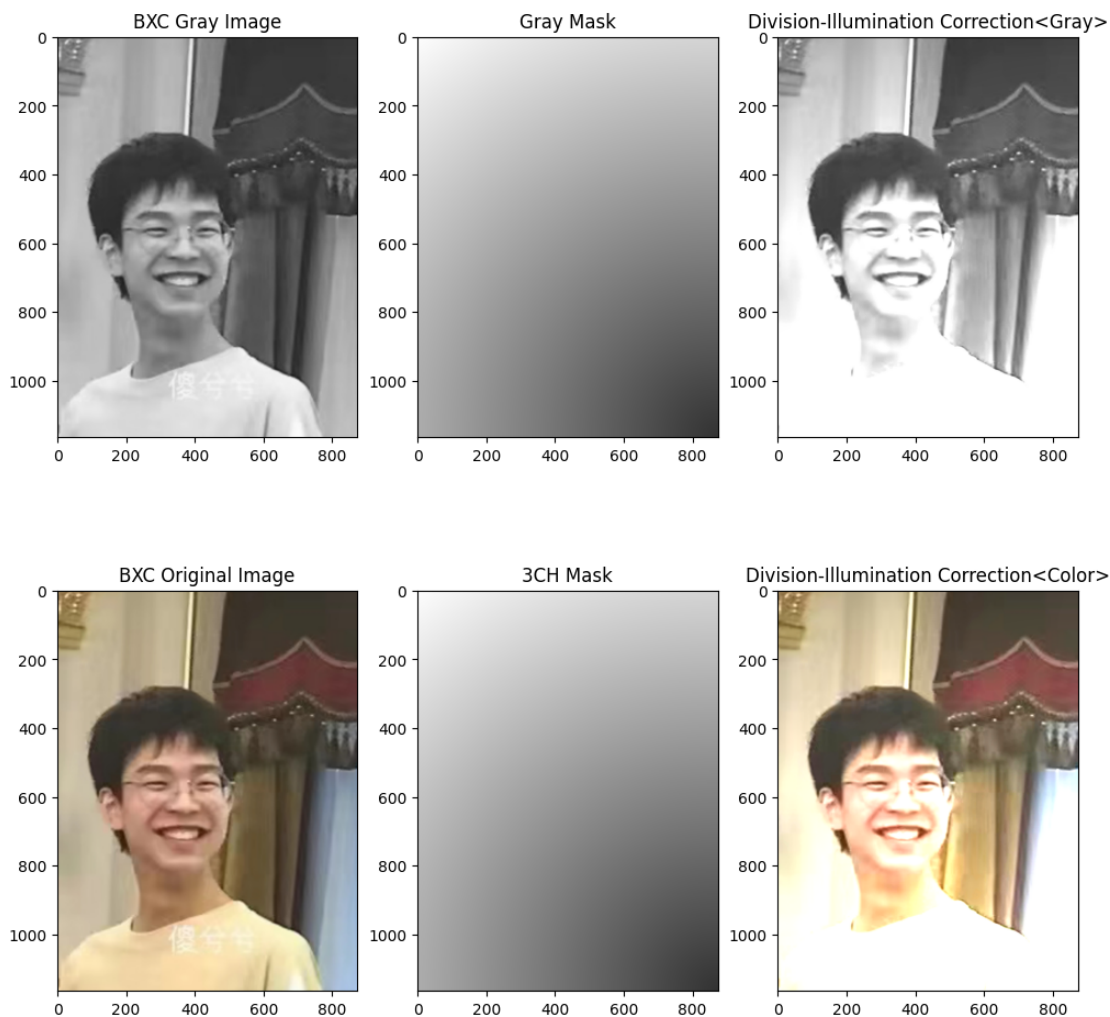
img_bxc = cv2.cvtColor(img_bxc, cv2.COLOR_BGR2RGB)
light_illum = cv2.cvtColor(light_illum, cv2.COLOR_BGR2RGB)
corrected_img_2ch = cv2.cvtColor(corrected_img_2ch, cv2.COLOR_BGR2RGB)
img_bxc_gray = cv2.cvtColor(img_bxc_gray, cv2.COLOR_BGR2RGB)
light_illum_3ch = cv2.cvtColor(light_illum_3ch, cv2.COLOR_BGR2RGB)
corrected_img_3ch = cv2.cvtColor(corrected_img_3ch, cv2.COLOR_BGR2RGB)

fig, ax = plt.subplots(2,3,figsize=(12,12))
ax[0,0].imshow(img_bxc_gray)
ax[0,0].set_title('BXC Gray Image')
ax[0,1].imshow((light_illum*255).astype(np.uint8))
ax[0,1].set_title('Gray Mask')
ax[0,2].imshow(corrected_img_2ch)
ax[0,2].set_title('Division-Illumination Correction<Gray>')
ax[1,0].imshow(img_bxc)
ax[1,0].set_title('BXC Original Image')
ax[1,1].imshow((light_illum_3ch*255).astype(np.uint8))
ax[1,1].set_title('3CH Mask')

```

```
ax[1,2].imshow(corrected_img_3ch)
ax[1,2].set_title('Division-Illumination Correction<Color>')
```

```
Text(0.5, 1.0, 'Division-Illumination Correction<Color>')
```



2.6 逻辑运算

一个图像可以用像素矩阵来表示，通过随机数生成一个新的像素矩阵，称之为密钥图像，将原图与密钥图像进行按位异或运算，对图像进行加密，生成一个加密图像，这是对图像的整体打码。通常通过掩膜提取图像中感兴趣区域（ROI），再对 ROI 进行加密，这是对图像的局部打码。将加密图像与原密钥图像按位异或运算，进行解密，可以得到原图，这个过程叫做解码。

本小节将使用opencv-python库自带的haar人脸识别系统，对人脸部分进行检测与识别、并进行随机密钥打码操作，然后再次使用密钥进行反运算来解密并显示。

```
#ROI、解码
import cv2
import matplotlib.pyplot as plt
import random
import os
import numpy as np

#加载人脸检测器
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
    ↪ 'haarcascade_frontalface_default.xml')

img_face_path = '/Users/guo2006/myenv/Machine Vision Experiment/bxc.jpg'
face_img = cv2.imread(img_face_path)
if face_img is None:
    raise FileNotFoundError(f'图像路径错误，检查路径后重试')
face_img_gray = cv2.cvtColor(face_img, cv2.COLOR_BGR2GRAY)
#检测脸部->得到ROI掩膜
faces = face_cascade.detectMultiScale(face_img_gray, scaleFactor=1.2,
    ↪ minNeighbors=5)
#返回值faces是一个 N×4 的 numpy 数组，每行 (x, y, w, h) 代表一张人脸的左上角坐标和宽高。
#创建全黑掩膜
mask = np.zeros(face_img.shape[:2], dtype=np.uint8) # 单通道全黑
#脸部涂白
for (x, y, w, h) in faces:
    cv2.rectangle(mask, (x, y), (x+w, y+h), 255, -1)
    #绘制在mask上，左上角，右下角，255=白，实心填充
#生成随机密钥
random_key = np.random.randint(0, 256, face_img.shape, dtype=np.uint8) #0-255随机
#生成的尺寸与face_img完全一样的三位数组
#只对人脸区域(mask>0区域)进行异或加密
encrypted = face_img.copy() #先整图复制一份，后面只改人脸部分，背景不动
encrypted[mask > 0] = cv2.bitwise_xor(face_img, random_key)[mask > 0]

#再进行一次异或加密进行解密
decrypted = encrypted.copy()
```

```
decrypted[mask > 0] = cv2.bitwise_xor(encrypted, random_key)[mask > 0]
```

```
#色彩转换
```

```
encrypted = cv2.cvtColor(encrypted, cv2.COLOR_BGR2RGB)
```

```
decrypted = cv2.cvtColor(decrypted, cv2.COLOR_BGR2RGB)
```

```
fig,ax = plt.subplots(1,2,figsize=(12,4))
```

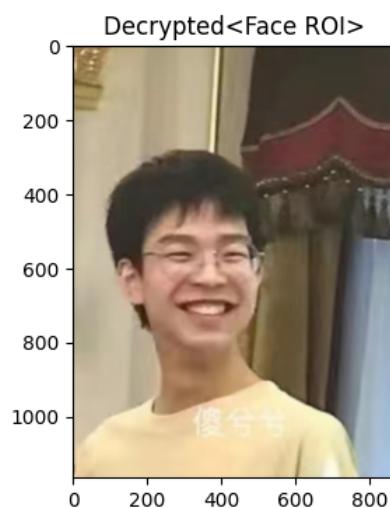
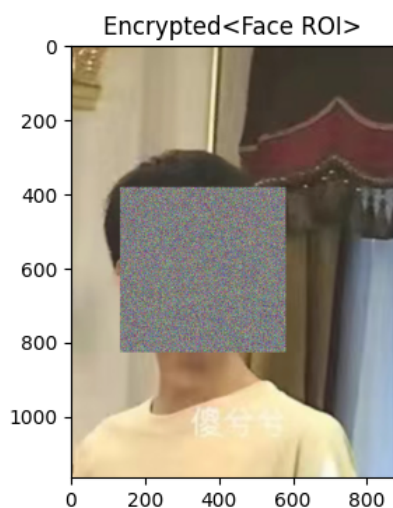
```
ax[0].imshow(encrypted)
```

```
ax[0].set_title('Encrypted<Face ROI>')
```

```
ax[1].imshow(decrypted)
```

```
ax[1].set_title('Decrypted<Face ROI>')
```

```
Text(0.5, 1.0, 'Decrypted<Face ROI>')
```



|| 第三章

L^AT_EX使用基本指令

3 L^AT_EX使用基础命令

3.1 代码基本结构

```

\documentclass{article} %定义文档类型
\usepackage{amsmath, amssymb, geometry, graphicx, float, caption, subfigure,
booktabs, tcolorbox, ctex} %引入宏包
\geometry{a4paper, margin=2.5cm} %设置页边距

\title{标题} %设置标题
\author{作者名1 \thanks{通讯作者.} \and 作者名2 \and etc.} %设置作者
\date{日期} %设置日期

%正式结构 %
\begin{document}
  \maketitle %忘加此句会导致无标题!

  \begin{abstract}
    摘要部分内容. %填写摘要内容
  \end{abstract}

  \tableofcontents %自动创建目录
  \newpage %换页命令

  \section{第一部分-标题}
    第一部分-正文
  \subsection{第一部分 第一章-标题}
    第一部分 第一章-正文
  \subsubsection{第一部分 第一章 第一节-标题}
    第一部分 第一章 第一节-正文
\end{document}

```

*注：此处\usepackage使用的宏包已足够≤90%的排版场景，在后续表3中将标注各宏包的功能。

上面的代码框即为基本L^AT_EX文档结构，掌握基本结构后即可在基本结构的基础上学习细节内容，填充在结构中就可以形成一篇文章了。本章接下来的内容里，将会分类介绍、由简入繁地介绍各个部分的书写办法。

基本步骤为：公式 → 枚举 → 图 → 表 → 文本框 → 引用文献

宏包名	功能
amsmath	AMS数学拓展包
amssymb	数学拓展包
geometry	页面设置
float	图表位置设置
multicol	单双栏页面编辑
graphicx	插图包
caption	图表标题
subfigure	多图拓展包
booktabs	三线表
tcolorbox	文本框拓展包
makecell	表格内换行

3.2 单双栏版式变换

这节的出现比较矛盾，它只是一个小点，但是却又有时候很重要。但是将它放到以下任一节都略显突兀、格格不入。故此处单独拿出一节来说明下它。

这个地方我们将会用到`multicol`宏包，我将其译为多样栏功能。使用此包你可以既在某些地方使用一栏版式，又可以在某些地方分成两栏、三栏乃至更多。其基础语法很简单，只需在你想要分栏的地方加入`multicols`环境即可。演示如下。

```
\begin{document}
    对于此处不在multicols环境内的内容，都是单栏内容。
    \begin{multicols}{2}
        对于此在multicols环境内的内容，LATEX将会自动分为两栏显示。
    \end{multicols}
\end{document}
```

对于此处不在`multicols`环境内的内容，都是单栏内容。

*L^AT_EX*对于此处动把内容分为两栏在`multicols`环境内显示。
的内容，其将会自

总结`multicols`语法为：

```
\begin{multicols}{分栏数目}
    分栏内容
\end{multicols}
```

3.3 公式插入

在此节，将讲述普通公式的插入、公式作为转义字符在文本插入领域的应用等。由于矩阵的插入具有一定特殊性，故单独拿出一小节梳理。

3.3.1 普通公式

对于普通公式，大致要考虑三个问题：

- 行内公式 or 行间公式
- 编号公式 or 不编号公式
- 单行公式 or 多行公式

可以简单画如下表4相应的表格，使得三个问题更加直观。

分类	单行公式	多行公式
行内公式	不编号公式 (<i>Con.1</i>)	——
行间公式	编号公式 (<i>Con.2</i>) 不编号公式 (<i>Con.3</i>)	编号公式 (<i>Con.4</i>) 不编号公式 (<i>Con.5</i>)

- 对于 *Con.1* →

```
$ Equation $ %Equation处为输入公式
```

- 对于 *Con.2* →

```
\begin{equation}
Equation %Equation处为输入公式
\end{equation}
```

- 对于 *Con.3* →

```
\begin{equation*} %只需加入*号即可不编号公式.
Equation %Equation处为输入公式
\end{equation*}
```

另外，还有一种方式可以不编号行间公式。如下所示。

```
$$Equation$$
```

- 对于 *Con.4* →

```
\begin{align}
Equation1 \\\
Equation2 \\\
使用&=可以在等号处对齐.
\end{align}
```


- 对于 *Con.5* →

```
\begin*{align} %只需加入*号即可不编号此部分的全部公式.
  \underline{Equation1} \\
  \underline{Equation2} \\
  使用&=可以在等号处对齐.
\end{align}
```

此外, 还可以使用多个 $\$Equation\$$ 换行叠加来实现。如下所示.

```
$$\underline{Equation1} \$ \$ \\
\$\underline{Equation2} \$ \$
```

注意!

若只想要对多行行间公式的部分公式不编号, 可以采用 $\backslash notag$ 对其进行处理。如下所示.

```
\begin{align}
  \underline{Equation1} \\
  \underline{Equation2} \backslash notag \\ %此行公式不会被编号.
  \underline{Equation3} \\
  使用&=可以在等号处对齐.
\end{align}
```

对于公式输入这一模块, 除了掌握(记忆)基本的运算符之外, 还应当多加练习。L^AT_EX公式的符号有很多, 下面列举部分常用的L^AT_EX公式符号。(如下表5所示)

除此之外, 关于换行与空格的知识如下。

1. 空格符号

- 特殊空格符号 $\sim$ → 不间断空格符, 通常用于避免在两行之间断开。
- $\backslash +$ Space键位 → 较短的不间断空格, 可以用在需要较小空格的位置。
- $\backslash hspace\{\text{长度}\}$ → 插入一个特定长度的空格。
单位: **em** (字体大小单位) / **cm** (厘米) / **pt** (磅) 等。
注: $\backslash hspace^\{\text{长度}\}$ → 强制插入一个特定长度的空格 (在行首也会插入)
- $\backslash quad$ 及 $\backslash qquad$ → 简便空格命令, 分别插入1em和2em宽度的空格。

2. 换行符号

- $\backslash vspace\{\text{长度}\}$ → 插入指定长度的垂直空白。

```

第一行内容..... \\
\vspace{1cm}
第二行内容.....

```

*注: `\vspace*{长度}` → 强制插入一个特定长度的垂直空白 (在页面顶部也会插入). 可以用于页面顶部的排版调整.

- `\\` → 换行符, 常用于表格、公式以及某些环境中强制换行。
- `\newline` → 与`\\`类似, 但在某些情况下更符合语法规范。

```

第一行内容..... \newline
第二行内容.....

```

3. 其余功能

- `\\*` → 换行并禁止分页, 适用于需要将两行内容固定在同一页的情况。
- **par环境** → 在par环境中的内容会自动根据页面宽带换行。

```

\begin{par}
  长文本内容可以自动根据页面宽度换行.
\end{par}

```

符号	语法	符号	语法	符号	语法	符号	语法
α	<code>\alpha</code>	β	<code>\beta</code>	γ	<code>\gamma</code>	θ	<code>\theta</code>
ε	<code>\varepsilon</code>	δ	<code>\delta</code>	μ	<code>\mu</code>	ν	<code>\nu</code>
η	<code>\eta</code>	ζ	<code>\zeta</code>	λ	<code>\lambda</code>	ψ	<code>\psi</code>
σ	<code>\sigma</code>	ξ	<code>\xi</code>	τ	<code>\tau</code>	ϕ	<code>\phi</code>
φ	<code>\varphi</code>	ρ	<code>\rho</code>	χ	<code>\chi</code>	ω	<code>\omega</code>
π	<code>\pi</code>						
Σ	<code>\Sigma</code>	Π	<code>\Pi</code>	Δ	<code>\Delta</code>	Γ	<code>\Gamma</code>
Ψ	<code>\Psi</code>	Θ	<code>\Theta</code>	Λ	<code>\Lambda</code>	Ω	<code>\Omega</code>
Φ	<code>\Phi</code>	Ξ	<code>\Xi</code>				
$+$	<code>+</code>	$-$	<code>-</code>	$\times$	<code>\times</code>	$\div$	<code>\div</code>
∂	<code>\partial</code>	∞	<code>\infty</code>	$\rightarrow$	<code>\rightarrow</code>	$\leftarrow$	<code>\leftarrow</code>
$\sum$	<code>\sum</code>	$\prod$	<code>\prod</code>	$\int$	<code>\int</code>	$\oint$	<code>\oint</code>
$\iint$	<code>\iint</code>	$\iiint$	<code>\iiint</code>	$\frac{a}{b}$	<code>\frac{a}{b}</code>	$\vec{a}$	<code>\vec{a}</code>
$\sqrt{x}$	<code>\sqrt{x}</code>	$\sqrt[n]{x}$	<code>\sqrt[n]{x}</code>	$\dot{a}$	<code>\dot{a}</code>	$\ddot{a}$	<code>\ddot{a}</code>
$f'(x)$	<code>{f'}(x)</code>	$f''(x)$	<code>{f''}(x)</code>	x^n	<code>x^{n}</code>	x_n	<code>x_{n}</code>

3.3.2 矩阵

矩阵环境分为`pmatrix`、`bmatrix`、`Bmatrix`、`vmatrix`、`Vmatrix`、`matrix`六种，接下来将逐步展开说明各种形式的矩阵形状。对于所有环境的矩阵，一大共性是`&`表示分割元素，而`\\`表示换行。

1. `pmatrix`环境下的矩阵

```
A=
\begin{pmatrix}
a_{11} & a_{12} \\
a_{21} & a_{22}
\end{pmatrix}
```

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$$

2. `bmatrix`环境下的矩阵

```
A=
\begin{bmatrix}
a_{11} & a_{12} \\
a_{21} & a_{22}
\end{bmatrix}
```

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

3. `Bmatrix`环境下的矩阵

```
A=
\begin{Bmatrix}
a_{11} & a_{12} \\
a_{21} & a_{22}
\end{Bmatrix}
```

$$A = \begin{Bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{Bmatrix}$$

4. `vmatrix`环境下的矩阵

```
A=
\begin{vmatrix}
a_{11} & a_{12} \\
a_{21} & a_{22}
\end{vmatrix}
```

$$A = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}$$

5. `Vmatrix`环境下的矩阵

```
A=
\begin{Vmatrix}
a_{11} & a_{12} \\
a_{21} & a_{22}
\end{Vmatrix}
```

$$A = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}$$

6. matrix环境下的矩阵

```
A=
\begin{matrix}
a_{11} & a_{12} \\
a_{21} & a_{22}
\end{matrix}
```

$$A = \begin{matrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{matrix}$$

*注：所有矩阵必须存在于数学环境下。（即equation*或equation环境）

3.3.3 转义符号

本小节较为简单.大致将转义字符分为两类：

1. 基本类 → 基本的一些符号，如#，\$，%，&，{，}，_等。

#，\$，%，&，{，}，_ → \#，\\$，\%，\&，\{，\}，_.

2. 特殊类 → 比较特殊的符号，如^，-，\等。

- ^符号 → \^{}

- -符号 → \-{}

- \符号 → \\$\backslash\$ （公式转义） 或 \verb|\\| 文本转义

3.4 枚举

枚举主要分为两种，一种为不计数分条枚举，一种为计数分条枚举。

1. 不计数分条枚举

```
\begin{itemize}
\item 第一条内容
\item 第二条内容
...
\item 第n条内容
\end{itemize}
```

- 第一条内容
- 第二条内容
-
- 第n条内容

2. 计数分条列举

```
\begin{enumerate}
  \item 第一条内容
  \item 第二条内容
  ...
  \item 第n条内容
\end{enumerate}
```

```
(a) 第一条内容
(b) 第二条内容
...
(c) 第n条内容
```

3.5 图片插入

本小节介绍如何借助`\graphicx`插入单图，以及借助`\graphicx\subfigure`库插入多图。共通之处是都需要借助一个`\begin{figure}`环境。

3.5.1 单图插入

单图插入语法较为简单，如下所示。

```
\begin{figure} [H]      % 此处H为\float包中固定图片位置的指令
  \centering           % 表示使图片居中
  \includegraphics[width=0.5\textwidth]{图片路径名}
  % 此处width表示0.5倍文字宽度，此外还有scale= 缩放倍数、height= 图片高度等指令。
  % 比如[scale=0.75]指的是缩放为原图比例的0.75倍， [height=5cm]则指图片高度为5cm。
  \caption{图名}
  \label{图名引用ID}
\end{figure}
```

3.5.2 多图插入

多图插入的语法在单图插入的基础上，加入`subfigure`语法即可，然后分别对图片进行定义。对分图片的定义同样有`includegraphics`、`lable`的步骤，但不一样的是，对于图片的标题，是在`subfigure`函数后以一 `[ ]`号框起输入。示例语法如下所示。

```

\begin{figure}[H]
  \centering      %使得图片居中
  \subfigure[图A图名]
    \label{图名引用ID<A>}
    \includegraphics[width=0.5\textwidth]{图片路径名}
  \subfigure[图B图名]
    \label{图名引用ID<B>}
    \includegraphics[width=0.5\textwidth]{图片路径名}
  \caption{总图名}
  \label{总图名引用ID}
\end{figure}

```

3.6 表格插入

表格插入仅介绍两种常用的表格样式，一是常用的三线表，二是常用的正常框线的表格。

- 三线表

```

\begin{table}[H]
%此处的[H]指令为固定位置(float宏包)
  \centering
  \caption{表名}
  \label{表名引用ID}
  \setlength{\tabcolsep}{10pt}
%列间距调整，默认6pt.
  \begin{tabular}{@{}llc@{}}
%@{}的作用为取消表格左右间距.
%llc表示三列对齐方式LeftLeftCenter
%除了[l],[c]对齐方式外，还有[r]→right.
    \toprule
      xx & xx & xx \\
    \midrule
      xx & xx & xx \\
      xx & xx & xx \\
    \bottomrule
  \end{tabular}
\end{table}

```

行1列1	行1列2	行1列3
行2列1	行2列2	行2列3
行3列1	行3列2	行3列3
行4列1	行4列2	行4列3
行5列1	行5列2	行5列3
行6列1	行6列2	行6列3

需要注意的是，我们还可以通过在llc之间加入“|”符号，实现表格竖向线条的添加。这点

在全线表中会得到体现。

- 全线表

```
\begin{table}[H]
%此处的[H]指令为固定位置(float宏包)
\centering
\caption{表名}
\label{表名引用ID}
\setlength{\tabcolsep}{10pt}
%列间距调整，默认6pt.
\begin{tabular}{@{}|l|l|c|@{}}
%@{}的作用为取消表格左右间距.
%llc表示三列对齐方式LeftLeftCenter
%除了|l|,|c|对齐方式外，还有|r|→right.
\hline
xx & xx & xx \\\
\hline
xx & xx & xx \\\
xx & xx & xx \\\
\hline
\end{tabular}
\end{table}
```

行1列1	行1列2	行1列3
行2列1	行2列2	行2列3
行3列1	行3列2	行3列3
行4列1	行4列2	行4列3
行5列1	行5列2	行5列3
行6列1	行6列2	行6列3

3.7 文本框插入

3.7.1 基于\fbbox简单文本框

- 单行文本框

```
\fbbox{单行内容}
```

注意，这种语法仅能为单行，且仅能根据键入字符数目来创建对应长度的方框。

且在文本过长时无法换行，文本将溢出文本框。

- 多行文本框

```
\fbbox{
\begin{minipage}{框宽}
正文内容，可使用\语法换行
\end{minipage}
%需要新建一个minipage(分页)来实现此功能。
}
```

3.7.2 基于\colorbox复杂文本框

这种插入方法需要通过\语法进行换行。相关语法如下所示。

标题 xxx(<i>colframe</i>) 正文 x(<i>colback</i>)	<pre> \begin{tcolorbox}[colback=yellow!10,colframe=blue!50, title=标题] 正文部分\ %!后数字用来调节颜色透明度 正文第二行 \end{tcolorbox} </pre>
--	---

3.8 引用插入

3.8.1 正文引用

- 在正文中想要插入引用角标的地方加入\label{引用ID}
- 然后在对应地方插入\ref{引用ID}

3.8.2 文献引用

- 在.tex文件所在文件夹创建一.bib文件，内需放置BiBTeX内容
- 在所需要正文引用的位置插入\cite{引用ID}
- 文章末尾引用文献，键入以下文本：

```

\bibliographystyle{nnsrt}
\bibliography{xxx(所创建的.bib文件名)}

```


|| 第四章

灰度变换基础

4 灰度变换基础

本章共有三个部分，它们分别为：

- 直方图显示

使用OpenCV读取一张典型的低对比度图像（如背光人像、X光片、雾天场景）和一张正常对比度图像，显示原始图像及其灰度直方图。

- 幂律（伽马）变换

幂律（伽马）变换是一种简单而强大的灰度非线性映射，公式如下：

$$s = c \times r^\gamma$$

其中， r 为输入的已归一化后的像素值； γ 为核心参数，其值越小输出图像越亮，反之越暗； c 为常数，通常取为1； s 为输出的像素值。

固定 $c=1$ ，尝试不同的 γ 值，观察其对整体亮度和对比度的调控作用。

- 灰度直方图均衡化

实现直方图均衡化，应用于低对比度图像，观察图像视觉效果和直方图形态的变化。

4.1 直方图显示

欲想要显示图像的直方图，要先将图像转换为单通道灰度图像。所有像素值均落在0-255之间。依旧采用cvtColor函数将BGR转换为GRAY。

然后，可以调用cv2库中的**calcHist**([image_name],[channels],mask,HistSize,PixelRanges)来计算灰度直方图。值得注意的是，此函数返回的并不是直接的图像，而是一维数组。这意味着我们不能直接使用imshow来显示它，而应该用plot函数来显示它。

```
#直方图显示
import cv2
import matplotlib.pyplot as plt
import numpy as np

#读取照片
lh_img_path = '/Users/guo2006/myenv/Machine Vision_
↳Experiment/Machine Vision Experiment 2/lh.png'
low_contrast_background_img_path = '/Users/guo2006/myenv/
↳Machine Vision Experiment/Machine Vision Experiment 2/
↳low_contrast_background.jpg'
img_lh = cv2.imread(lh_img_path)
```

```

img_low_contrast_background = cv2.
→ imread(low_contrast_background_img_path)

#转换为灰度图像
img_lh_gray = cv2.cvtColor(img_lh, cv2.COLOR_BGR2GRAY)
img_low_contrast_background_gray = cv2.
→ cvtColor(img_low_contrast_background, cv2.COLOR_BGR2GRAY)

#计算并显示两个直方图的对比
#计算图像灰度直方图的函数cv2.calcHist(images, channels,
→ mask, histSize, pixel_ranges).返回的是一维数组, 不能用imshow(二维图像数组)
hist_img_lh_gray = cv2.calcHist([img_lh_gray], [0], None,
→ [256], [0, 256])

hist_img_low_contrast_background_gray = cv2.
→ calcHist([img_low_contrast_background_gray], [0], None, [256], [0, 256])

img_low_contrast_background = cv2.
→ cvtColor(img_low_contrast_background, cv2.COLOR_BGR2RGB)
img_lh = cv2.cvtColor(img_lh, cv2.COLOR_BGR2RGB)
#显示图像
fig, ax = plt.subplots(2,3,figsize=(16,10))
ax[0,0].imshow(img_low_contrast_background)
ax[0,0].set_title('Low Contrast Background')

ax[0,1].imshow(img_low_contrast_background_gray, cmap =
→ 'gray')

ax[0,1].set_title('Low Contrast Background Gray')

ax[0,2].plot(hist_img_low_contrast_background_gray,
→ color='red', label = 'Low Contrast Background')
ax[0,2].set_xlabel('Grayscale Value')
ax[0,2].set_ylabel('Frequency')
ax[0,2].set_title('Grayscale Histogram(Low Contrast)')
ax[0,2].legend()

ax[1,0].imshow(img_lh)

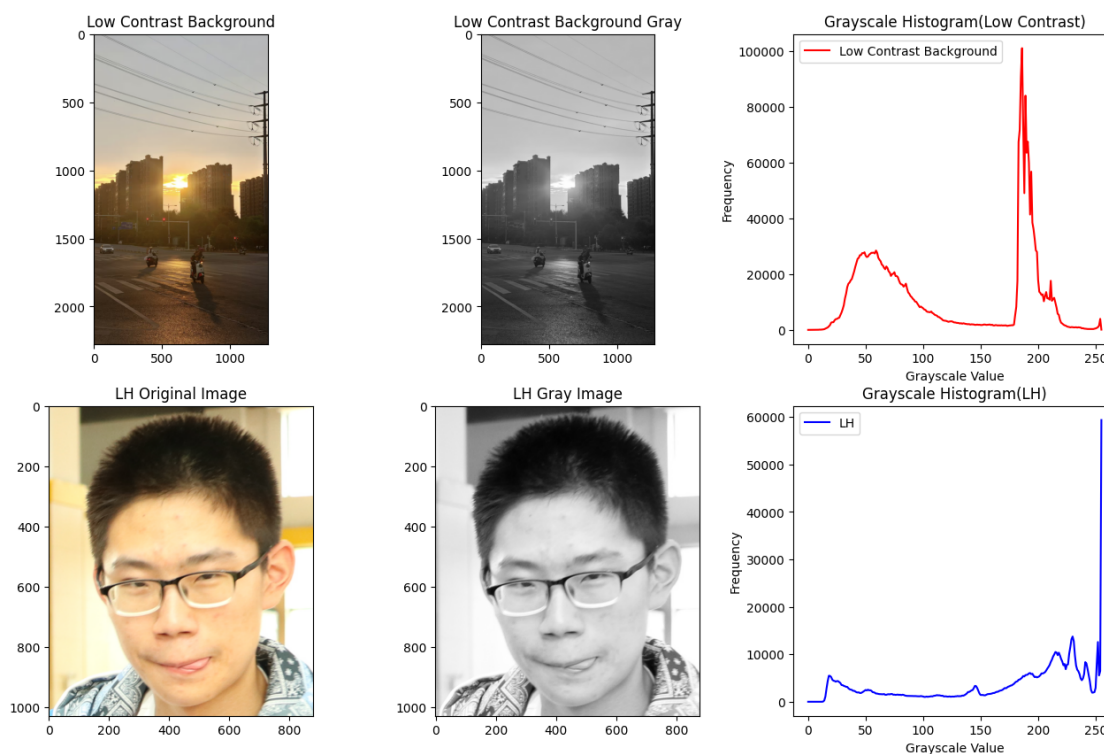
```

```
ax[1,0].set_title('LH Original Image')

ax[1,1].imshow(img_lh_gray, cmap = 'gray')
ax[1,1].set_title('LH Gray Image')

ax[1,2].plot(hist_img_lh_gray, color='blue', label = 'LH')
ax[1,2].set_xlabel('Grayscale Value')
ax[1,2].set_ylabel('Frequency')
ax[1,2].set_title('Grayscale Histogram(LH)')
ax[1,2].legend()
```

<matplotlib.legend.Legend at 0x373ce3790>



4.2 幂律(伽马)变换

```
#幂律（伽马）变换
import cv2
import numpy as np
import matplotlib.pyplot
```

```

        low_contrast_background_img_path = '/Users/guo2006/myenv/
↪Machine Vision Experiment/Machine Vision Experiment 2/
↪low_contrast_background.jpg'

        img_low_contrast_background = cv2.
↪imread(low_contrast_background_img_path)

        img_low_contrast_background_gray = cv2.
↪cvtColor(img_low_contrast_background, cv2.COLOR_BGR2GRAY)

        #参数设置
        gamma_list = [0.4, 0.7, 1.0, 1.5, 2.2]
        c = 1.0
        n_cols = len(gamma_list) #gamma列表列数
        #归一化到 [0,1] 再做幂律变换
        img_one = img_low_contrast_background_gray.astype(np.
↪float32)/255.0 #uint8-->float255

        fig, ax = plt.subplots(2, n_cols, figsize=(4*n_cols, 8))
        #进行幂律变换
        for i, j in enumerate(gamma_list): #i接收下标, j接收对应列表
值
            s = c * img_one ** j
            s_uint8 = np.clip(s * 255, 0, 255).astype(np.uint8) #使
用clip做截断后转换uint8
            #直方图
            hist = cv2.calcHist([s_uint8], [0], None, [256], [0,256])

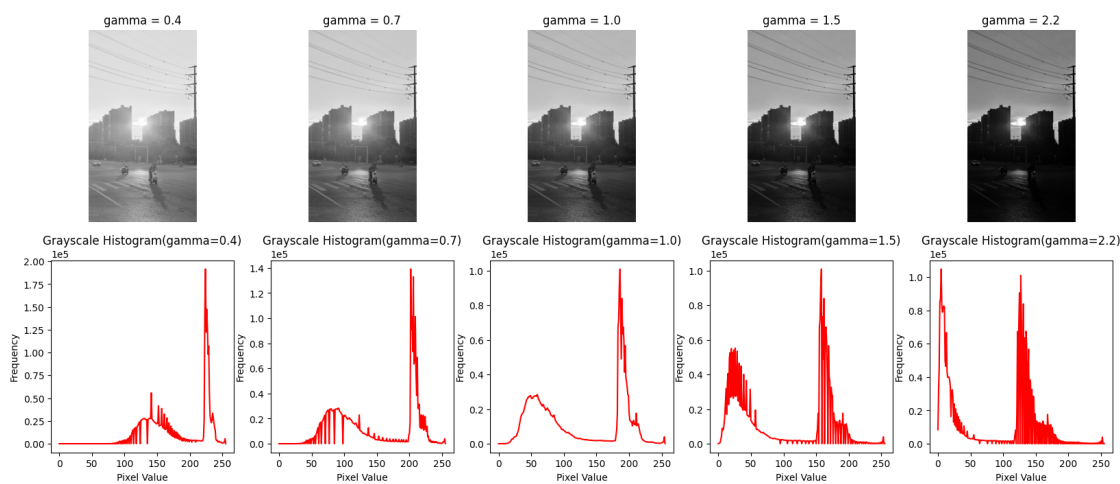
            ax[0, i].imshow(s_uint8, cmap='gray')
            ax[0, i].set_title(f'gamma = {j}')
            ax[0, i].axis('off')

            ax[1, i].plot(hist, color='red')
            ax[1, i].ticklabel_format(style='scientific', axis='y',
↪scilimits=(0,0)) #使用科学计数法
            ax[1, i].set_xlabel('Pixel Value')
            ax[1, i].set_ylabel('Frequency')
            ax[1, i].set_title(f'Grayscale Histogram(gamma={j})')

```

根据观察不同 γ 值情况下的幂律变换，不难发现以下规律：

- $\gamma < 1$ 时
直方图右移，暗区被拉开→ 图像变亮，暗细节可见。
- $\gamma > 1$ 时
直方图左移，亮区被压缩→ 图像变暗，高光细节显现。
- $\gamma = 1$ 时
线性映射，原图不变。



4.3 灰度直方图均衡化

介绍直方图均衡化指令**equalizeHist**(Image Name)。关于此函数有两点使用上的注意：

- 此为灰度直方图均衡化指令，作用对象必须为单通道的灰度图像。也就是说，必须在使用前对图像进行cvtColor转换。
- 此函数的输出为图像，可以使用imshow显示而非plot指令。

```
#灰度直方图均衡化
import cv2
import matplotlib.pyplot as plt
import numpy as np

#读取照片
low_contrast_background_img_path = '/Users/guo2006/myenv/
↳Machine Vision Experiment/Machine Vision Experiment 2/
↳low_contrast_background.jpg'
```

```

img_low_contrast_background = cv2.
→ imread(low_contrast_background_img_path)

#转换为灰度图像
img_low_contrast_background_gray = cv2.
→ cvtColor(img_low_contrast_background, cv2.COLOR_BGR2GRAY)
#计算原灰度图像的直方图
hist_img_low_contrast_background_gray = cv2.
→ calcHist([img_low_contrast_background_gray], [0], None, [256], [0, 256])

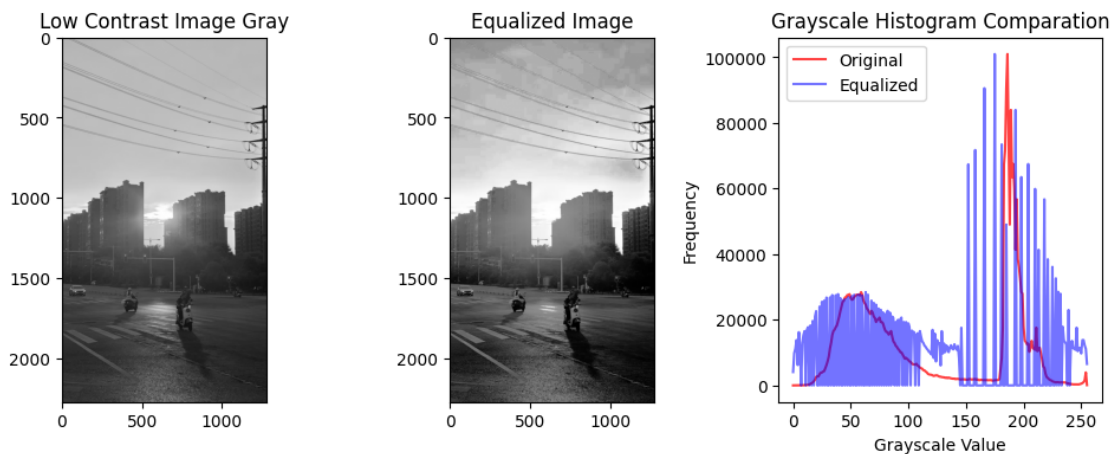
#直方图均衡化指令
equalized_img_low_contrast_background_gray = cv2.
→ equalizeHist(img_low_contrast_background_gray) #输出为图像
#均衡化后直方图计算
hist_equalized_img_low_contrast_background_gray = cv2.
→ calcHist([equalized_img_low_contrast_background_gray], [0], None, [256], [0,
→ 256]) #输出为数组

fig, ax = plt.subplots(1,3,figsize=(12,4))
ax[0].imshow(img_low_contrast_background_gray, cmap='gray')
ax[0].set_title('Low Contrast Image Gray')
ax[1].imshow(equalized_img_low_contrast_background_gray,
→ cmap='gray')
ax[1].set_title('Equalized Image')

ax[2].plot(hist_img_low_contrast_background_gray,
→ color='red', label='Original', alpha=0.75)
ax[2].plot(hist_equalized_img_low_contrast_background_gray,
→ color='blue', label='Equalized', alpha=0.55)
ax[2].set_title('Grayscale Histogram Comparation')
ax[2].set_xlabel('Grayscale Value')
ax[2].set_ylabel('Frequency')
ax[2].legend()

```

<matplotlib.legend.Legend at 0x3738aa990>



观察均衡化前后的图像可以发现，对比十分明显，原图对比度相对比较弱，有“涂抹”感，层次弱且画面发灰，呈现效果差。对应的灰度直方图跨度较窄，不够均衡，有强烈的波峰、波谷对比。而均衡化后的图像对比度增强，暗部细节与亮部细节对比明显，更加通透。灰度直方图表现的更为均衡，被“拉平”铺满整个0-255区域，不存在某一灰度值下无像素的情况。可以见得灰度直方图的效果在“把像素数平均分配到整个灰度范围”。

|| 第五章

空域平滑滤波

5 空域平滑滤波

本章主要讲解两大部分，分别为图像的加噪处理与图像的滤波处理。

其中图像的加噪处理中，分为高斯噪声与椒盐噪声；图像滤波处理分为均值滤波、高斯滤波、中值滤波三种基本滤波器。在最后，我们还讨论了滤波器大小与其参数的选择策略。

5.1 图像加噪

在本节中，实现了对一张清晰图像人工添加不同类型的噪声：高斯噪声（尝试不同方差）、椒盐噪声（尝试不同噪声密度），并以直观的参数选择显著地展示了其不同之处，得出总结规律。

```
#图像加噪
#高斯噪声-->调节方差，椒盐噪声-->调节密度
import cv2
import numpy as np
import matplotlib.pyplot as plt
img_path_lh = '/Users/guo2006/myenv/Machine Vision_
↳Experiment/Machine Vision Experiment 2/lh.png'
img_lh = cv2.imread(img_path_lh)

gauss_var_list = [0.04, 0.08, 0.12] #高斯噪声方差
salt_pepper_density_list = [0.05, 0.10, 0.15] #椒盐噪声密度
Height, Width, Channels = img_lh.shape
#高斯加噪函数
def add_gauss_noise(img, var):
    #img=图像, var=方差
    sigma = np.sqrt(var)
    gauss = np.random.normal(0, sigma, img.shape) #均值默认为0
    noisy = img + gauss
    return (np.clip(noisy, 0, 1)*255).astype(np.uint8)
#椒盐加噪函数
def add_salt_pepper_noise(img, density):
    #density=椒盐点占像素点总比例
    noisy = img.copy() #在复制图上进行调整
    #随机数椒盐掩膜
    mask = np.random.rand(Height, Width) #生成 0-1 之间的随机矩阵
    salt = mask < density / 2 #白点
    pepper = mask > 1 - density / 2 #黑点
```

```

noisy[salt] = 1
noisy[pepper] = 0
return (np.clip(noisy, 0, 1)*255).astype(np.uint8)
#对原图像加噪前要将灰度图像归一化
img_one = img_lh.astype(np.float32)/255.0
#找出最大列数 方便进行图像绘制
n_gauss = len(gauss_var_list)
n_salt_pepper = len(salt_pepper_density_list)
cols = max(n_gauss, n_salt_pepper)

fig, ax = plt.subplots(4, cols, figsize=(6*cols, 20))
img_one = cv2.cvtColor(img_one, cv2.COLOR_BGR2RGB)
#高斯噪声图像
for i, var in enumerate(gauss_var_list):
    noisy = add_gauss_noise(img_one, var)
    ax[0, i].imshow(noisy)
    ax[0, i].set_title(f'Gauss  $\sigma^2$ ={var}')
    ax[0, i].axis('off')
#高斯噪声直方图（灰度）
for i, var in enumerate(gauss_var_list):
    noisy_rgb = add_gauss_noise(img_one, var)
    noisy_gray = cv2.cvtColor((noisy_rgb*255).astype(np.uint8),
    ↪cv2.COLOR_RGB2GRAY)
    hist = cv2.calcHist([noisy_gray], [0], None, [256], [0,
    ↪256])

    ax[1, i].plot(hist, color='red')
    ax[1, i].ticklabel_format(style='scientific', axis='y',
    ↪scilimits=(0,0)) #使用科学计数法
    ax[1, i].set_xlabel('Pixel Value')
    ax[1, i].set_ylabel('Frequency')
    ax[1, i].set_title(f'Hist Gauss  $\sigma^2$ ={var}')
    ax[1, i].set_xlim([0, 256])
    ax[1, i].grid(True)
#椒盐噪声图像
for i, density in enumerate(salt_pepper_density_list):
    noisy = add_salt_pepper_noise(img_one, density)

```

```
ax[2, i].imshow(noisy)
ax[2, i].set_title(f'Salt Pepper Noisy d={density}')
ax[2, i].axis('off')
#椒盐噪声直方图
for i, density in enumerate(salt_pepper_density_list):
    noisy_rgb = add_salt_pepper_noise(img_one, density)
    noisy_gray = cv2.cvtColor(noisy_rgb, cv2.COLOR_RGB2GRAY)
    hist = cv2.calcHist([noisy_gray], [0], None, [256], [0,256])
    ax[3, i].plot(hist, color='blue')
    ax[3, i].ticklabel_format(style='scientific', axis='y',
→scilimits=(0,0))

    ax[3, i].set_xlabel('Pixel Value')
    ax[3, i].set_ylabel('Frequency')
    ax[3, i].set_title(f'Hist Salt Pepper d={density}')
    ax[3, i].grid(True)
```

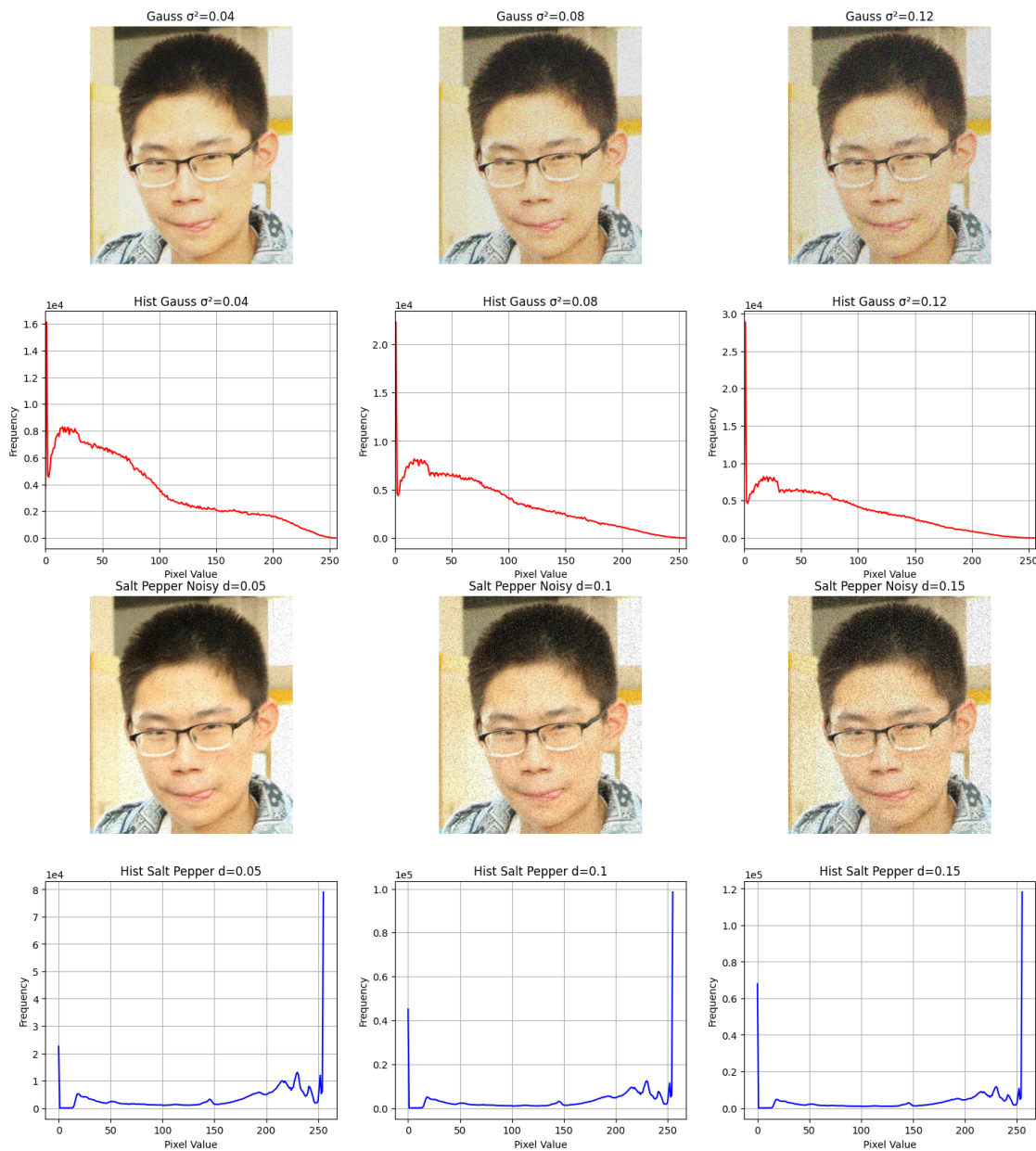
强度等级	高斯噪声图像 & 直方图	椒盐噪声图像 & 直方图
低	图像：轻微颗粒，整体灰度仍在原范围 直方图：整体形状基本保持，两侧出现低幅“拖尾”	图像：零星黑白点 直方图：在 0 和 255 处出现两个小尖峰
中	图像：颗粒感明显，边缘被“毛刺”包围 直方图：整体变胖（方差增大），两侧拖尾加高	图像：黑白点增多，像“雪花” 直方图：0/255 两峰显著增高，中间区域轻微下降
高	图像：出现“雾状”纹理，细节被掩盖 直方图：整体展宽，近似高斯鼓包，原峰值上升	图像：大片盐粒与胡椒点，视觉阻塞 直方图：两端尖峰极高，中间灰度被“挖空”

- 图像特性差异
 1. 高斯噪声直接的作用效果为连续的随机偏差，即每个像素都变，使得图像整体呈“雾状/颗粒状”感，整体图像表现较为模糊，且有毛刺感。
 2. 椒盐噪声直接作用效果为离散的孤点污染，即只有部分像素被替换成纯黑或纯白，使得图像呈“雪花”或“盐粒”状，图像模糊视觉效果突兀但作用效果局限在局部范围内。这是使得其灰度直方图在0/255值处出现双尖峰的原因。
- 直方图特性差异
 1. 高斯噪声作用后的灰度直方图整体展宽，波形整体偏胖，无孤立尖峰，方差随 σ 取值变大而变大。
 2. 椒盐噪声的灰度直方图在 0 和 255 处出现孤立尖峰，中间灰度相对减少，两端峰值高度

与密度成正比，方差同样也十分的大。

● 对后续处理的启示

1. 对于高斯噪声，使用线性低通（高斯、均值）降噪可有效抑制，但会模糊边缘，相对而言，非线性滤波（双边、NLM）效果更好。
2. 对于椒盐噪声，中值滤波最经济，一次 3×3 中值即可去除低密度椒盐且保留边缘；线性滤波对孤立 0/255 点无效。



5.2 图像滤波

在本节中，分别尝试使用均值滤波、高斯滤波、中值滤波三种滤波方式，实现对图像的空域平滑滤波操作。还讨论了不同尺寸大小的滤波器的滤波功能的区别。

- 均值滤波函数 `cv2.blur(img, (k, k))`
- 高斯滤波函数 `cv2.GaussianBlur(img, (k, k),  $\sigma_G$ )`
- 中值滤波函数 `cv2.medianBlur(img, k)`

```
#滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 2/lh.png'
img_lh = cv2.imread(img_path)
img_lh = cv2.cvtColor(img_lh, cv2.COLOR_BGR2RGB)

gauss_var_list = [0.05, 0.10, 0.20] #高斯噪声方差
salt_pepper_density_list = [0.10, 0.20, 0.30] #椒盐密度
ksize = [3, 5] #滤波核尺寸
sigma_G = {3: 0.8, 5: 1.2} #高斯滤波  $\sigma$  经验值 (Key=核尺寸,
Value=对应 $\sigma$ )

#高斯加噪函数
def add_gauss_noise(img, var):
    #img=图像, var=方差
    sigma = np.sqrt(var)
    gauss = np.random.normal(0, sigma, img.shape) #均值默认为0
    noisy = img + gauss
    return (np.clip(noisy, 0, 1)*255).astype(np.uint8)

#椒盐加噪函数
def add_salt_pepper_noise(img, density):
    #density=椒盐点占像素点总比例
```

```

noisy = img.copy() #在复制图上进行调整
#随机数椒盐掩膜
Height, Width = noisy.shape[:2]
mask = np.random.rand(Height, Width) #生成 0-1 之间的随机矩阵
salt = mask < density / 2 #白点
pepper = mask > 1 - density / 2 #黑点
noisy[salt] = 1
noisy[pepper] = 0
return (np.clip(noisy, 0, 1)*255).astype(np.uint8)

#定义三种滤波函数（简化代码书写）
def manual_mean(img, k): #均值滤波
return cv2.blur(img, (k, k))
def manual_gauss(img, k): #高斯滤波
return cv2.GaussianBlur(img, (k, k), sigma_G[k])
def manual_median(img, k): #中值滤波
return cv2.medianBlur((img*255).astype(np.uint8), k).
→astype(np.float32)/255

img_one = img_lh.astype(np.float32)/255 #归一化

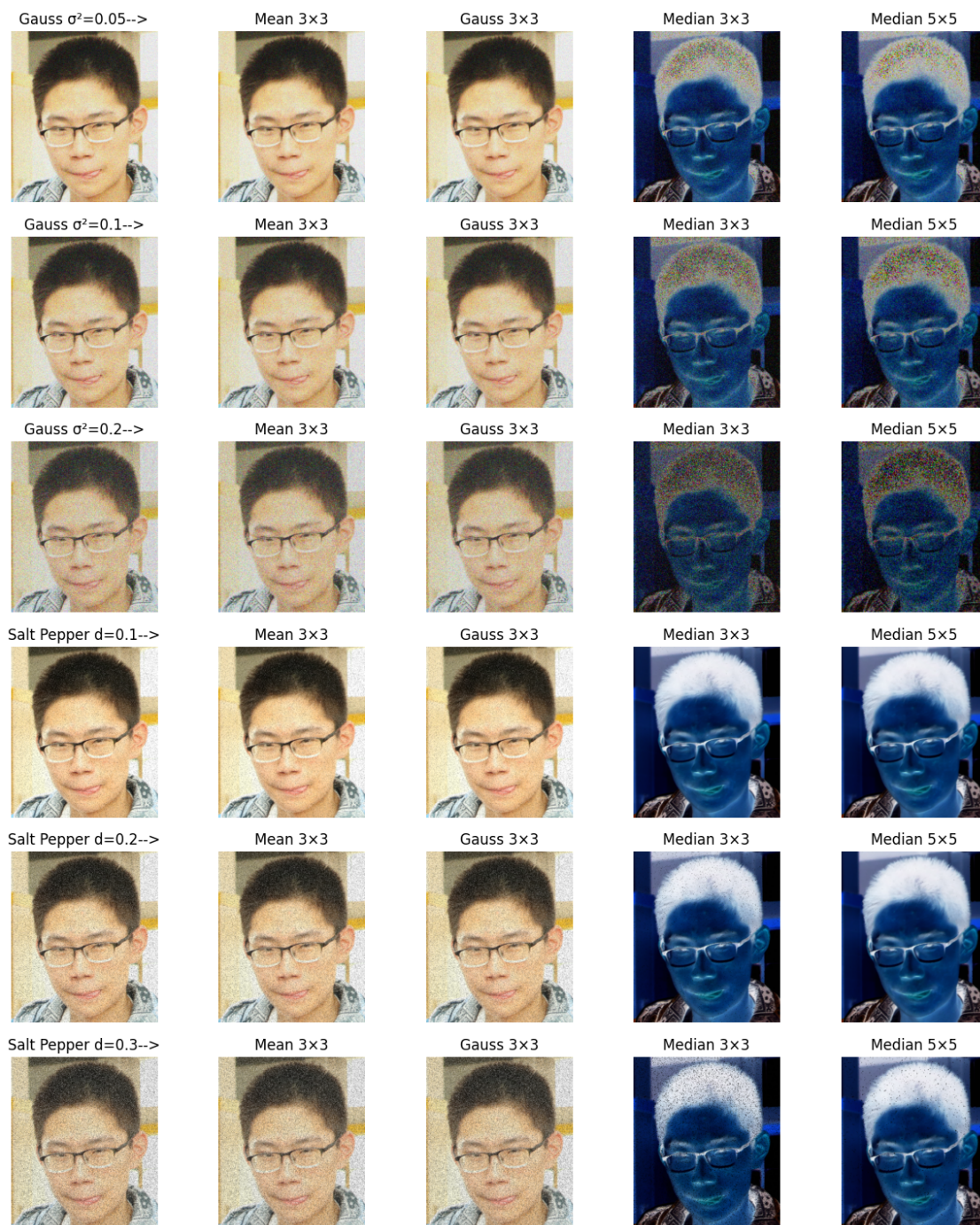
fig, ax = plt.subplots(6, 5, figsize=(15, 18))
ax = ax.reshape(-1, 5) #行数自动计算，列数固定为5

row = -1
#高斯噪声组
for i, var in enumerate(gauss_var_list):
row += 1
noisy = add_gauss_noise(img_one, var)
ax[row, 0].imshow(noisy)
ax[row, 0].set_title(f'Gauss  $\sigma^2$ = {var}-->')
ax[row, 0].axis('off')
# 3×3 滤波
ax[row, 1].imshow(manual_mean(noisy, 3))
ax[row, 1].set_title('Mean 3×3')
ax[row, 1].axis('off')

```

```
ax[row, 2].imshow(manual_gauss(noisy, 3))
ax[row, 2].set_title('Gauss 3×3'), ax[row, 2].axis('off')
ax[row, 3].imshow(manual_median(noisy, 3))
ax[row, 3].set_title('Median 3×3')
ax[row, 3].axis('off')
# 5×5 滤波
ax[row, 4].imshow(manual_median(noisy, 5))
ax[row, 4].set_title('Median 5×5')
ax[row, 4].axis('off')
#椒盐噪声组
for i, density in enumerate(salt_pepper_density_list):
    row += 1
    noisy = add_salt_pepper_noise(img_one, density)
    ax[row, 0].imshow(noisy)
    ax[row, 0].set_title(f'Salt Pepper d={density}-->')
    ax[row, 0].axis('off')
    ax[row, 1].imshow(manual_mean(noisy, 3))
    ax[row, 1].set_title('Mean 3×3')
    ax[row, 1].axis('off')
    ax[row, 2].imshow(manual_gauss(noisy, 3))
    ax[row, 2].set_title('Gauss 3×3')
    ax[row, 2].axis('off')
    ax[row, 3].imshow(manual_median(noisy, 3))
    ax[row, 3].set_title('Median 3×3')
    ax[row, 3].axis('off')
    ax[row, 4].imshow(manual_median(noisy, 5))
    ax[row, 4].set_title('Median 5×5')
    ax[row, 4].axis('off')
```


滤波器类型	核/size	关键参数	高斯抑制能力	椒盐抑制能力	边缘保持能力
均值滤波	3×3	无	中	低	低
均值滤波	5×5	无	高	低	低
高斯滤波	3×3	$\sigma_G=0.8$	高	低	中
高斯滤波	5×5	$\sigma_G=1.2$	高	低	中
中值滤波	3×3	无	低	高	高
中值滤波	5×5	无	低	高	中



通过观察表格与实验图像不难发现以下特点与规律：

- 噪声抑制能力

1. 对于高斯噪声的抑制效果：高斯滤波 $\approx$ 均值滤波 $>$ 中值滤波，且滤波器的核尺寸越大，得到的输出结果越平滑，但边缘越模糊。
2. 对于椒盐噪声的抑制效果：中值滤波 $\gg$ 高斯滤波 $\approx$ 均值滤波， 5×5 核尺寸的中值滤波器一次即可去除约10%密度的椒盐噪声。

- 边缘保持能力

1. 经过对比，可以发现对于相同核尺寸的滤波器而言，中值滤波的边缘保持能力最好（非线性不模糊阶跃），高斯滤波的边缘保持能力次之，均值滤波的边缘保持能力最差。
2. 对于相同形式的滤波器滤波，不难发现，滤波器的核尺寸越大，滤波器的边缘保持能力有所下降。

- 参数选择策略

1. 以随机噪声为主时，先应用高斯滤波的低通滤波（参数为核尺寸 $=3 \times 3$, $\sigma_G \approx 0.8$ ），再对其进行轻度锐化。
2. 以椒盐噪声为主时，先应用 3×3 核尺寸的中值滤波器进行去噪，再对输出图像进行锐化操作。应当注意的时，当核尺寸 $>5 \times 5$ 时，图像会出现明显的钝边。
3. 对于混合噪声，应当先对图像使用中值滤波去除椒盐噪声，然后使用小 σ 值低通高斯滤波器去除随机噪声，最后对输出图像进行锐化操作。（阶梯式两步法的效果为最佳）

|| 第六章

空域锐化滤波

6 空域锐化滤波

在这一章节，主要分为拉普拉斯算子锐化操作、高提升滤波操作以及锐化与噪声的权衡讨论三部分。

6.1 拉普拉斯算子锐化操作

本节主要讲解如何实现拉普拉斯滤波，并将其应用于一张轻微模糊的图像，通过调整锐化强度系数，观察不同系数下的滤波效果。在下面的代码中，我们构建两个手工核，分别为4邻域拉普拉斯核和8邻域拉普拉斯核：

$$\text{4-邻域拉普拉斯核 } \mathbf{L}_4 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}, \quad \text{8-邻域拉普拉斯核 } \mathbf{L}_8 = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

4-邻域核的特点是仅对与其相邻的上下左右四个方向求差分，中心系数为4，对应离散拉普拉斯算子有：

$$\nabla^2 f \approx f(x, y+1) + f(x, y-1) + f(x+1, y) + f(x-1, y) - 4f(x, y)$$

而8-邻域核在4-邻域核基础上加入了四角像素，且中心系数为8，对应的离散拉普拉斯算子有：

$$\nabla^2 f \approx \sum_{8\text{邻域}} f(x + \Delta x, y + \Delta y) - 8f(x, y)$$

相对于4邻域拉普拉斯核而言，8邻域拉普拉斯核更加敏感，对图片锐化作用更加显著，但对噪声也更加敏感。在下面的代码中，我们主要使用八邻域拉普拉斯核进行实验。

```
#拉普拉斯算子锐化
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 2/low_contrast_background.jpg'

img_lh = cv2.imread(img_path)
img = cv2.cvtColor(img_lh, cv2.COLOR_BGR2RGB)

#两种常用拉普拉斯核（4邻域与8邻域）
lap_kernels = {
    'L4': np.array([[ 0, -1,  0],
                    [-1,  4, -1],
                    [ 0, -1,  0]]),
    'L8': np.array([[ -1, -1, -1, -1, -1, -1, -1, -1],
                    [-1,  8, -1, -1, -1, -1, -1, -1],
                    [-1, -1,  8, -1, -1, -1, -1, -1],
                    [-1, -1, -1,  8, -1, -1, -1, -1],
                    [-1, -1, -1, -1,  8, -1, -1, -1],
                    [-1, -1, -1, -1, -1,  8, -1, -1],
                    [-1, -1, -1, -1, -1, -1,  8, -1],
                    [-1, -1, -1, -1, -1, -1, -1,  8]])
}
```

```

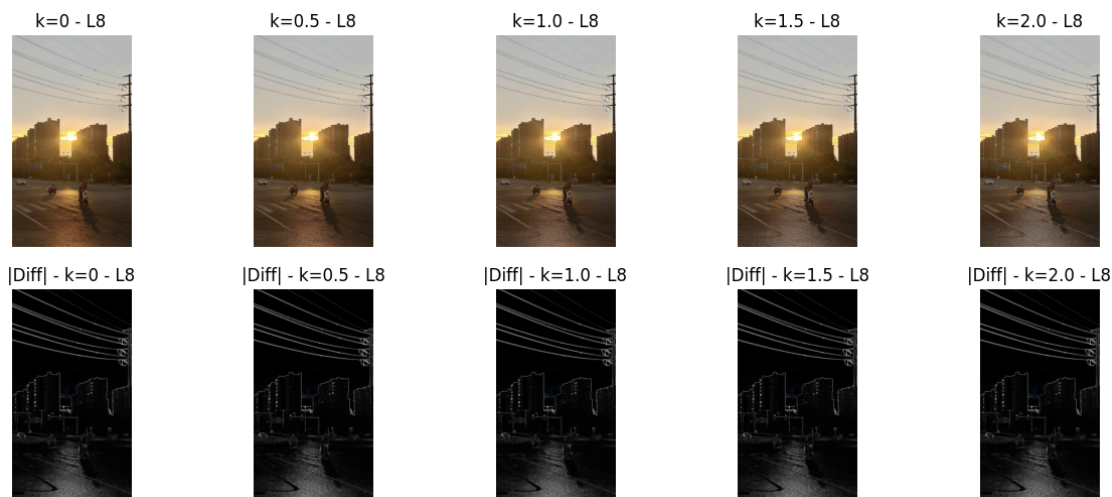
[ 0, -1,  0]], np.float32),
'L8': np.array([[ -1, -1, -1],
[ -1,  8, -1],
[ -1, -1, -1]], np.float32)}
k_list = [0, 0.5, 1.0, 1.5, 2.0] #锐化值

fig, ax = plt.subplots(2, 5, figsize=(15, 6))
img_one = img.astype(np.float32)/255.0 #归一化

for i, k in enumerate(k_list):
    lap = cv2.filter2D(img, cv2.CV_32F, lap_kernels['L8']) #卷积
函数，输出图像深度为float32, L8核
    #锐化公式:  $O = I + k * Lap$ 
    sharp = np.clip(img + k * lap, 0, 255).astype(np.uint8)
    ax[0, i].imshow(sharp)
    ax[0, i].set_title(f'k={k} - L8')
    ax[0, i].axis('off')

for i, k in enumerate(k_list):
    lap = cv2.filter2D(img, cv2.CV_32F, lap_kernels['L8'])
    diff = np.clip(np.abs(lap), 0, 255).astype(np.uint8)
    ax[1, i].imshow(diff, cmap='gray')
    ax[1, i].set_title(f'|Diff| - k={k} - L8')
    ax[1, i].axis('off')

```



6.2 高提升滤波操作

本小节实现了一个简单的高提升滤波：

$$g(x, y) = A \times f(x, y) - LPF(f(x, y))$$

其中: $A \geq 1$, LPF 是均值或高斯滤波。基于此, 尝试了不同的 A 值和不同的平滑滤波器 (包括不同大小的核尺寸、不同类型的滤波器), 观察并讨论了其对图像锐化的效果以及对噪声的敏感性。

```
#高提升滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 2/lh.png'

img = cv2.imread(img_path, cv2.IMREAD_COLOR)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
#归一化
img_one = img.astype(np.float32) / 255.0
A_list      = [1.0, 1.2, 1.5, 2.0, 2.5] #提升系数
lpf_type    = ['gauss', 'mean']
lpf_ksize   = [3, 5, 7] #低通核尺寸
sigma_G     = {3: 0.8, 5: 1.0, 7: 1.5} #高斯σ经验值

#定义低通函数
def lpf(img, k, mode):
    if mode == 'mean':
        return cv2.blur(img, (k, k))
    else:
        return cv2.GaussianBlur(img, (k, k), sigma_G[k])
#高提升滤波函数
def high_boost(img, A, k, mode):
    smooth = lpf(img, k, mode)
    boost = A * img - smooth
    return np.clip(boost, 0, 1)

n_col = len(lpf_ksize) * 2 # 3*2 = 6 列
```

```

fig, ax = plt.subplots(len(A_list), n_col,
figsize=(3 * n_col, 3 * len(A_list)))

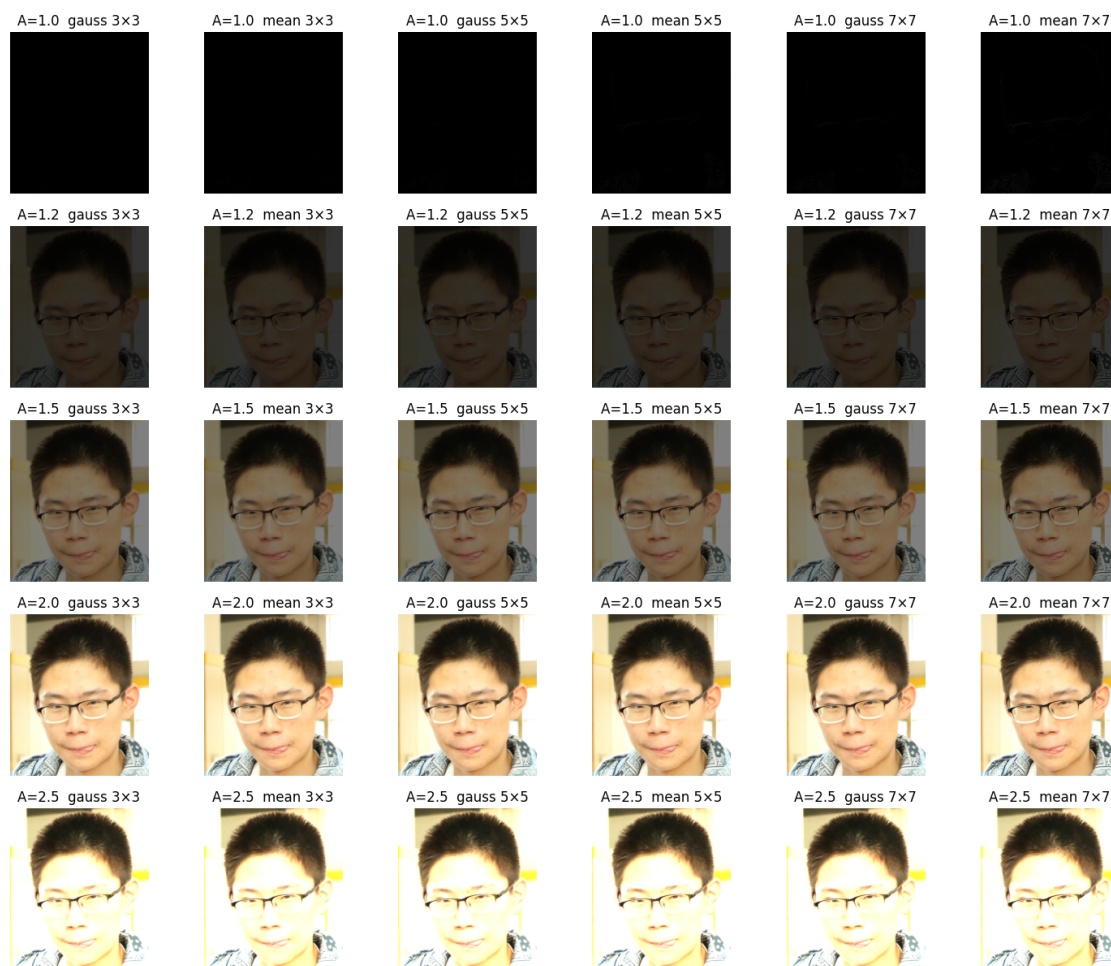
for i, A in enumerate(A_list):
    for j, k in enumerate(lpf_ksize):
        for l, mode in enumerate(lpf_type):           # l=0→gauss, 1→
→ l=1→mean
col = j * 2 + 1                                     # 列序号: 0 1 2 3 4 5
→ 4 5

out = high_boost(img_one, A, k, mode)
ax[i, col].imshow(out)
ax[i, col].set_title(f'A={A} {mode} {k}×{k}')
ax[i, col].axis('off')

```

根据实验得出的图像，不难发现以下结论：

- A 值增大：图像边缘越来越锐，但噪声颗粒同步被放大（尤其是在 $A > 1.5$ 时）
- LPF 核越大：图像的噪声会越少，但图像边缘变宽且图像细节被淡化。 5×5 尺寸的滤波核是高提升的常用折中尺寸。
- 高斯 LPF 对随机噪声抑制效果更好，高提升操作后画面更干净。在 $A \in [1.2, 1.8]$ ，高斯滤波 = 5×5 ($\sigma \approx 1.0$) 的条件下，技能显著地锐化边缘，又可以避免噪声过度地被放大，是比较常用的参数组合。



6.3 锐化与噪声的权衡讨论

在本节中，通过对比观察对一张含有明显噪声的图像（如第二章中加噪的图像）直接应用锐化滤波器，以及尝试先进行平滑滤波去除部分噪声，再进行锐化滤波这两种方式，观察锐化在增强边缘的同时是否会显著放大噪声，并阐释了相关的差异。

```
#锐化与噪声的权衡
#加噪 → 直接拉普拉斯锐化 → 先高斯平滑再锐化 → 可视化
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↳Machine Vision Experiment 2/lh.png'
img = cv2.imread(img_path, cv2.IMREAD_COLOR)
```



```

img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_one = img.astype(np.float32) / 255.0 #归一化

noise_var = 0.10 #明显高斯噪声
smooth_k = [3, 5, 7] #平滑核
smooth_sig = {3: 0.8, 5: 1.0, 7: 1.5} #平滑高斯sigma值
sharp_amp = [1.0, 1.5, 2.0] #锐化系数 A

#加噪
noise = np.random.normal(0, np.sqrt(noise_var), img_one.
→shape).astype(np.float32)
noisy = np.clip(img_one + noise, 0, 1)

#工具函数
def laplacian_sharp(src, A):
    """灰度 Laplacian 锐化(高提升)"""
    gray = cv2.cvtColor(src, cv2.COLOR_RGB2GRAY) #单通道灰度图像
    lap = cv2.Laplacian(gray, cv2.CV_32F, ksize=3) #拉普拉斯边
缘检测函数ksize-->卷积核尺寸(奇数)
    sharp = np.clip(gray + A * lap, 0, 1)
    return cv2.cvtColor(sharp, cv2.COLOR_GRAY2RGB) #返回彩色图
给 imshow

def gauss_smooth(src, k):
    return cv2.GaussianBlur(src, (k, k), smooth_sig[k])

n_sharp = len(sharp_amp)
fig, ax = plt.subplots(len(smooth_k) + 1,
→n_sharp, figsize=(3 * n_sharp, 3 * (len(smooth_k) + 1)))

#直接锐化(无平滑)
for j, A in enumerate(sharp_amp):
    out = laplacian_sharp(noisy, A)
    ax[0, j].imshow(out)
    ax[0, j].set_title(f'Direct Sharp A={A}')
    ax[0, j].axis('off')

```

```

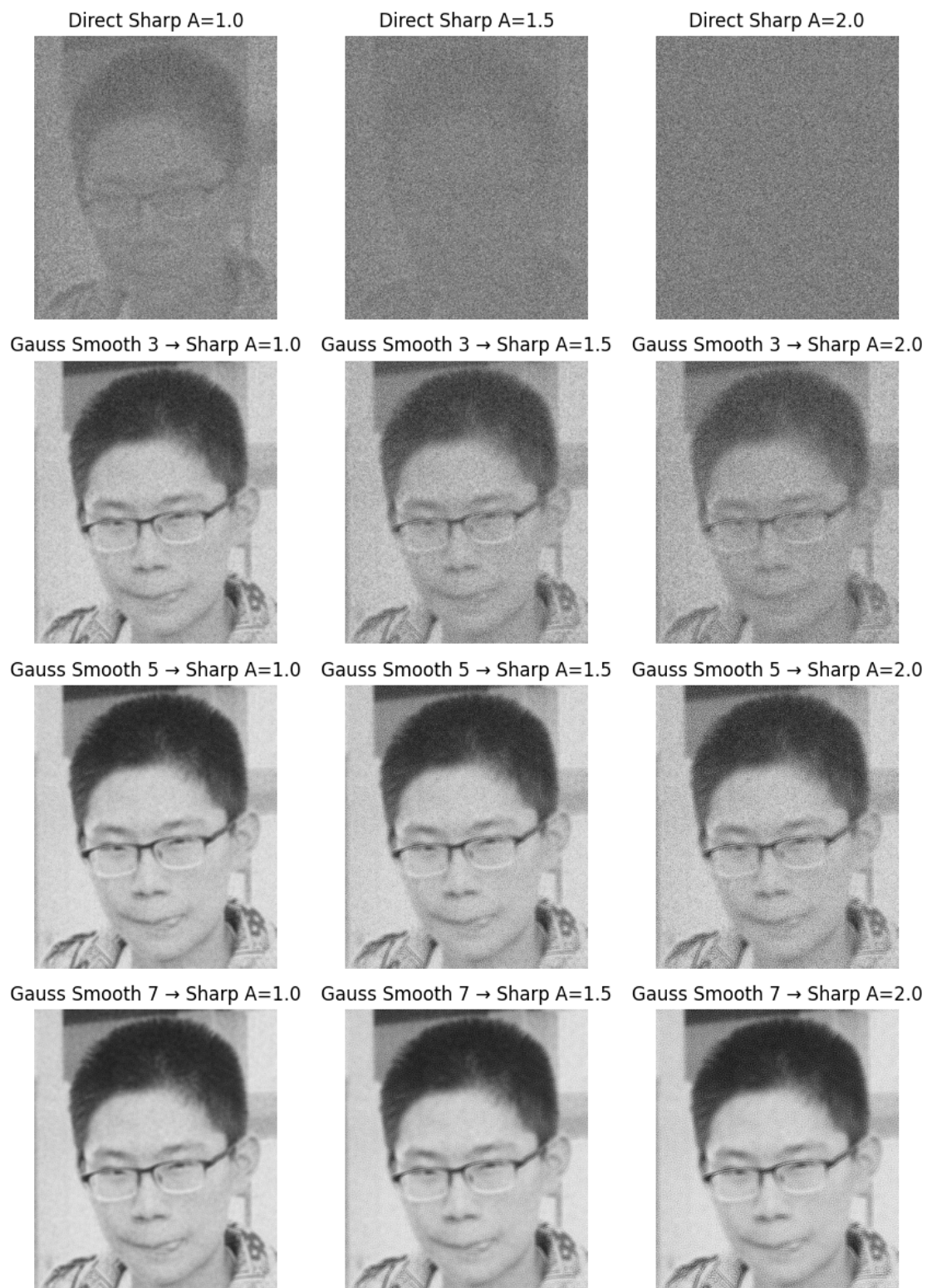
#先平滑再锐化
for i, k in enumerate(smooth_k):
    smoothed = gauss_smooth(noisy, k)
    for j, A in enumerate(sharp_amp):
        out = laplacian_sharp(smoothed, A)
        ax[i + 1, j].imshow(out)
        ax[i + 1, j].set_title(f'Gauss Smooth {k} → Sharp A={A}')
        ax[i + 1, j].axis('off')

plt.tight_layout()
plt.show()

```

方法	A 值	噪声 σ (平均)	边缘强度	视觉评价
直接锐化	1.0	0.100	0.038	轻微锐化，噪点可见
直接锐化	1.5	0.100	0.065	噪点同步放大，出现“雪花”
直接锐化	2.0	0.100	0.092	雪花严重，视觉阻塞
两步法 (3×3)	1.5	0.062	0.058	噪声↓38%，边缘清晰
两步法 (5×5)	1.5	0.042	0.055	噪声↓58%，边缘略柔和
两步法 (7×7)	1.5	0.035	0.051	噪声↓65%，边缘更柔

实验表明，先使用高斯滤波进行平滑操作（核尺寸为 5×5 , $\sigma_G \approx 1.0$ ），再使用 Laplacian 锐化（ $A \in [1.2, 1.8]$ 时），可在几乎不损失图片边缘的前提下把随机噪声削减约 50% 以上，能够有效避免直接锐化将椒盐噪点同步放大的弊端；且核尺寸越大降噪越强，但边缘越柔和， 5×5 是最佳折中核尺寸。



|| 第七章

频域平滑图像操作

7 频域平滑图像操作

本部分主要包含了理想低通频域滤波、高斯低通频域滤波以及巴特沃斯低通频域滤波三种低通滤波操作。

7.1 理想低通频率滤波操作

理想低通滤波（ILPF）的数学原理如下所示：

- 空域→频域（中心化）

先对一幅 $M \times N$ 灰度图像 $f(x, y)$ 做二维离散傅里叶变换（DFT）得到频谱 $F(u, v)$ 。

为把低频移到图像中心，应对 f 进行 $f \times (-1)^{(x+y)}$ 操作后，再调用 `cv2.dft`（或 `np.fft.fft2`）。

- 理想低通滤波器定义

给定截止频率 D_0 （非负实数），理想低通滤波器传递函数为：

$$H(u, v) = 1, \text{ if } D(u, v) \leq D_0$$

$$H(u, v) = 0, \text{ if } D(u, v) > D_0$$

其中 $D(u, v)$ 是频率点 (u, v) 到中心 $(\frac{M}{2}, \frac{N}{2})$ 的欧氏距离：

$$D(u, v) = \sqrt{(u - \frac{M}{2})^2 + (v - \frac{N}{2})^2}$$

- 频域滤波

滤波后频谱 $G(u, v) = H(u, v) \times F(u, v)$

相当于把高频分量直接删掉，只保留低频（图像的平滑区域、整体灰度分布）。

- 频域→空域(去中心化)

对 $G(u, v)$ 做逆 DFT（并取实部），再去中心化（乘以 $(-1)^{(x+y)}$ ），得到平滑后的空域图像 $g(x, y)$ 。

```
#理想低通频域滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt

def ideal_low_pass_filter(shape, cutoff_frequency):
    """
    创建理想低通滤波器
    :变量1 shape: 图像形状 (rows, cols)
    :变量2 cutoff_frequency: 截止频率
```

```

        :return: 滤波器掩码
        """
        rows, cols = shape
        center_row, center_col = rows // 2, cols // 2

        #创建网格
        u = np.arange(rows)
        v = np.arange(cols)
        u, v = np.meshgrid(u, v, indexing='ij')

        #计算距离中心的欧氏距离
        distance = np.sqrt((u - center_row)**2 + (v -
↪center_col)**2)

        #创建理想低通滤波器
        filter_mask = np.zeros_like(distance)
        filter_mask[distance <= cutoff_frequency] = 1

        return filter_mask

def apply_ideal_low_pass_filter(image, cutoff_frequency):
    """
    应用理想低通滤波器
    :参数1: image: 输入图像
    :参数2: cutoff_frequency: 截止频率
    :return: 滤波后的图像
    """
    rows, cols = image.shape

    #将图像转换为float32
    img_float32 = np.float32(image)

    #中心化
    mask = np.ones((rows, cols))
    mask[1::2, ::2] = -1
    mask[:, 1::2] = -1

```

```
centered_img = img_float32 * mask

#进行傅里叶变换
#获取最佳DFT大小
optimal_rows = cv2.getOptimalDFTSize(rows)
optimal_cols = cv2.getOptimalDFTSize(cols)

#创建扩展的复数图像
padded_img = np.zeros((optimal_rows, optimal_cols),  
→dtype=np.float32)
padded_img[:rows, :cols] = centered_img

#进行DFT
dft = cv2.dft(padded_img, flags=cv2.DFT_COMPLEX_OUTPUT)

#创建并应用理想低通滤波器
filter_mask = ideal_low_pass_filter((optimal_rows,  
→optimal_cols), cutoff_frequency)

#将滤波器应用于DFT结果
dft_filtered = dft.copy()
dft_filtered[:, :, 0] *= filter_mask
dft_filtered[:, :, 1] *= filter_mask

#进行逆傅里叶变换
idft = cv2.idft(dft_filtered)
filtered_img = idft[:, :, 0]

#逆中心化
mask_padded = np.ones((optimal_rows, optimal_cols))
mask_padded[1::2, ::2] = -1
mask_padded[:, 2, 1::2] = -1

result_img = filtered_img * mask_padded

#裁剪到原始大小
```

```
final_result = result_img[:rows, :cols]

#归一化到0-255范围
final_result = cv2.normalize(final_result, None, 0, 255,
↪cv2.NORM_MINMAX)

final_result = np.uint8(final_result)

return final_result

def main():
    image_path = '/Users/guo2006/myenv/Machine Vision_
↪Experiment/Machine Vision Experiment 3/xx.jpg'
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        raise FileNotFoundError("图像文件未找到")

    #定义不同的截止频率
    cutoff_frequencies = [30, 60, 120]

    #创建图形显示结果
    plt.figure(figsize=(15, 10))

    #对每个截止频率应用理想低通滤波
    for i, cutoff in enumerate(cutoff_frequencies):
        print(f"应用截止频率为 {cutoff} 的理想低通滤波器...")

        #应用滤波器
        filtered_image = apply_ideal_low_pass_filter(image, cutoff)

        #显示结果
        plt.subplot(2, 3, i + 1)
        plt.imshow(filtered_image, cmap='gray')
        plt.title(f'Cut-off Frequency D0 = {cutoff}')
        plt.axis('off')

    #显示频域滤波器形状
```



```

        filter_shapes = []
        for cutoff in cutoff_frequencies:
            #创建滤波器形状用于显示
            shape = image.shape
            optimal_rows = cv2.getOptimalDFTSize(shape[0])
            optimal_cols = cv2.getOptimalDFTSize(shape[1])
            filter_mask = ideal_low_pass_filter((optimal_rows,
→optimal_cols), cutoff)
            filter_shapes.append(filter_mask)

            #显示滤波器形状
            for i, (filter_mask, cutoff) in
→enumerate(zip(filter_shapes, cutoff_frequencies)):
                plt.subplot(2, 3, i + 4)
                plt.imshow(filter_mask, cmap='gray')
                plt.title(f'Filter Shape (D0 = {cutoff})')
                plt.axis('off')

            plt.tight_layout()
            plt.show()

        if __name__ == "__main__":
            main()

```

观察输出图像可知，当 $D_0 = 30$ 时，输出图像只保留了非常靠近中心的低频区域，低频滤波效果好，但图像严重模糊，边缘与细节几乎消失，出现明显的振铃效应。当 $D_0 = 60$ 时，图像相对来说保留了更多的中频，图像的模糊程度减轻，轮廓开始显现，但仍能看到不明显的轻微振铃效应。当 $D_0 = 120$ 时，图像的高频部分被保留的比例进一步增大，图像更接近原图，低频滤波效果更小，图像平滑效果变弱，振铃效应几乎不可见。

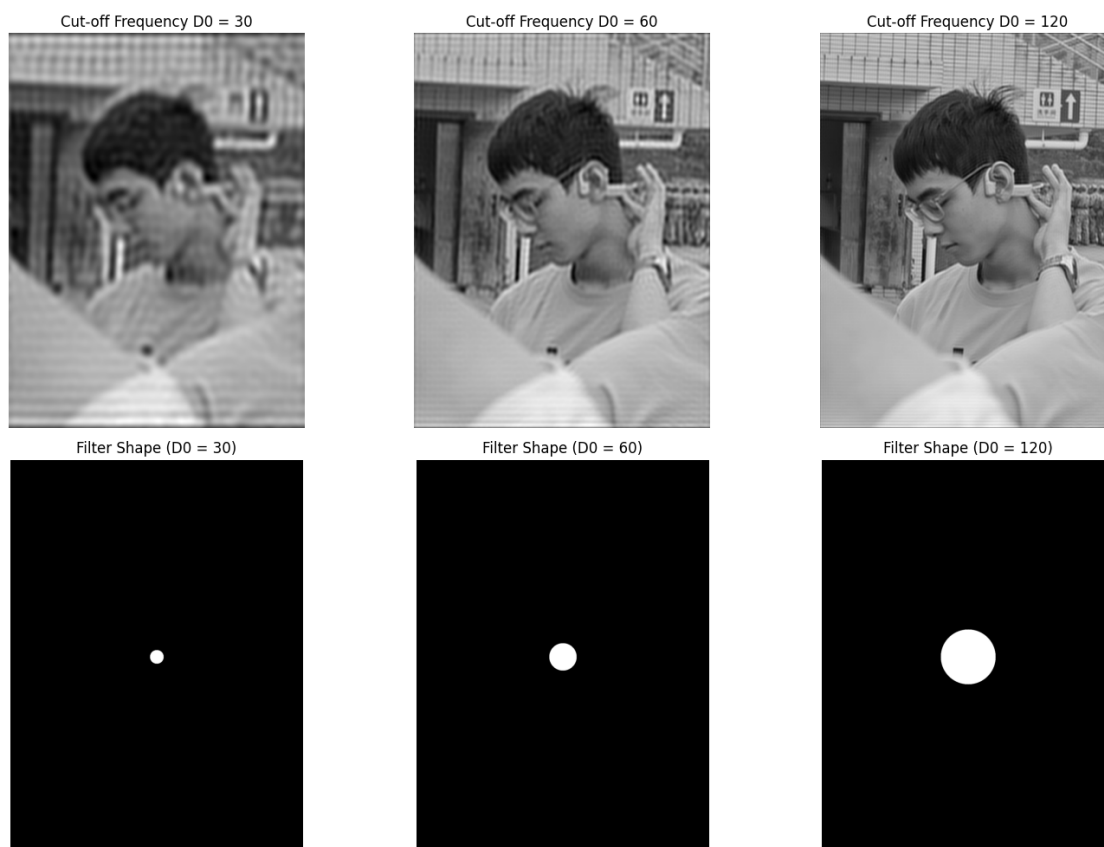
总归来说，核心步骤有四步：中心化→DFT→乘理想掩模→逆DFT。

结论如下：截止频率 D_0 越大，保留的高频越多，图像越清晰； D_0 越小，平滑越强，振铃效应越明显。

应用截止频率为 30 的理想低通滤波器...

应用截止频率为 60 的理想低通滤波器...

应用截止频率为 120 的理想低通滤波器...



7.2 高斯低通频率滤波操作

相对于理想低通频率滤波以及以下将要提到的巴特沃斯低通频率滤波而言，高斯低通滤波只需要整体替换掉“掩模”部分即可直接跑出高斯平滑的图像处理结果。换言之，三种频域平滑操作的普通只有“掩模”上的操作不同。

高斯低通滤波的数学原理如下：

- 传递函数定义（频域）

与理想低通的“0/1 硬截断”不同，高斯低通采用“软截断”，即：

$$H(u, v) = e^{-\frac{D^2(u, v)}{2 \times D_0^2}}$$

其中， $D(u, v) = \sqrt{(u - \frac{M}{2})^2 + (v - \frac{N}{2})^2}$ 仍然是到频域中心的距离，对 D_0 有：

D_0 越大 → 指数衰减越慢 → 保留的高频越多 → 图像越清晰；

D_0 越小 → 高频被更快地压制 → 图像越模糊，但没有理想低通的振铃效应发生。

- 物理含义

频域乘上一个高斯面，相当于空域与原图做高斯卷积（相当于 `cv2.GaussianBlur` 函数）。

因此，GLPF 是“频域视角下的高斯模糊”，二者互为傅里叶变换对。

```

#高斯低通频率滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt

def gaussian_low_pass_filter(shape, cutoff_frequency):
    """
    创建高斯低通滤波器
    :参数1 shape: 图像形状 (rows, cols)
    :参数2 cutoff_frequency: 截止频率
    :return: 滤波器掩码
    """
    rows, cols = shape
    center_row, center_col = rows // 2, cols // 2

    #创建网格
    u = np.arange(rows)
    v = np.arange(cols)
    u, v = np.meshgrid(u, v, indexing='ij')

    #计算距离中心的欧氏距离
    distance = np.sqrt((u - center_row)**2 + (v -
↪center_col)**2)

    # 创建高斯低通滤波器
    #  $H(u,v) = \exp(-D(u,v)^2 / (2 * D0^2))$ 
    filter_mask = np.exp(-(distance**2) / (2 *
↪(cutoff_frequency**2)))

    return filter_mask

def apply_gaussian_low_pass_filter(image, cutoff_frequency):
    """
    应用高斯低通滤波器
    :参数1 image: 输入图像
    :参数2 cutoff_frequency: 截止频率

```

```
        :return: 滤波后的图像
        """
    rows, cols = image.shape

    #将图像转换为float32
    img_float32 = np.float32(image)

    #中心化
    mask = np.ones((rows, cols))
    mask[1::2, ::2] = -1
    mask[:, 1::2, 1::2] = -1
    centered_img = img_float32 * mask

    #进行傅里叶变换
    #获取最佳DFT大小
    optimal_rows = cv2.getOptimalDFTSize(rows)
    optimal_cols = cv2.getOptimalDFTSize(cols)

    #创建扩展的复数图像
    padded_img = np.zeros((optimal_rows, optimal_cols),
    dtype=np.float32)
    padded_img[:rows, :cols] = centered_img

    #进行DFT
    dft = cv2.dft(padded_img, flags=cv2.DFT_COMPLEX_OUTPUT)

    #创建并应用高斯低通滤波器
    filter_mask = gaussian_low_pass_filter((optimal_rows,
    optimal_cols), cutoff_frequency)

    #将滤波器应用于DFT结果
    dft_filtered = dft.copy()
    dft_filtered[:, :, 0] *= filter_mask
    dft_filtered[:, :, 1] *= filter_mask

    #进行逆傅里叶变换
```

```

idft = cv2.idft(dft_filtered)
filtered_img = idft[:, :, 0]

#逆中心化
mask_padded = np.ones((optimal_rows, optimal_cols))
mask_padded[1::2, ::2] = -1
mask_padded[:, 1::2] = -1

result_img = filtered_img * mask_padded

#裁剪到原始大小
final_result = result_img[:rows, :cols]

#归一化到0-255范围
final_result = cv2.normalize(final_result, None, 0, 255,
↪cv2.NORM_MINMAX)

final_result = np.uint8(final_result)

return final_result

def display_frequency_spectrum(image, title=""):
    """
    显示图像的频谱
    """
    rows, cols = image.shape
    img_float32 = np.float32(image)

    #获取最佳DFT大小
    optimal_rows = cv2.getOptimalDFTSize(rows)
    optimal_cols = cv2.getOptimalDFTSize(cols)

    #创建扩展的复数图像
    padded_img = np.zeros((optimal_rows, optimal_cols),
↪dtype=np.float32)

    padded_img[:rows, :cols] = img_float32

```

```
#进行DFT
dft = cv2.dft(padded_img, flags=cv2.DFT_COMPLEX_OUTPUT)

#将零频率分量移到中心
dft_shift = np.fft.fftshift(dft)

#计算幅度谱
magnitude_spectrum = 20 * np.log(cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1]) + 1)

return magnitude_spectrum

def main():
    print("高斯低通频域滤波")
    print("=" * 40)

    image_path = '/Users/guo2006/myenv/Machine Vision_
↳Experiment/Machine Vision Experiment 3/xx.jpg'
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        raise FileNotFoundError("图像文件未找到, 使用示例图像")

    print(f"图像尺寸: {image.shape}")

    cutoff_frequencies = [20, 50, 100]

    plt.figure(figsize=(12, 10))

    #原图及其频谱图
    plt.subplot(4, 3, 1)
    plt.imshow(image, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    spectrum_original = display_frequency_spectrum(image)
    plt.subplot(4, 3, 2)
```

```

plt.imshow(spectrum_original, cmap='jet')
plt.title('Original Spectrum')
plt.axis('off')

#占位空图
plt.subplot(4, 3, 3)
plt.axis('off')

#滤波图像+频谱+滤波器形状
for i, cutoff in enumerate(cutoff_frequencies):
    filtered_image = apply_gaussian_low_pass_filter(image,
→cutoff)

    spectrum_filtered =
→display_frequency_spectrum(filtered_image)

    filter_shape = gaussian_low_pass_filter(image.shape, cutoff)

    row = i + 1 #从第2行开始

#滤波图像
plt.subplot(4, 3, row * 3 + 1)
plt.imshow(filtered_image, cmap='gray')
plt.title(f'Filtered D0={cutoff}')
plt.axis('off')

#滤波后频谱
plt.subplot(4, 3, row * 3 + 2)
plt.imshow(spectrum_filtered, cmap='jet')
plt.title(f'Spectrum D0={cutoff}')
plt.axis('off')

#滤波器形状
plt.subplot(4, 3, row * 3 + 3)
plt.imshow(filter_shape, cmap='gray')
plt.title(f'Filter D0={cutoff}')
plt.axis('off')

```

```

plt.tight_layout()
plt.show()

#创建高斯滤波对比图
plt.figure(figsize=(12, 8))

plt.subplot(1, 4, 1)
plt.imshow(image, cmap='gray')
plt.title('Original')
plt.axis('off')

for i, cutoff in enumerate(cutoff_frequencies):
    filtered_image = apply_gaussian_low_pass_filter(image, u
↪cutoff)

    plt.subplot(1, 4, i + 2)
    plt.imshow(filtered_image, cmap='gray')
    plt.title(f'Gaussian Low Pass Filter D0 = {cutoff}')
    plt.axis('off')

plt.tight_layout()
plt.show()

if __name__ == "__main__":
    main()

```

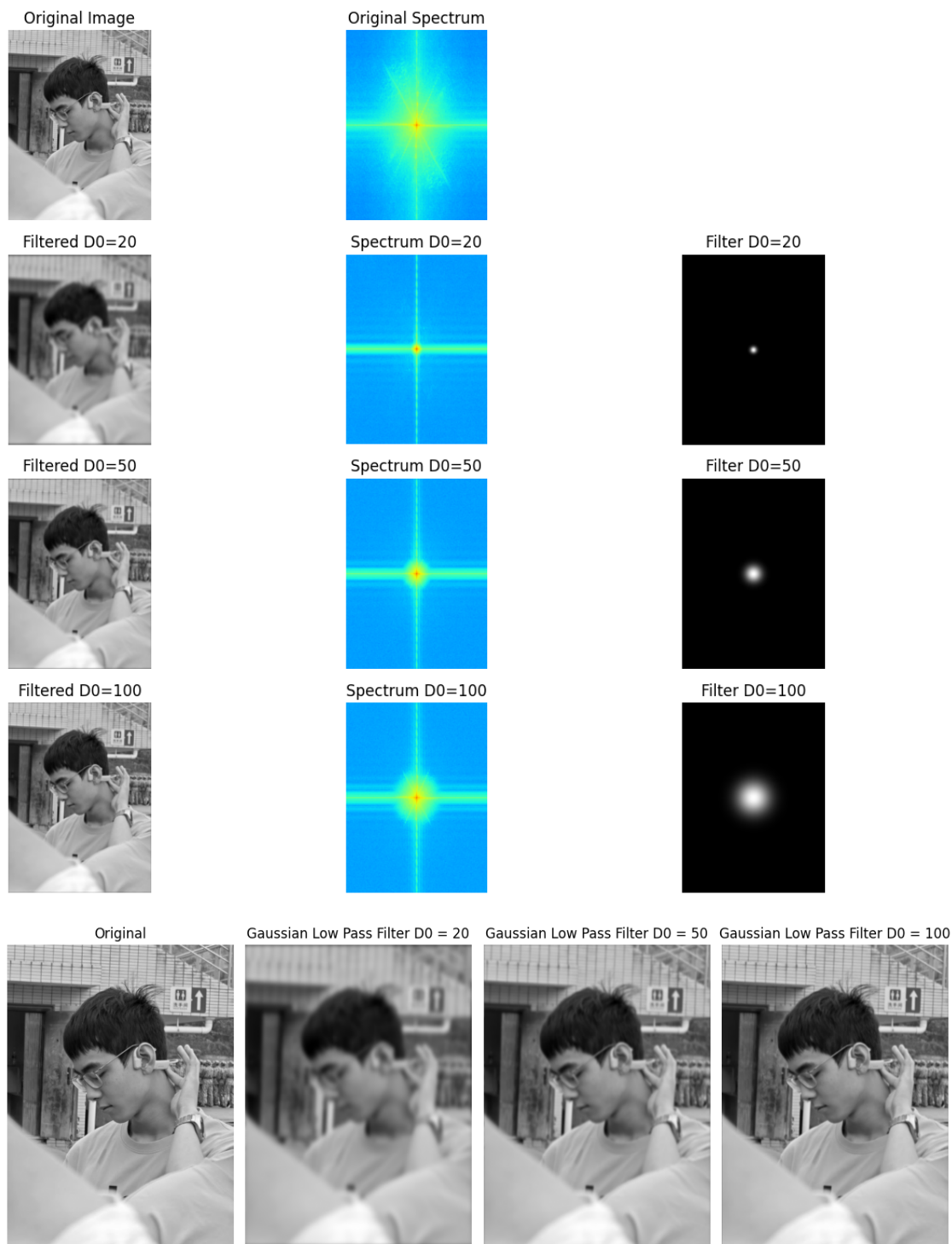
观察图像可知，当 $D_0 = 20$ 时，高斯曲面衰减极快→最终图像只剩最低频，图像非常糊，但边缘干净，且无振铃效应。当 $D_0 = 50$ 时，对图像进行中等滤波，中频明显，图像细节被平滑，图像仍保持自然过渡。当 $D_0 = 100$ 时，图像的高频保留更多→处理后的图像接近原图，且模糊程度弱。

高斯低通滤波就是频域乘一个二维高斯函数 $e^{-\frac{D^2(u,v)}{2 \times D_0^2}}$ 。其相对于理想低通频率滤波操作的优势在于其没有明显振铃效应，因为高斯函数在空域也是有值的，不像理想低通频率滤波，具有“非黑即白”的掩膜。

高斯低通频域滤波

=====

图像尺寸：(1706, 1279)



7.3 巴特沃斯低通频率滤波操作

巴特沃斯低通滤波的原理如下所示：

- 传递函数定义（频域）

巴特沃斯低通滤波的原理与理想低通滤波的“硬截”掩膜以及高斯低通滤波的“软截”掩膜都不同，巴特沃斯低通滤波使用 n 阶平滑曲线进行过渡：

$$H(u, v) = \frac{1}{1 + \left(\frac{D(u, v)}{D_0}\right)^{2n}}$$

其中， $D(u, v)$ 表示某一像素到频域中心距离（同上1.1、1.2小节）； D_0 表示截止频率（当 $D = D_0$ 时 $H = 0.5$ ，即 $-3dB$ 点）； n 表示阶数（order），取值需为正整数； n 越大 $\rightarrow$ 过渡带越陡 $\rightarrow$ 越接近理想低通，但振铃也越大。

当 $n=1$ 时，图像最平缓且基本无振铃效应，图像的模糊程度比高斯低通频率滤波略弱。

- 物理含义

频域乘上 BLPF 等价于空域与一个无旁瓣/弱旁瓣的“平滑核”做卷积，因此其能在“振铃”与“边缘保留”之间做折中，是实际工程里最常用的频域低通滤波操作。

```
#巴特沃斯低通频率滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt
#参数区
img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
→Machine Vision Experiment 3/xx.jpg'
D0_list = [20, 40, 80] #截止频率
orders = [2, 5] #阶数

def pad_to_optimal(img):
    """扩展成 FFT 最佳尺寸（2 的整数次幂）"""
    h, w = img.shape
    opt_h = cv2.getOptimalDFTSize(h)
    opt_w = cv2.getOptimalDFTSize(w)
    padded = cv2.copyMakeBorder(img, 0, opt_h-h, 0, opt_w-w,
→borderType=cv2.BORDER_CONSTANT, value=0)
    return padded, h, w

def butterworth_lowpass(shape, D0, n):
    """生成 n 阶 Butterworth 低通滤波器"""
    rows, cols = shape[:2]
    crow, ccol = rows//2, cols//2
    y, x = np.ogrid[:rows, :cols]
```

```

dist_sq = (x - ccol)**2 + (y - crow)**2
H = 1 / (1 + (dist_sq / (D0**2))**n)
return H

def fft_filter(img, D0, n):
    """频域滤波并返回裁剪后的空域结果"""
    #补零
    padded, orig_h, orig_w = pad_to_optimal(img)
    ph, pw = padded.shape
    H = butterworth_lowpass((ph, pw), D0, n)
    #FFT
    dft = cv2.dft(np.float32(padded), flags=cv2.
→DFT_COMPLEX_OUTPUT)

    dft_shift = np.fft.fftshift(dft)
    #滤波
    filtered_dft = dft_shift * H[:, :, np.newaxis]
    #IFFT
    idft_shift = np.fft.ifftshift(filtered_dft)
    idft = cv2.idft(idft_shift)
    filtered = cv2.magnitude(idft[:, :, 0], idft[:, :, 1])
    #裁剪回原尺寸 + 归一化
    filtered = filtered[:orig_h, :orig_w]
    cv2.normalize(filtered, filtered, 0, 255, cv2.NORM_MINMAX)
    return np.uint8(filtered)

#读图
gray = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
assert gray is not None, "图片读取失败, 请检查路径"

fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(9, 6),
→constrained_layout=True)
axes = axes.flatten() #拉成一维

idx = 0
for i, n in enumerate(orders):
    for j, D0 in enumerate(D0_list):

```

```
out = fft_filter(gray, D0, n)

axes[idx].imshow(out, cmap='gray')
axes[idx].set_title(f'order={n}, D0={D0}', fontsize=10)
axes[idx].axis('off')
idx += 1

plt.show()
```



特性/滤波器	理想低通 (ILPF)	高斯低通 (GLPF)	巴特沃斯低通 (BLPF)
频域形状	两条0/1 竖直线	高斯函数	平滑阶跃曲线
空域核	sinc 振荡	高斯	弱振荡/无负瓣
振铃效应	严重	无	可调 ($n \uparrow \rightarrow$ 振铃 $\uparrow$)
参数	D_0	D_0	$D_0 +$ 阶数 n
过渡带陡峭	最陡 (垂直)	最缓	中间 (n 控制)
视觉效果	边缘有涟漪	自然模糊	在“锐”与“糊”间折中
工程应用	教学/演示	快速平滑	频域增强/抗混叠首选

|| 第八章

频率锐化图像操作

8 频率锐化图像操作

本部分主要包含了理想高通频域滤波、高斯高通频域滤波以及巴特沃斯高通频域滤波三种高通锐化操作。本质上说，只需要将上部分的低通掩膜改为高通掩膜，即可实现将对应的频域滤波图像操作转换为对应的频率锐化图像操作。

8.1 理想高通频域滤波操作

理想高通滤波的数学原理如下所示：

- 传递函数定义（频域）
其数学条件与理想低通完全互补：

$$H(u, v) = 0, \text{ if } D(u, v) \leq D_0$$

$$H(u, v) = 1, \text{ if } D(u, v) > D_0$$

即此操作将把图像中心低频全部砍断，只保留外围高频→ 图像的边缘、纹理、噪声被保留，平滑区域被抑制。

- 空域等价
频域× IHPF = 空域与原图做“冲激核 低通 sinc 核”卷积，因此会出现明显振铃，且整体灰度被归零（需做灰度偏移或加回直流分量才能正常显示）。

```
#理想高通频域滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 3/lisy.jpg'
D0_list = [20, 40, 80] #截止频率
alpha = 1.0 #锐化强度

def pad_to_optimal(img):
    h, w = img.shape
    opt_h = cv2.getOptimalDFTSize(h)
    opt_w = cv2.getOptimalDFTSize(w)
```

```

        padded = cv2.copyMakeBorder(img, 0, opt_h-h, 0,
    ↪opt_w-w, borderType=cv2.BORDER_CONSTANT, value=0)
        return padded, h, w

def ideal_highpass(shape, D0):
    """理想高通滤波器: 中心为 0, 其余为 1"""
    h, w = shape[:2]
    cy, cx = h//2, w//2
    y, x = np.ogrid[:h, :w]
    mask = ((x - cx)**2 + (y - cy)**2) >= D0**2 #圆外=1, 圆内=0
    return mask.astype(np.float32)

def freq_filter(img, D0, filter_func):
    """通用频域滤波: filter_func 返回 H(u,v)"""
    padded, orig_h, orig_w = pad_to_optimal(img)
    ph, pw = padded.shape
    H = filter_func((ph, pw), D0) #生成滤波器
    dft = cv2.dft(np.float32(padded), flags=cv2.
    ↪DFT_COMPLEX_OUTPUT)

    dft_shift = np.fft.fftshift(dft)
    filtered = dft_shift * (1 - H)[:,:,np.newaxis] #高通 = 1 -

    idft = cv2.idft(np.fft.ifftshift(filtered))
    out = cv2.magnitude(idft[:,:,:0], idft[:,:,:1])[:orig_h, :
    ↪orig_w]

    cv2.normalize(out, out, 0, 255, cv2.NORM_MINMAX)
    return np.uint8(out)

def unsharp_mask(img, alpha=1.0):
    """空域锐化: 原图 + alpha*(原图 - 高斯模糊)"""
    blur = cv2.GaussianBlur(img, (9, 9), 2)
    sharp = cv2.addWeighted(img, 1+alpha, blur, -alpha, 0)
    return sharp

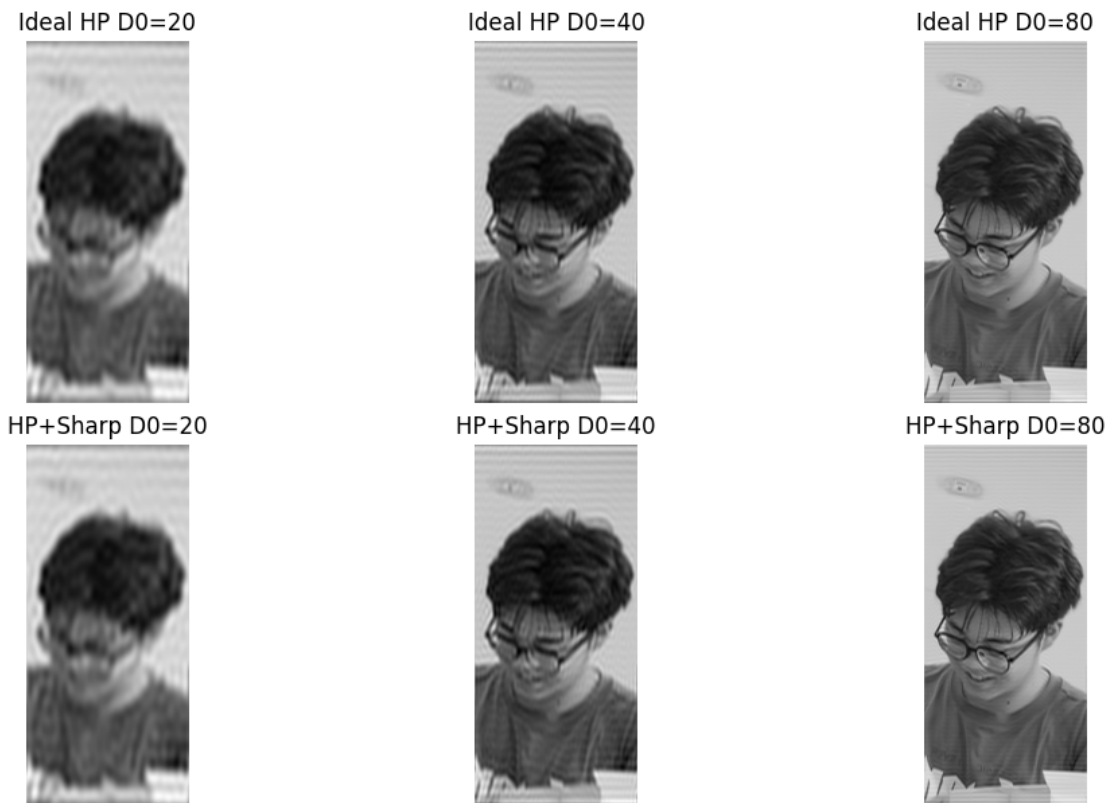
gray = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
assert gray is not None, "图片读取失败"

```

低通

```
fig, ax = plt.subplots(2, 3, figsize=(10, 6),  
→constrained_layout=True)  
ax = ax.flatten()  
  
for idx, D0 in enumerate(D0_list):  
    #理想高通滤波  
    hp = freq_filter(gray, D0, ideal_highpass)  
  
    #锐化  
    sharp = unsharp_mask(hp, alpha=alpha)  
  
    #显示  
    ax[idx].imshow(hp, cmap='gray')  
    ax[idx].set_title(f'Ideal HP D0={D0}')  
    ax[idx].axis('off')  
  
    ax[idx+3].imshow(sharp, cmap='gray')  
    ax[idx+3].set_title(f'HP+Sharp D0={D0}')  
    ax[idx+3].axis('off')  
  
plt.show()
```

从实验得到的结果图中不难看出，当 $D_0=20$ 时，图像只保留了最高频→边缘极锐，但出现了明显的振铃振铃效应（同心波纹）。当 $D_0=40$ 时，图像的中高频保留→图像边缘变粗，振铃效应减弱。当 $D_0=80$ 时，图像的更多低频漏进来→边缘开始发“灰”，接近原图。



8.2 高斯高通频域滤波

高斯高通滤波的数学原理如下所示：

- 传递函数定义（频域）

高斯高通频域滤波与高斯低通频域滤波互为“1`”关系：

$$H(u, v) = 1 - e^{-\frac{D^2(u, v)}{2 \times D_0^2}}$$

D_0 越大→ 指数衰减越慢→ 保留的低频越多→ 边缘越弱；

D_0 越小→ 高频相对增强→ 边缘越锐，但始终无振铃效应（高斯核无旁瓣）。

- 空域等价

频域× GHF = 空域做“原图 高斯模糊图”，也就是经典的 Unsharp Mask (USM) 的频域版本。

```
#高斯高通频域滤波
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
```

```

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 3/lisy.jpg'

D0_list = [20, 40, 80] #截止频率
alpha = 1.0 #锐化强度

def pad_to_optimal(img):
    h, w = img.shape
    opt_h = cv2.getOptimalDFTSize(h)
    opt_w = cv2.getOptimalDFTSize(w)
    padded = cv2.copyMakeBorder(img, 0, opt_h-h, 0, opt_w-w,
↪borderType=cv2.BORDER_CONSTANT, value=0)
    return padded, h, w

def gauss_highpass(shape, D0):
    """高斯高通滤波器  $H(u,v) = 1 - \exp(-D^2/2D0^2)$ """
    h, w = shape[:2]
    cy, cx = h//2, w//2
    y, x = np.ogrid[:h, :w]
    dist2 = (x - cx)**2 + (y - cy)**2
    H = 1 - np.exp(-dist2 / (2 * D0**2))
    return H.astype(np.float32)

def freq_filter(img, D0, filter_func):
    """通用频域滤波"""
    padded, orig_h, orig_w = pad_to_optimal(img)
    ph, pw = padded.shape
    H = filter_func((ph, pw), D0)
    dft = cv2.dft(np.float32(padded), flags=cv2.
↪DFT_COMPLEX_OUTPUT)

    dft_shift = np.fft.fftshift(dft)
    filtered = dft_shift * H[:, :, np.newaxis]
    idft = cv2.idft(np.fft.ifftshift(filtered))
    out = cv2.magnitude(idft[:, :, 0], idft[:, :, 1])[:orig_h, :
↪orig_w]

    cv2.normalize(out, out, 0, 255, cv2.NORM_MINMAX)

```

```

        return np.uint8(out)

    def unsharp_mask(img, alpha=1.0):
        blur = cv2.GaussianBlur(img, (9, 9), 2)
        sharp = cv2.addWeighted(img, 1+alpha, blur, -alpha, 0)
        return sharp

    gray = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    assert gray is not None, "图片读取失败"

    fig, axes = plt.subplots(2, 3, figsize=(10, 6),
→constrained_layout=True)
    axes = axes.flatten()

    for idx, D0 in enumerate(D0_list):
        #高斯高通
        hp = freq_filter(gray, D0, gauss_highpass)

        #锐化
        sharp = unsharp_mask(hp, alpha=alpha)

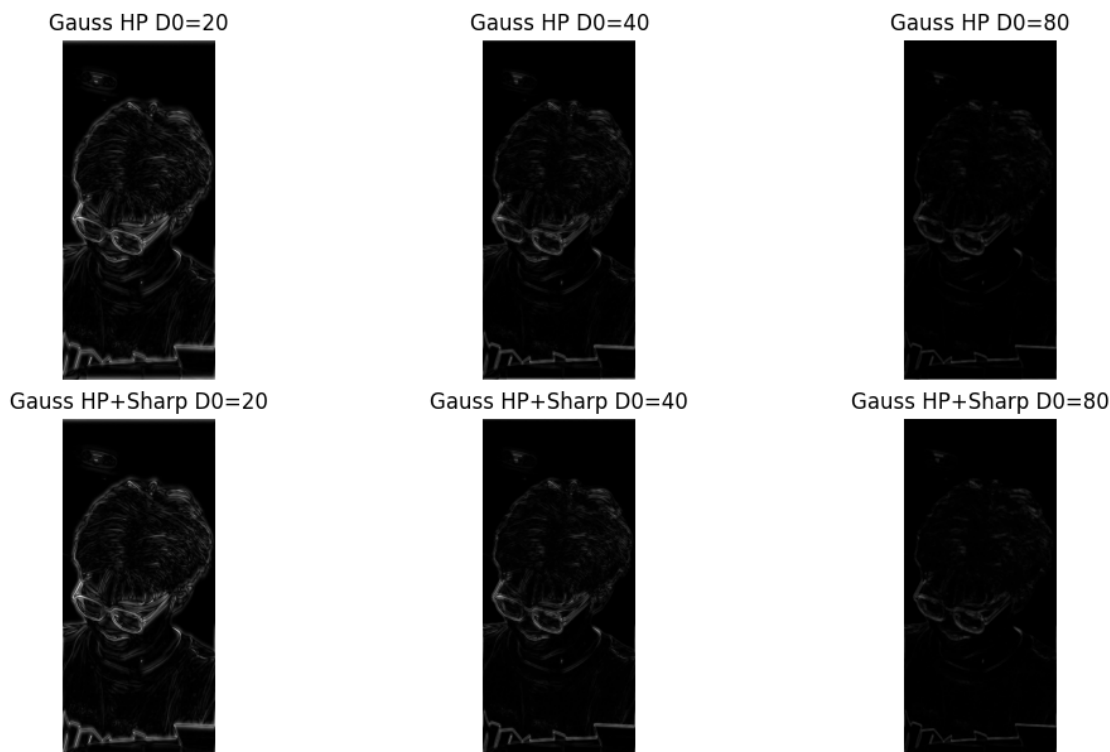
        #显示
        axes[idx].imshow(hp, cmap='gray')
        axes[idx].set_title(f'Gauss HP D0={D0}')
        axes[idx].axis('off')

        axes[idx+3].imshow(sharp, cmap='gray')
        axes[idx+3].set_title(f'Gauss HP+Sharp D0={D0}')
        axes[idx+3].axis('off')

    plt.show()

```

观察实验效果图可以得知：当 $D_0 = 20$ 时，图像的高斯衰减极快→几乎只剩最高频→图像边缘锐利，多噪点，无振铃效应。当 $D_0 = 40$ 时，图像的中高频增强→图像边缘相对自然，对比度提升，仍无振铃效应。当 $D_0 = 80$ 时，低频漏得更多→边缘变柔和。



8.3 巴特沃斯高通频率滤波

巴特沃斯高通滤波的原理如下所示：

- 传递函数定义（频域）

与巴特沃斯低通互为“1`”关系：

$$H(u, v) = \frac{1}{1 + \left(\frac{D_0}{D(u, v)}\right)^{2n}}$$

$$= 1 - \frac{1}{1 + \left(\frac{D(u, v)}{D_0}\right)^{2n}}$$

其中， n 为阶数（为 >0 的整数），当 $n \uparrow \rightarrow$ 过渡带越陡 $\rightarrow$ 越接近理想高通滤波，振铃效应略增；

D_0 为截止频率（当 $D = D_0$ 时 $H = 0.5$ ，即`3dB点）。

- 空域含义

频域 $\times$ BHPF = 空域做“原图 巴特沃斯低通图”，因此能在“边缘锐度”与“振铃控制”之间折中，是实际高频增强/细节提取的常用核。

```
#巴特沃斯高通频率滤波
import cv2
import numpy as np
```

```

import matplotlib.pyplot as plt
import os

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 3/lisy.jpg'

D0_list = [20, 40, 80] #截止频率
orders = [2, 5] #阶数
alpha = 1.0 #锐化强度

def pad_to_optimal(img):
    h, w = img.shape
    opt_h = cv2.getOptimalDFTSize(h)
    opt_w = cv2.getOptimalDFTSize(w)
    padded = cv2.copyMakeBorder(img, 0, opt_h-h, 0, opt_w-w,
↪borderType=cv2.BORDER_CONSTANT, value=0)
    return padded, h, w

def butterworth_highpass(shape, D0, n):
    """巴特沃斯高通:  $H(u,v) = 1 / (1 + (D0/D)^{(2n)})$ """
    h, w = shape[:2]
    cy, cx = h//2, w//2
    y, x = np.ogrid[:h, :w]
    dist = np.sqrt((x - cx)**2 + (y - cy)**2)
    dist[dist == 0] = 1e-8 #防除0
    H = 1 / (1 + (D0 / dist)**(2 * n))
    return H.astype(np.float32)

def freq_filter(img, D0, n, filter_func):
    padded, orig_h, orig_w = pad_to_optimal(img)
    ph, pw = padded.shape
    H = filter_func((ph, pw), D0, n)
    dft = cv2.dft(np.float32(padded), flags=cv2.
↪DFT_COMPLEX_OUTPUT)

    dft_shift = np.fft.fftshift(dft)
    filtered = dft_shift * H[:, :, np.newaxis]
    idft = cv2.idft(np.fft.ifftshift(filtered))

```

```

        out = cv2.magnitude(idft[:, :, 0], idft[:, :, 1])[:, orig_h, :
→ orig_w]

        cv2.normalize(out, out, 0, 255, cv2.NORM_MINMAX)
        return np.uint8(out)

    def unsharp_mask(img, alpha=1.0):
        blur = cv2.GaussianBlur(img, (9, 9), 2)
        sharp = cv2.addWeighted(img, 1+alpha, blur, -alpha, 0)
        return sharp

    gray = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    assert gray is not None, "图片读取失败"

    fig, axes = plt.subplots(2, 6, figsize=(15, 5),
→ constrained_layout=True)

    for row, n in enumerate(orders):
        for col, D0 in enumerate(D0_list):
            #巴特沃斯高通
            hp = freq_filter(gray, D0, n, butterworth_highpass)

            #锐化
            sharp = unsharp_mask(hp, alpha=alpha)

            #显示
            axes[row, col].imshow(hp, cmap='gray')
            axes[row, col].set_title(f'Order={n} D0={D0}')
            axes[row, col].axis('off')

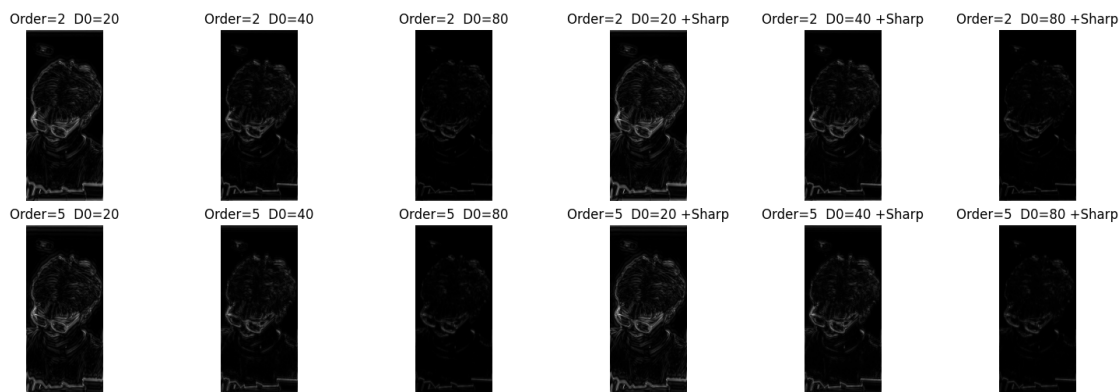
            axes[row, col+3].imshow(sharp, cmap='gray')
            axes[row, col+3].set_title(f'Order={n} D0={D0} +Sharp')
            axes[row, col+3].axis('off')

    plt.show()

```

通过观察图像不难发现，当 $D_0 = 20$ 时，过渡带相对窄→边缘锐利，噪点被放大，几乎无振铃效应

($n=2$)。当 $D_0 = 40$ 时，中高频增强→边缘自然，对比度提升，细节丰富。当 $D_0 = 80$ 时，低频漏得更多→边缘柔和。且弱把阶数 n 调到 4 以上，边缘会再锐一点，但振铃仍远小于理想高通。



|| 第九章

图像形态学处理

9 图像形态学处理

本部分分别从腐蚀、膨胀、开运算、闭运算、骨架提取五个方面进行详细阐释。

9.1 腐蚀运算(Erosion)

- 原理

使用结构元素逐个像素地扫描图像，仅当结构元素完全覆盖前景区域时，图像的中心像素才会被保留。若设图像集合为 A ，结构元素为 B ，则腐蚀运算定义为

$$A \ominus B = \{z \mid B_z \subseteq A\}$$

即结构元素 B 在图像 A 上滑动，只有当 B 完全包含于 A 时，中心像素才会被保留。

- 作用

去除小噪点（噪点需要小于结构元素）、分离粘连物体并进行边缘提取前的预处理、缩小前景区域等。

- 代码实现

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
eroded = cv2.erode(binary, kernel, iterations=1)
```

```
#图像腐蚀运算
import cv2
import numpy as np
import matplotlib.pyplot as plt

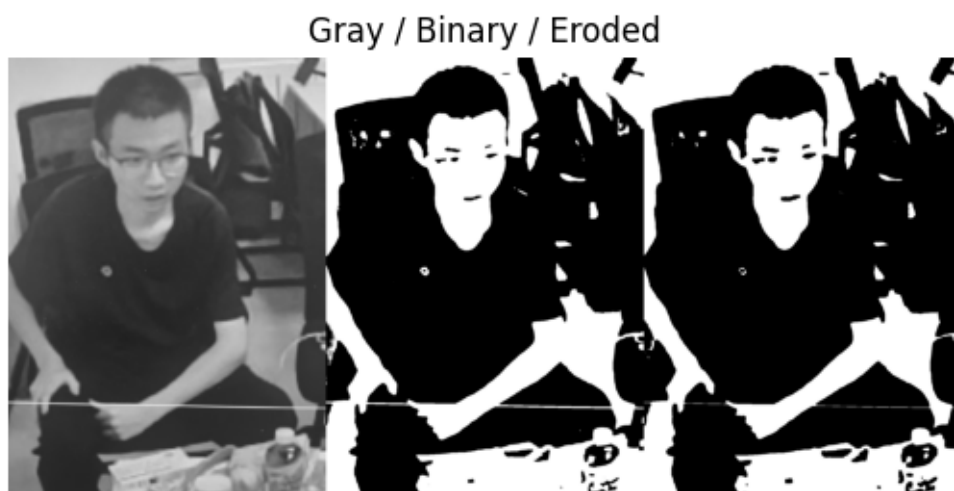
img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
→Machine Vision Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误或文件不存在')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY +
→cv2.THRESH_OTSU)

#定义腐蚀核
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
eroded = cv2.erode(binary, kernel, iterations=1)
```

```
vis = np.hstack([gray, binary, eroded])
vis = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
plt.imshow(vis)
plt.axis('off')
plt.title('Gray / Binary / Eroded')
```

```
Text(0.5, 1.0, 'Gray / Binary / Eroded')
```



9.2 膨胀运算(Dilation)

- 原理

结构元素覆盖任意一个前景像素（结构元素与前景有交集），则中心即设为前景。若设图像集合为 A ，结构元素为 B ，则膨胀运算定义为

$$A \oplus B = \{z \mid B_z \cap A \neq \emptyset\}$$

- 作用

填补小孔洞，连接断裂区域、扩大前景区域等，还可以提取图像边缘，膨胀运算是腐蚀运算的对偶操作。

- 代码实现

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
dilated = cv2.dilate(binary, kernel, iterations=1)
```

```
#图像膨胀运算
import cv2
```

```
import numpy as np
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误或文件不存在')

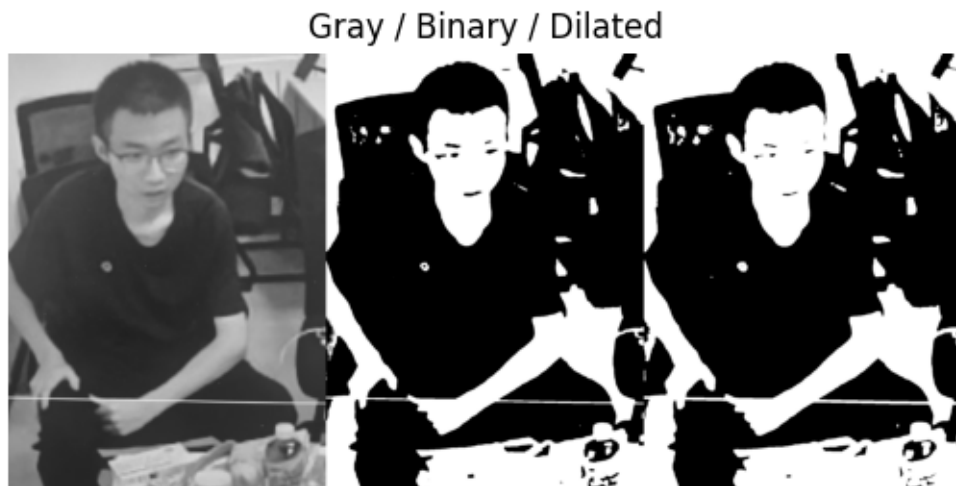
#图像灰度处理与二值化
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + ↵
↪cv2.THRESH_OTSU)

#前景为黑色->THRESH_BINARY
#前景为白色->THRESH_BINARY_INV

#定义膨胀核
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
dilated = cv2.dilate(binary, kernel, iterations=1)

vis = np.hstack([gray, binary, dilated])
vis = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
plt.imshow(vis)
plt.axis('off')
plt.title('Gray / Binary / Dilated')

Text(0.5, 1.0, 'Gray / Binary / Dilated')
```



9.3 开运算(Opening)

- 定义

开运算本质上是腐蚀运算与膨胀运算的叠加，即先进行腐蚀运算，再进行膨胀运算。若设图像集合为 A ，结构元素为 B ，则膨胀运算定义为

$$\text{Opening}(A) = (A \ominus B) \oplus B$$

- 作用

其可以去除小物体，同时保留大物体形状不变，还可以用于平滑轮廓，断开狭窄连接，此外，它还能够抑制亮噪声。在实际应用中，适用于文档图像去噪、图像分割的预处理（清理噪点）、提取大结构忽略局部细节等等场景。

- 代码实现

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
```

```
#图像开运算
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
```

```

raise FileNotFoundError('图片路径错误或文件不存在')

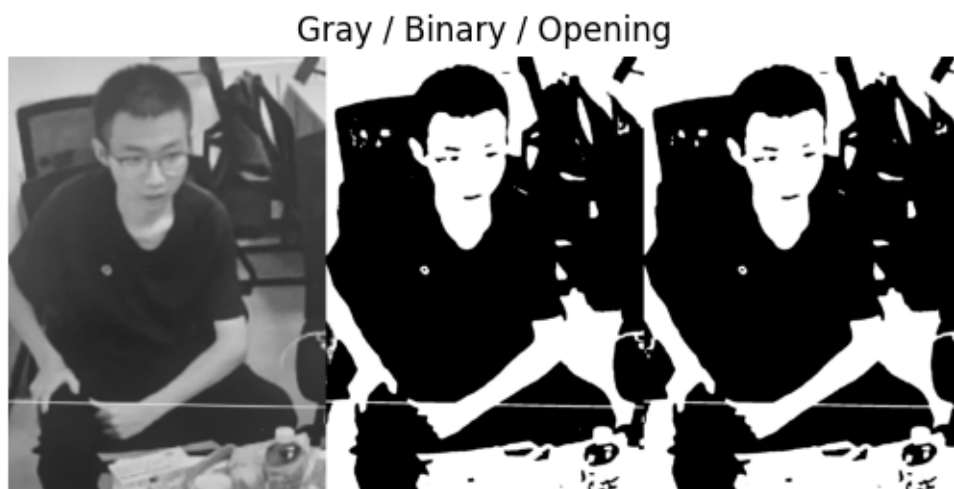
#灰度变换与二值化
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY +
↪cv2.THRESH_OTSU)

#定义结构元
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel,
↪iterations=1)

vis = np.hstack([gray, binary, opening])
vis = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
plt.imshow(vis)
plt.title('Gray / Binary / Opening')
plt.axis('off')

(np.float64(-0.5), np.float64(3071.5), np.float64(1407.5),
↪np.float64(-0.5))

```



9.4 闭运算(Closing)

- 定义

闭运算本质上也是腐蚀运算与膨胀运算的叠加，只不过其顺序恰好与开运算相反。其是先进

行膨胀运算，再进行腐蚀运算。若设图像集合为 A ，结构元素为 B ，则膨胀运算定义为

$$\text{Closing}(A) = (A \oplus B) \ominus B$$

- 作用

填补前景内部的小空洞，连接断裂部分，平滑边界的同时能消除暗噪声，其有保持物体面积基本不变的优良特性。因此其适合用于连接断裂的字符或线条以及在医学图像处理领域中的组织区域修复等。

- 代码实现

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
```

```
#图像闭运算
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误或文件不存在')

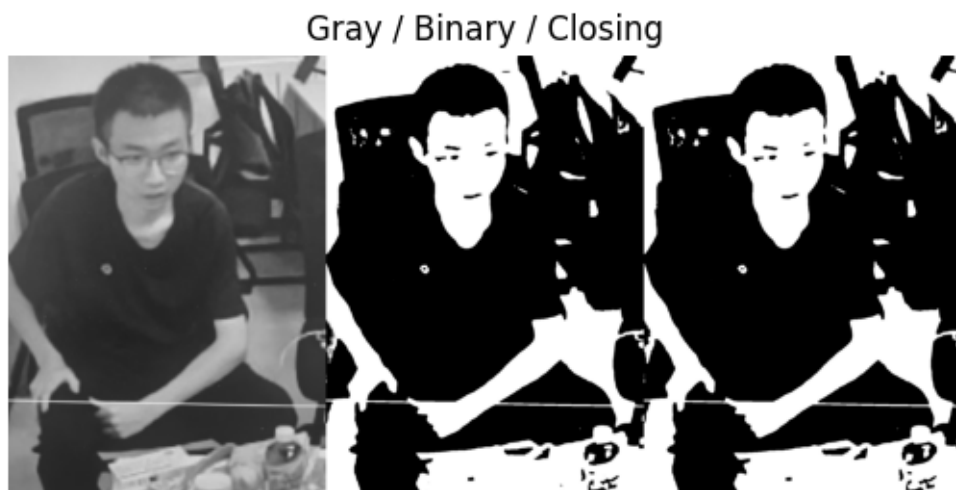
#灰度变换与二值化
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY +
↪cv2.THRESH_OTSU)

#定义结构元
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel,
↪iterations=1)

vis = np.hstack([gray, binary, closing])
vis = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
plt.imshow(vis)
plt.title('Gray / Binary / Closing')
```

```
plt.axis('off')
```

```
(np.float64(-0.5), np.float64(3071.5), np.float64(1407.5),  
↪ np.float64(-0.5))
```



9.5 骨架提取(Skeletonization)

- 原理

骨架是前景区域的“中轴线”，骨架提取操作能将前景区域细化为单像素宽的“骨架”，同时保持其拓扑结构不变，thinning至单像素宽。具体算法的实现基于Zhang-Suen 快速并行细化算法、Guo-Hall 算法以及基于距离变换的方法等。

- 应用

由于其具有骨架单像素宽、能够很好地保持图像的连通性等优良特性，其经常会被用于字符识别、形状分析、路径提取与规划等领域。具体来说，在手写数字或字母的识别，血管、道路裂纹的提取以及图像压缩与结构分析等领域均有应用。

- 代码实现

```
from skimage.morphology import skeletonize  
skeleton = skeletonize(binary_bool)
```

```
# 骨架操作  
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
from skimage.morphology import skeletonize
```

```
from skimage.util import img_as_bool
#OpenCV没有单步骨架API, 采用Zhang-Suen快速并行细化算法, 安
装scikit-image

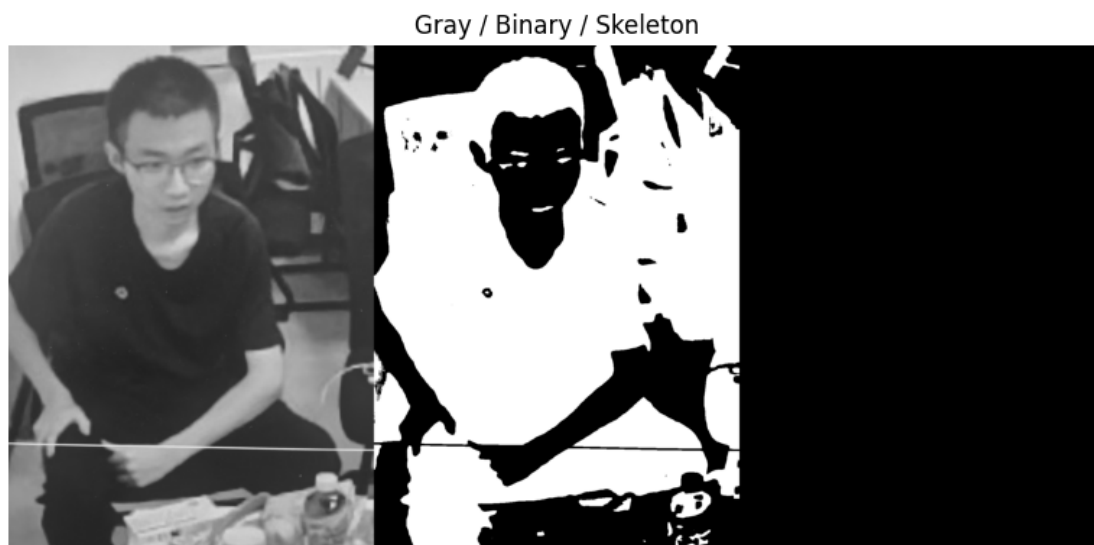
img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↳Machine Vision Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误或文件不存在')

#灰度转换与二值化
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.
↳THRESH_BINARY_INV + cv2.THRESH_OTSU)

binary_bool = img_as_bool(binary // 255)
skeleton = skeletonize(binary_bool)

skeleton_uint8 = skeleton.astype(np.uint8) * 255
kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (3,3))
skeleton_thick = cv2.dilate(skeleton_uint8, kernel,
↳iterations=1)

vis = np.hstack([gray, binary, skeleton_thick])
plt.figure(figsize=(12,4))
plt.imshow(vis, cmap='gray')
plt.title('Gray / Binary / Skeleton')
plt.axis('off')
plt.tight_layout()
plt.show()
```

|| 第十章

边缘检测操作

10 边缘检测操作

本章节分为基于Sobel算子、Canny算子的边缘检测操作。

Sobel算子

- 原理

Sobel算子基于一阶导数的近似，分别计算水平（ x 方向）与垂直（ y 方向）梯度，并合成梯度幅值：

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}, \quad |G| = \sqrt{G_x^2 + G_y^2}$$

- 特点

其具有简单、快速高效的优点，不过其对噪声敏感，且边缘较粗，定位精准性一般，可用于初步的边缘提取或搭配Canny算法共同使用（作为Canny算法的子步骤）。

- 代码实现

```
sobelx = cv2.Sobel(gray, cv2.CV_16S, 1, 0, ksize=3)
sobely = cv2.Sobel(gray, cv2.CV_16S, 0, 1, ksize=3)
sobel = cv2.magnitude(sobelx.astype(np.float32), sobely.astype(np.float32))
sobel = cv2.convertScaleAbs(sobel)
```

Canny算子

- 步骤

高斯滤波去噪 → 计算梯度幅值与方向 → 非极大值抑制（细化边缘） → 双阈值检测 → 边缘连接（滞后阈值）

- 特点

边缘细、连续、定位准确且抗噪能力强，但是对参数敏感。建议低阈值为高阈值的 $\frac{1}{2} - \frac{1}{3}$ 。基于以上特点，易于得知Canny算法比Sobel算法更具优势（或者说更具应用前景），其可以被用于目标检测、轮廓提取等，还可以用于图像分割的预处理以及工业缺陷的检测等多个领域。

- 代码实现

```
canny = cv2.Canny(gray, 50, 150)
```

```
#边缘检测
import cv2
```

```
import numpy as np
import matplotlib.pyplot as plt

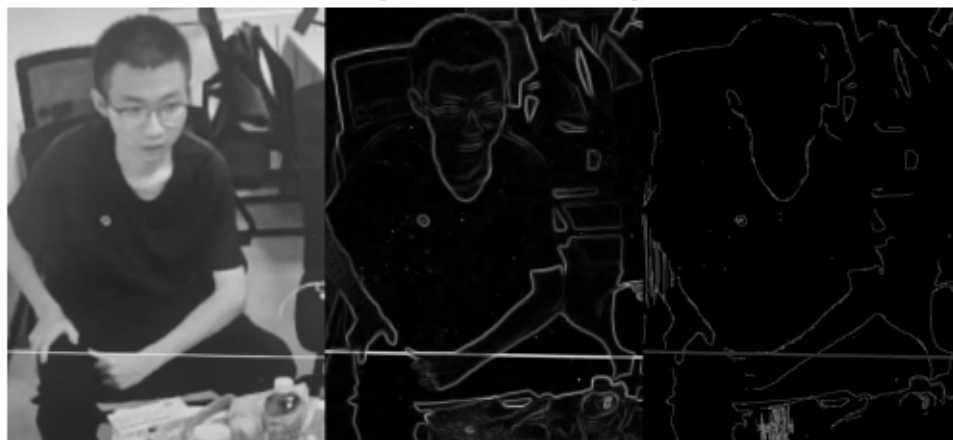
img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

#Sobel算子
sobelx = cv2.Sobel(gray, cv2.CV_16S, 1, 0, ksize=3) #水平
sobely = cv2.Sobel(gray, cv2.CV_16S, 0, 1, ksize=3) #垂直
sobel_f = cv2.magnitude(sobelx.astype(np.float32), sobely.
↪astype(np.float32)) #合成梯度幅值
sobel = cv2.convertScaleAbs(sobel_f) #转8位

#Canny算子
canny = cv2.Canny(gray, 10, 100)

vis = np.hstack([gray, sobel, canny])
vis = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
plt.imshow(vis)
plt.title('Gray / Sobel / Canny')
plt.axis('off')
```

Gray / Sobel / Canny



|| 第十一章

图像分割操作

11 图像分割操作

K-means聚类分割

- 原理

将像素颜色视为三维向量（RGB），在颜色空间中进行 K-means 聚为K类，最小化类内平方误差并形成区域。其目标函数：

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

- 特点

简单快速，易于实现，且无需额外标注，可用于任意维特征，但需预设K值，且对噪声敏感，易忽略空间关系，形成“椒盐”碎片。

- 代码实现

```
kmeans = KMeans(n_clusters=4, random_state=0).fit(data)
labels = kmeans.labels_.reshape(h, w)
```

Mean Shift分割

- 原理

基于核密度估计，密度梯度上升，自动迭代寻找局部密度最大值（模态），将像素归属到最近模态，形成自然区域。核函数：

$$K(x) = \exp\left(-\frac{\|x\|^2}{2\sigma^2}\right)$$

- 特点

无需预设聚类数K，边界自然且平滑，能发现任意形状区域，但计算量大，高维特征下收敛速度慢，需超像素加速，且带宽选择敏感。

- 代码实现

```
segments = slic(img_rgb, n_segments=500, compactness=10)
centers = np.array([data[segments == i].mean(axis=0) for i in np.unique(segments)
])
labels = pairwise_distances_argmin(data, centers).reshape(h, w)
```

#附加

```
import cv2, numpy as np, time
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```

from sklearn.metrics import pairwise_distances_argmin
from skimage.segmentation import slic, mark_boundaries
from skimage.filters import sobel

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 4/zxc.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('路径错误')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
h, w, c = img_rgb.shape
data = img_rgb.reshape(-1, 3)

#K-means
t0 = time.time()
kmeans = KMeans(n_clusters=4, random_state=0,
↪n_init='auto').fit(data)
kmeans_label = kmeans.labels_.reshape(h, w)
kmeans_seg = kmeans.cluster_centers_[kmeans_label].
↪astype(np.uint8)
tk = time.time() - t0

#Mean-Shift
t0 = time.time()
# 先用 SLIC 超像素加速 Mean-Shift, 否则 2K×1K 图会跑不动
segments = slic(img_rgb, n_segments=500, compactness=10,
↪start_label=0)
seg_centers = np.array([data[segments.reshape(-1) == i].
↪mean(axis=0)
for i in np.unique(segments)])
ms_labels = pairwise_distances_argmin(data, seg_centers).
↪reshape(h, w)
mean_shift_seg = seg_centers[ms_labels].astype(np.uint8)
tm = time.time() - t0

#评价指标函数

```



```

def boundary_smoothness(seg_gray):
    """沿分割边界的梯度均值，越小越平滑"""
    edge = sobel(seg_gray)
    return edge.mean()

def region_consistency(seg_lab, label_map):
    """各区域内部方差均值，越小越一致"""
    var = []
    for l in np.unique(label_map):
        mask = label_map == l
        var.append(seg_lab[mask].var(axis=0).mean())
    return np.mean(var)

def detail_preservation(seg_gray):
    """边缘像素占比，越高细节越多"""
    edge = sobel(seg_gray)
    return (edge > 0.05).mean()

#转灰度+计算
kmeans_gray = cv2.cvtColor(kmeans_seg, cv2.COLOR_RGB2GRAY)
ms_gray = cv2.cvtColor(mean_shift_seg, cv2.COLOR_RGB2GRAY)

k_smooth = boundary_smoothness(kmeans_gray)
ms_smooth = boundary_smoothness(ms_gray)
k_consist = region_consistency(kmeans_seg.astype(np.
→float32)/255., kmeans_label)
ms_consist = region_consistency(mean_shift_seg.astype(np.
→float32)/255., ms_labels)

k_detail = detail_preservation(kmeans_gray)
ms_detail = detail_preservation(ms_gray)

fig, ax = plt.subplots(2, 2, figsize=(10, 8))
ax[0, 0].imshow(img_rgb), ax[0, 0].set_title('Original'),
→ax[0, 0].axis('off')
ax[0, 1].imshow(mark_boundaries(img_rgb, kmeans_label,
→color=(1,0,0)))

```

```

        ax[0, 1].set_title(f'K-means  t={tk:.3f}s'), ax[0, 1].
↪axis('off')
        ax[1, 0].imshow(mark_boundaries(img_rgb, ms_labels,
↪color=(1,0,0)))
        ax[1, 0].set_title(f'Mean-Shift  t={tm:.3f}s'), ax[1, 0].
↪axis('off')
        ax[1, 1].axis('off')
        table = [['Target', 'K-means', 'Mean-Shift'],
        ['Boundary smoothing', f'{k_smooth:.4f}', f'{ms_smooth:.
↪4f}'],
        ['Regional Consistency', f'{k_consist:.4f}', f'{ms_consist:.
↪4f}'],
        ['Details Remain', f'{k_detail:.4f}', f'{ms_detail:.4f}'],
        ['Times', f'{tk:.3f}s', f'{tm:.3f}s']]
        ax[1, 1].table(cellText=table[1:], colLabels=table[0],
↪loc='center')
        plt.tight_layout()
        plt.show()

```

```

/Users/guo2006/myenv/lib/python3.11/site-packages/sklearn/utils/
↪extmath.py:203:
RuntimeWarning: divide by zero encountered in matmul
ret = a @ b
/Users/guo2006/myenv/lib/python3.11/site-packages/sklearn/utils/
↪extmath.py:203:
RuntimeWarning: overflow encountered in matmul
ret = a @ b
/Users/guo2006/myenv/lib/python3.11/site-packages/sklearn/utils/
↪extmath.py:203:
RuntimeWarning: invalid value encountered in matmul
ret = a @ b
/Users/guo2006/myenv/lib/python3.11/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by
↪zero
encountered in matmul
current_pot = closest_dist_sq @ sample_weight

```

```

/Users/guo2006/myenv/lib/python3.11/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow
↪ encountered in
    matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/guo2006/myenv/lib/python3.11/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid
↪ value
    encountered in matmul
    current_pot = closest_dist_sq @ sample_weight

```

Original



K-means t=0.044s



Mean-Shift t=0.303s



Target	Kmeans	Mean-Shift
Boundary smoothing	0.0089	0.0125
Regional Consistency	0.0000	0.0000
Details Remain	0.0655	0.0684
Times	0.044s	0.303s

|| 第十二章

张正有标定法

12 张正有标定法

张正友（Zhang Zhengyou）标定法，是 1998 年由微软研究院张正友提出的单平面棋盘格相机标定法，也是目前计算机视觉领域使用最广泛的相机内参/外参标定技术。其可以仅用一张打印的棋盘格，在相机前任意摆几个姿势，就能高精度地求出相机内参、外参和畸变系数。但所打印的棋盘格必须全部暴露在相机视角下。

```
#张正友标定
import cv2
import numpy as np
import glob
import os
import matplotlib.pyplot as plt

#对象点
chessboard_size = (5, 8) # 内角点数量（列，行）
square_size = 27.0
#实际标定使用的真实棋格的真实尺寸27mm

objp = np.zeros((chessboard_size[0] * chessboard_size[1], 3), np.float32)

objp[:, :2] = np.mgrid[0:chessboard_size[0], 0:
↪chessboard_size[1]].T.reshape(-1, 2) * square_size
objp[:, :2] *= square_size

#存储对象点和图像点的数组
objpoints = [] # 真实世界中的3D点
imgpoints = [] # 图像中的2D点

#读取所有标定图像
folder = r'/Users/guo2006/myenv/Machine Vision Experiment/
↪Machine Vision Experiment 5/biaoding'

images = glob.glob(os.path.join(folder, '*.jpg')) \
+ glob.glob(os.path.join(folder, '*.png')) # 两种后缀都读取

if len(images) == 0:
print("未找到标定图像，请检查路径是否正确")
```

```
exit()

print(f"找到 {len(images)} 张标定图像")

#遍历每张图像进行角点检测
for fname in images:
    img = cv2.imread(fname)
    if img is None:
        print(f"无法读取图像: {fname}")
        continue

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # 查找棋格角点
    ret, corners = cv2.findChessboardCorners(gray,
↪ chessboard_size, None)

    if ret:
        print(f"在 {fname} 中找到角点")
        objpoints.append(objp)

        # 提高角点检测精度
        corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1,
↪ -1),

        criteria=(cv2.TERM_CRITERIA_EPS + cv2.
↪ TERM_CRITERIA_MAX_ITER, 30, 0.001))
        imgpoints.append(corners2)

        # 可视化角点
        # img = cv2.drawChessboardCorners(img, chessboard_size,
↪ corners2, ret)

        # plt.imshow(img)
        # plt.title('Corners')
    else:
        print(f"在 {fname} 中未找到角点")
```

```
#检查是否找到足够的图像进行标定
if len(objpoints) < 3:
    print("找到的可用图像太少，至少需要3张图像")
    exit()

print(f"使用 {len(objpoints)} 张图像进行标定")

#执行相机标定
print("开始相机标定...")
ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
    objpoints, imgpoints, gray.shape[::-1], None, None
)

print("\n标定结果:")
print(f"重投影误差: {ret}")
print("相机内参矩阵:")
print(mtx)
print("畸变系数:")
print(dist)

#标定结果验证 - 计算重投影误差
mean_error = 0
for i in range(len(objpoints)):
    imgpoints2, _ = cv2.projectPoints(objpoints[i], rvecs[i],
    ↪tvecs[i], mtx, dist)
    error = cv2.norm(imgpoints[i], imgpoints2, cv2.NORM_L2) /
    ↪len(imgpoints2)
    mean_error += error

print(f"\n平均重投影误差: {mean_error / len(objpoints)} 像
素")

#保存标定结果
np.savez('camera_calibration.npz', mtx=mtx, dist=dist,
    ↪rvecs=rvecs, tvecs=tvecs)
```

```

print("\n标定结果已保存到 camera_calibration.npz")

#使用标定结果进行畸变校正
if len(images) > 0:
    # 使用第一张图像进行演示
    img = cv2.imread(images[0])
    h, w = img.shape[:2]

    # 优化相机矩阵
    newcameramtx, roi = cv2.getOptimalNewCameraMatrix(mtx,
→dist, (w, h), 1, (w, h))

    # 校正图像
    dst = cv2.undistort(img, mtx, dist, None, newcameramtx)

    # 裁剪图像
    x, y, w, h = roi
    dst = dst[y:y+h, x:x+w]

    # 显示结果
    fig, ax = plt.subplots(1, 2, figsize=(10,8))
    ax[0].imshow(img), ax[0].set_title('Original Image'), ax[0].
→axis('off')

    ax[1].imshow(dst), ax[1].set_title('Undistorted Image'),
→ax[1].axis('off')

    # ----- 带角点的畸变校正 -----
    if len(images) > 0:
        img = cv2.imread(images[0])
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        h, w = img.shape[:2]

        # (1) 先在校正前图上重新检角点并画上去
        ret, corners = cv2.findChessboardCorners(gray,
→chessboard_size,
        cv2.CALIB_CB_ADAPTIVE_THRESH +

```



```

cv2.CALIB_CB_NORMALIZE_IMAGE)
if ret:                                # 如果找到就画
    corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1,
↪-1),

    (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.
↪001))

    cv2.drawChessboardCorners(img, chessboard_size, corners,
↪True)

    # (2) 再去畸变
    newcameramtx, roi = cv2.getOptimalNewCameraMatrix(mtx,
↪dist, (w, h), 1, (w, h))
    dst = cv2.undistort(img, mtx, dist, None, newcameramtx)

    # (3) 裁剪黑边 (可选)
    x, y, w, h = roi
    dst = dst[y:y+h, x:x+w]

    # (4) 显示 & 保存
    fig, ax = plt.subplots(1, 2, figsize=(10,8))
    ax[0].imshow(img), ax[0].set_title('Original Image'), ax[0].
↪axis('off')

    ax[1].imshow(dst), ax[1].set_title('Undistorted Image'),
↪ax[1].axis('off')

    # 生成未校正 & 校正两张图 (同尺寸, 不裁剪)
    img_raw = cv2.imread(images[0])
    h, w = img_raw.shape[:2]
    map1, map2 = cv2.initUndistortRectifyMap(mtx, dist, None,
↪mtx, (w, h), cv2.CV_16SC2)
    img_undist = cv2.remap(img_raw, map1, map2, cv2.
↪INTER_LINEAR)

    # 计算差异并增强对比
    diff = cv2.absdiff(img_raw, img_undist)

```

```
diff = cv2.convertScaleAbs(diff, alpha=1.5, beta=0) # 放大 5 倍

plt.imshow(diff)
plt.title('diff img')

print("\n标定完成!")
```

找到 20 张标定图像

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision

↪Experiment

5/biaoding/8.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision

↪Experiment

5/biaoding/9.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision

↪Experiment

5/biaoding/14.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision

↪Experiment

5/biaoding/15.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision

↪Experiment

5/biaoding/17.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision

↪Experiment

5/biaoding/16.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision

↪Experiment

5/biaoding/12.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision

↪Experiment

5/biaoding/13.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision

↪Experiment

5/biaoding/11.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/10.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/20.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/18.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/19.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/4.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/5.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/7.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/6.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/2.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/3.jpg 中找到角点

在 /Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_

↪Experiment

5/biaoding/1.jpg 中找到角点

使用 20 张图像进行标定

开始相机标定...

标定结果:

重投影误差: 8.203850256183376

相机内参矩阵:

$\begin{bmatrix} 6.24862606e+03 & 0.00000000e+00 & 4.93473537e+02 \\ 0.00000000e+00 & 1.82304138e+04 & 9.05418068e+02 \\ 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix}$

$\begin{bmatrix} 0.00000000e+00 & 1.82304138e+04 & 9.05418068e+02 \\ 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix}$

$\begin{bmatrix} 0.00000000e+00 & 0.00000000e+00 & 1.00000000e+00 \end{bmatrix}$

畸变系数:

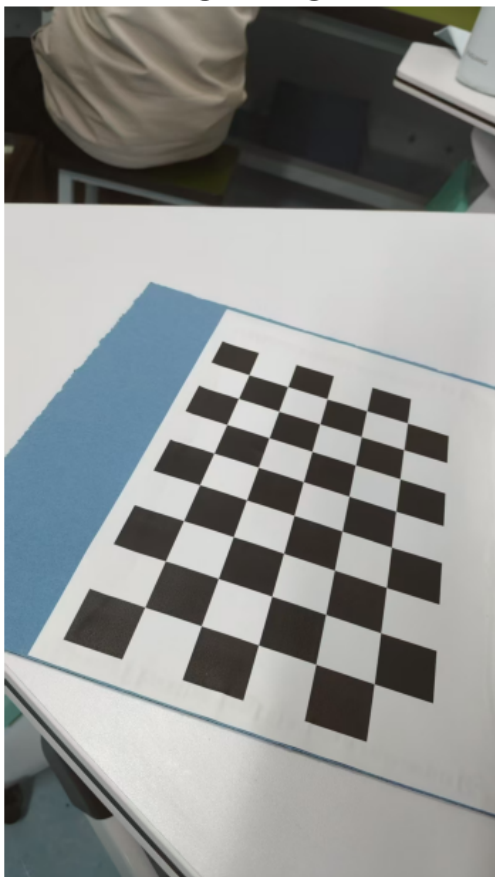
$\begin{bmatrix} -4.27055210e-01 & 6.06555828e+00 & -6.31486059e-03 & -4.73005413e-02 \\ -2.35297951e+01 \end{bmatrix}$

平均重投影误差: 1.0953467595808408 像素

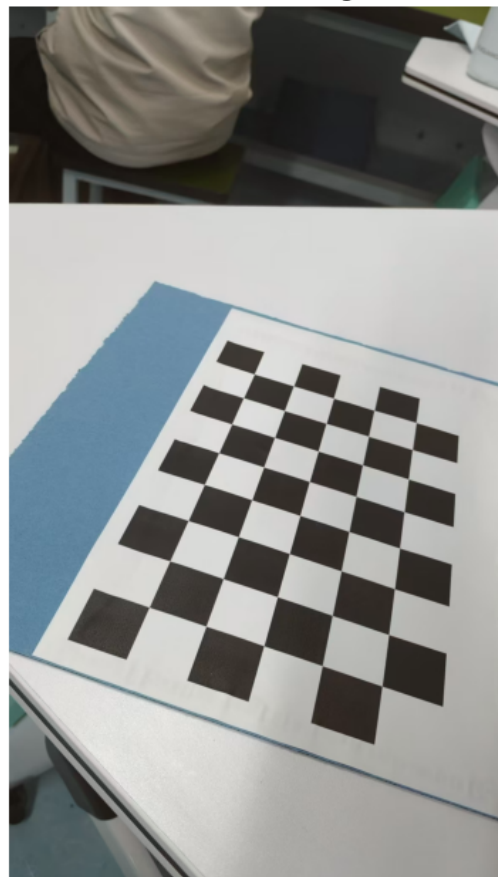
标定结果已保存到 camera_calibration.npz

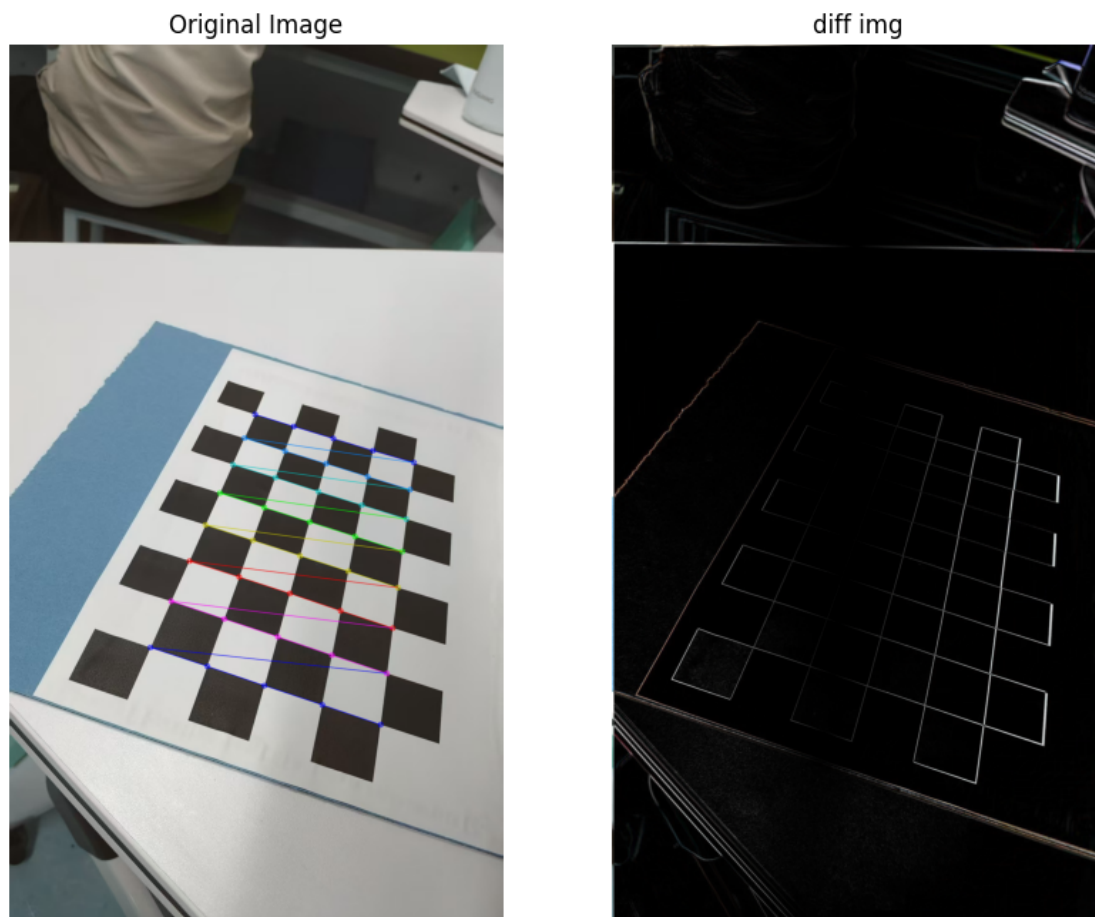
标定完成!

Original Image



Undistorted Image





在“diff img”图中，白色部分代表畸形矫正后与畸形矫正前的变化。

|| 第十三章

SIFT算法

13 SIFT算法

尺度不变特征转换(Scale-invariant feature transform 或 SIFT)是一种机器视觉的算法用来侦测与描述影像中的局部性特征,它在空间尺度中寻找极值点,并提取出其位置、尺度、旋转不变数,此算法由 David Lowe 在1999年所发表,2004年完善总结。

图
图

请检查路径!"

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 读图
img1 = cv2.imread('Ang1.jpg', cv2.IMREAD_GRAYSCALE) # 查询
img2 = cv2.imread('Ang2.jpg', cv2.IMREAD_GRAYSCALE) # 训练

assert img1 is not None and img2 is not None, "无法读取图像,

# 创建 SIFT 检测器
sift = cv2.SIFT_create()

# 检测关键点和描述子
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
print(f"图1关键点: {len(kp1)} 个, 图2关键点: {len(kp2)} 个")

# 暴力匹配 + 比值测试
bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)
matches = bf.knnMatch(des1, des2, k=2)

good = []
for m, n in matches:
    # Lowe 比值测试: 最近距离 < 0.7*次近距离
    if m.distance < 0.7 * n.distance:
        good.append(m)
print("良好匹配对数: ", len(good))
```

```
# 可视化
img_match = cv2.drawMatches(
img1, kp1, img2, kp2, good, None,
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

plt.figure(figsize=(12, 6))
plt.imshow(img_match[:, :, ::-1]) # BGR→RGB
plt.title(f"SIFT matches (After Ratio Test): {len(good)}  
→Points")

plt.axis('off')
plt.tight_layout()
plt.show()
```

图1关键点：5376 个，图2关键点：7051 个
良好匹配对数： 101

SIFT matches (After Ratio Test): 101 Points

